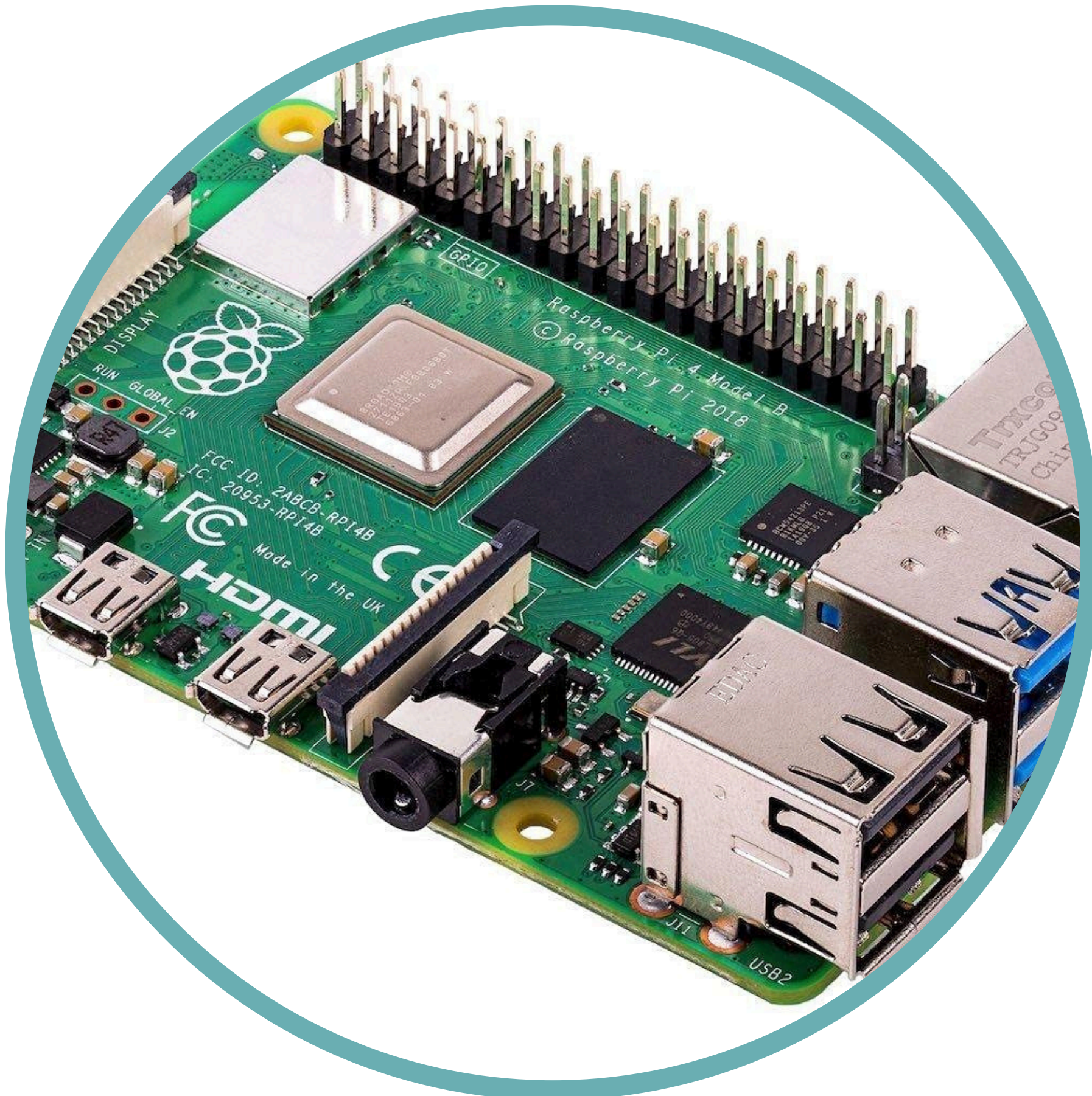


Du 15/09 au 29/09

Rapport de laboratoire

Enseignant laboratoire : Benkreira Hanchour  
Omar  
Enseignant cours : Lokman Sboui



Matys Benoist  
Programme court de 1er cycle 2025-2026

**ÉTS**

Le génie pour l'industrie

**ÉCOLE DE  
TECHNOLOGIE  
SUPÉRIEURE**

Université du Québec

**PIRANHAS  
ÉTS**



# Table des matières

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>Table des matières</b>                         | <b>2</b>  |
| <b>Buts et objectifs du laboratoire</b>           | <b>4</b>  |
| <b>Démarche de conception</b>                     | <b>5</b>  |
| <b>Clignotement de deux DEL</b>                   | <b>5</b>  |
| <b>Clignotement de deux DEL avec deux threads</b> | <b>6</b>  |
| <b>Lecture de la température sur le MAX31855</b>  | <b>7</b>  |
| <b>Commande de la lampe avec un signal PWM</b>    | <b>9</b>  |
| <b>La commande de type thermostat</b>             | <b>10</b> |
| <b>La commande de type PID</b>                    | <b>11</b> |
| <b>Conclusion</b>                                 | <b>13</b> |
| <b>Expérience personnelle</b>                     | <b>13</b> |

# Table des figures

|                                                                                     |    |
|-------------------------------------------------------------------------------------|----|
| Figure 1 : Organigramme du programme de clignotement de deux DEL                    | 5  |
| Figure 2 : Organigramme du programme de clignotement de deux DEL avec deux threads. | 6  |
| Figure 3 : Organigramme du code de la lecture de la température avec le MAX31855    | 7  |
| Figure 4 : Organigramme du code du contrôle de la lampe avec un signal PWM          | 9  |
| Figure 5 : Organigramme du code de la commande de type thermostat de la lampe       | 10 |
| Figure 6 : Organigramme du code de la commande de type PID de la lampe              | 12 |

# Buts et objectifs du laboratoire

Le but de ce laboratoire est de prendre en main la programmation avec la raspberry pi 3 B et de se familiariser avec le matériel présent dans les salles de laboratoire. Cela correspond à l'utilisation de Geany pour l'écriture, la compilation et le test des programmes de langage C, la compréhension et l'utilisation des différentes broches GPIO de la raspberry et la programmation de courts programmes sous la forme de thread permettant de faire clignoter des LEDS en opposition de phase à une certaine fréquence, et d'arrêter le programme avec la pression de boutons, de relever la température d'une lampe à l'aide du protocole de communication SPI et de réaliser l'asservissement de la température d'une lampe avec deux types de commandes.

L'objectif de ce premier laboratoire est de concevoir une première couche de perception qui va être dans ce premier laboratoire l'asservissement de la température d'une lampe avec un capteur de température en utilisant le thermocouple de la lampe.

Enfin le dernier objectif de ce laboratoire est d'implanter deux types d'asservissements différents avec, dans un premier temps une commande de type ON/OFF et, dans un deuxième temps une commande de type PID.

Pour ce faire, une lecture du thermocouple de la lampe via le protocole SPI est nécessaire. Nous devons donc nous familiariser avec ce protocole et nous devons l'implémenter de manière locale afin de pouvoir lire le thermocouple de la lampe à l'aide de la raspberry pi.

Ensuite, une configuration de la lampe avec une PWM est nécessaire pour contrôler l'intensité avec laquelle la lampe est allumée et ainsi contrôler sa température.

Enfin l'implémentation des deux types d'asservissement est nécessaire pour contrôler la température de la lampe avec un certain temps de réaction.

# Démarche de conception

## Clignotement de deux DEL

Durant cette manipulation, nous devons faire clignoter deux DEL en opposition de phase à une fréquence de 1 Hz. Le but de cette manipulation est de nous familiariser avec le logiciel Geany. Le listing donné nous permet de faire clignoter une seule DEL à une fréquence de 1 Hz. Nous devons l'adapter pour faire clignoter deux DEL en opposition de phase à 1 Hz.

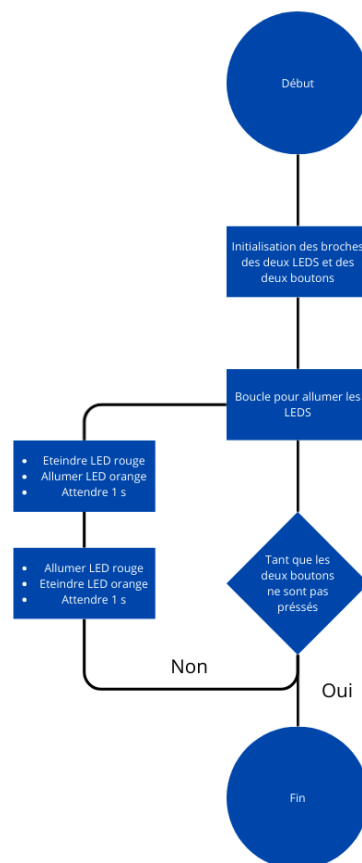


Figure 1 : Organigramme du programme de clignotement de deux DEL

D'après la Figure 1, dans le programme, nous commençons par configurer les broches des deux boutons et des deux DEL. Puis nous rentrons dans une boucle "while" avec comme condition : les deux boutons ne sont pas pressés. Si la condition est respectée, nous allumons une DEL et éteignons l'autre. Nous attendons un délai de 1s puis éteignons la première DEL et allumons l'autre. Dans le cas contraire, nous libérons les broches et arrêtons le programme avec une valeur de retour.

Nous observons qu'avec ce programme le temps de réaction pour arrêter le programme est trop élevé. Une solution pour réduire ce temps de programme et ainsi arrêter le programme lorsque nous le souhaitons, est de passer par des threads.

## Clignotement de deux DEL avec deux threads

Durant cette manipulation, nous devons faire clignoter deux DEL en opposition de phase à une fréquence de 1 Hz en utilisant deux threads. Le but de cette manipulation est de nous familiariser avec la réalisation de courts programmes sous la forme de threads. Le listing donné nous permettait de faire clignoter une seule DEL à une fréquence de 1 Hz, et nous devons l'adapter pour faire clignoter deux DEL en opposition de phase à 1 Hz avec deux threads afin d'aboutir à un arrêt du programme davantage réactif.

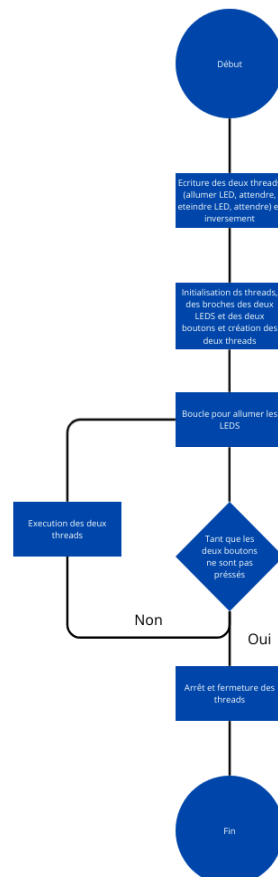


Figure 2 : Organigramme du programme de clignotement de deux DEL avec deux threads.

D'après la Figure 2, dans le programme, nous commençons par écrire le contenu des deux threads, c'est-à-dire : allumer une DEL ou l'autre, attendre une seconde puis l'éteindre et attendre de nouveau une seconde. Ensuite, nous créons les threads, puis, nous configurons les broches des deux boutons et des deux DEL. Après cela, nous associons les contenus des deux threads aux deux threads créés. Enfin, nous rentrons dans une boucle "while" avec comme condition : les deux boutons ne sont pas pressés. Si cette condition est bien respectée, nous allumons une DEL et éteignons l'autre. Nous attendons un délai de 1s puis éteignons la première DEL et allumons l'autre. Dans le cas contraire, nous libérons les broches et arrêtons le programme avec une valeur de retour.

Nous observons que le système est plus réactif, il met fin au programme dès lors que nous pressons les deux boutons.

## Lecture de la température sur le MAX31855

Durant cette manipulation, nous devons lire la valeur de la température d'une lampe grâce au capteur MAX31855 en suivant le protocole de communication SPI et en convertissant la valeur analogique reçue en valeur numérique. Le but de cette manipulation est de nous familiariser avec le matériel de laboratoire, avec le protocole de communication SPI et avec la conversion d'une donnée analogique en donnée numérique. Un organigramme nous a été fourni pour construire le code.

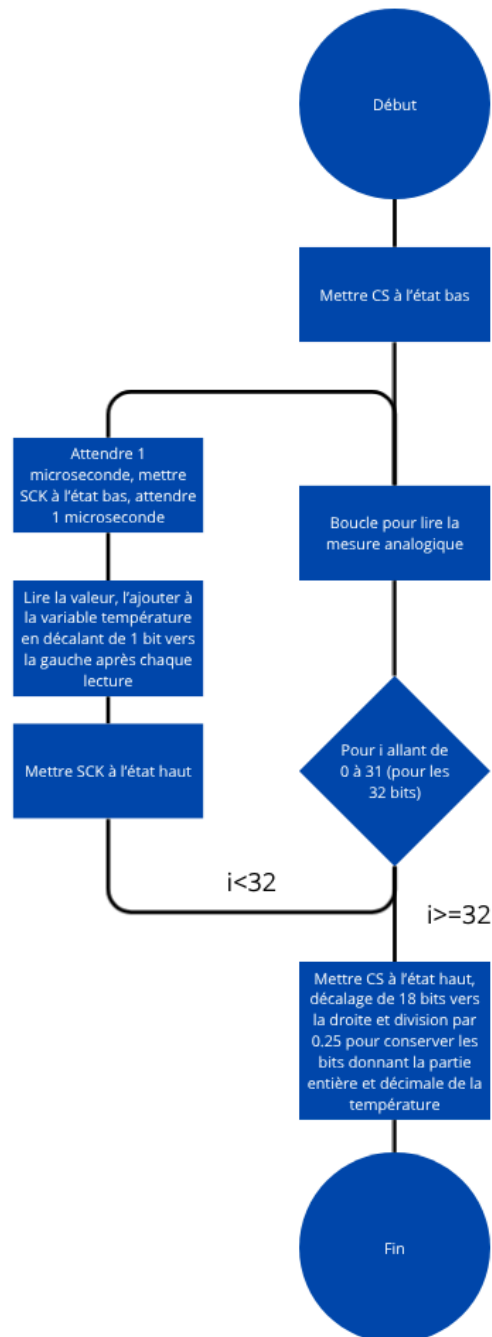


Figure 3 : Organigramme du code de la lecture de la température avec le MAX31855

D'après la Figure 3, dans cette fonction, nous commençons par créer une variable de type double, qui sera la température renvoyée, et une variable de type unsigned int sur 32 bits qui sera la température relevée. Nous mettons CS à l'état bas pour permettre la lecture des bits. Ensuite, nous créons une boucle pour i allant de 0 à 31 pour les 32 bits à lire. A chaque fois, nous mettons SCK à l'état haut, nous lisons la valeur du bit relevé en décalant de 1 bit vers la gauche la dernière valeur relevée. Puis nous attendons une milliseconde, nous remettons SCK à l'état bas et nous attendons de nouveau une milliseconde. A la fin de la boucle, nous mettons CS à l'état haut, nous intégrons la variable de la température relevée décalée de 18 bits vers la droite et multipliée par 0.25 à la variable de retour. Cela permet de ne conserver que les bits de température. Enfin, nous retournons la valeur calculée. Nous exécutons cette fonction dans le thread créé pour l'allumage et l'extinction d'une DEL, nous l'appelons entre l'allumage et l'extinction de la DEL tant que les deux boutons poussoirs ne sont pas pressés. Dans le cas contraire, nous arrêtons les threads, nous libérons les broches, nous éteignons les DEL et arrêtons le programme avec une valeur de retour. Ce qui diffère dans le main, est la configuration des broches pour le protocole SPI après la configuration des broches des boutons poussoirs.

La difficulté dans cette partie était de réaliser de manière locale la communication avec le protocole SPI. De plus, il fallait correctement incrémenter la valeur de la température relevée pour chaque bit de donnée reçue. Enfin, il fallait retourner la valeur correcte en ne gardant que les bits dédiés à la valeur de la température relevée.

## Commande de la lampe avec un signal PWM

Durant cette manipulation, nous devons allumer la lampe avec une certaine intensité. Pour ce faire, nous utilisons une PWM. Le but de cette manipulation est de se familiariser avec la PWM, ses paramètres et son duty cycle. Le listing fourni nous permettait de configurer la PWM.

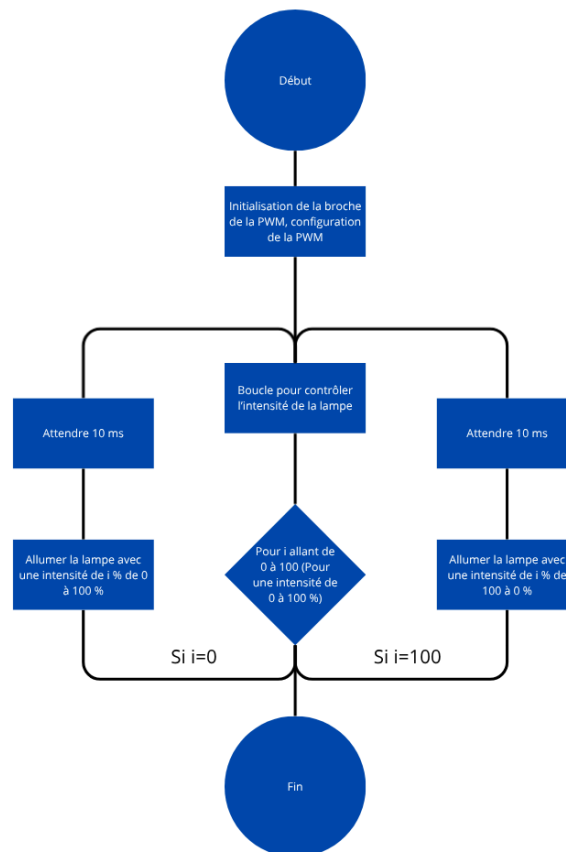


Figure 4 : Organigramme du code du contrôle de la lampe avec un signal PWM

D'après la Figure 4, dans cette fonction, nous commençons par créer deux variables de type int (étant les paramètres de configuration de la PWM) et une variable de type int (permettant d'incrémenter ou de décrémenter l'intensité de la lampe). Ensuite, nous configurons la broche de la PWM et nous configurons la PWM. Si nous sommes à 0% de l'intensité de la lampe, pour i allant de 0 à 100, nous allumons la lampe à i %. Nous attendons un délai de 10 microsecondes et, si i atteint 100 nous changeons la valeur de la variable et effectuons les mêmes tâches mais pour i allant de 100 à 0. Nous exécutons cette fonction dans le thread créé pour l'allumage et l'extinction d'une DEL, entre l'allumage et l'extinction de la DEL (tant que les deux boutons poussoirs ne sont pas pressés). Dans le cas contraire, nous arrêtons les threads, nous libérons les broches, nous éteignons les DEL et arrêtons le programme avec une valeur de retour. Ce qui diffère dans le main, est la libération de la PWM avant la libération du thread.

La difficulté dans cette partie était de correctement configurer la PWM et de choisir les bons paramètres de range et de diviseur pour que la lampe puisse s'allumer avec le bon range cycle. Les valeurs choisies permettaient une bonne division de l'intensité de la lampe.

## La commande de type thermostat

Durant cette manipulation, nous devons commander une lampe avec une commande de type Thermostat donc 100 % ou 0 %. Le but de cette manipulation est de réaliser un premier asservissement de la température de la lampe.

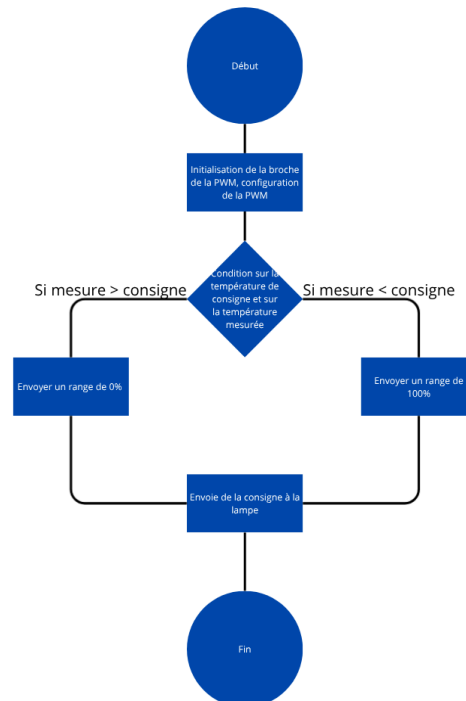


Figure 5 : Organigramme du code de la commande de type thermostat de la lampe

D'après la Figure 5, dans cette fonction qui utilise comme paramètres la température mesurée et la température de consigne de type double, si la température mesurée est inférieure à la température de consigne, nous envoyons un rang correspondant à 100% de l'intensité de la lampe sinon, nous envoyons un rang correspondant à 0% de l'intensité de la lampe. Nous utilisons cette fonction à l'intérieur d'une autre fonction (permettant d'allumer la lampe à une certaine intensité selon le rang envoyé par la première fonction) après qu'elle ait configuré la PWM. Enfin, cette deuxième fonction est appelée dans le thread créé pour l'allumage et l'extinction d'une DEL entre l'allumage et l'extinction de la DEL (tant que les deux boutons poussoirs ne sont pas pressés). Dans le cas contraire, nous arrêtons les threads, nous libérons les broches, nous éteignons les DEL et arrêtons le programme avec une valeur de retour.

La limite de cette commande est sa réaction, en effet, la fonction va réagir trop brutalement et s'éloigne de la température de consigne dès lors qu'elle se retrouve en dessous de la température de commande. De plus, elle met du temps à réagir à partir du moment où nous dépassons la commande. Ainsi, le système a un fort dépassement, il va avoir un temps de réponse trop élevé et ne se stabilise pas. Pour améliorer cette commande et avoir un meilleur asservissement de la température de la lampe, nous pouvons remplacer cette commande par une commande de type PID.

## La commande de type PID

Durant cette manipulation, nous devons commander la température de la lampe avec une commande de type PID. Le but de cette manipulation est de réaliser un second asservissement de la température de la lampe avec un temps de réponse plus faible, une stabilité meilleure et plus rapide. De plus, le but est de nous familiariser avec le PID, ses gains et son mode de fonctionnement.

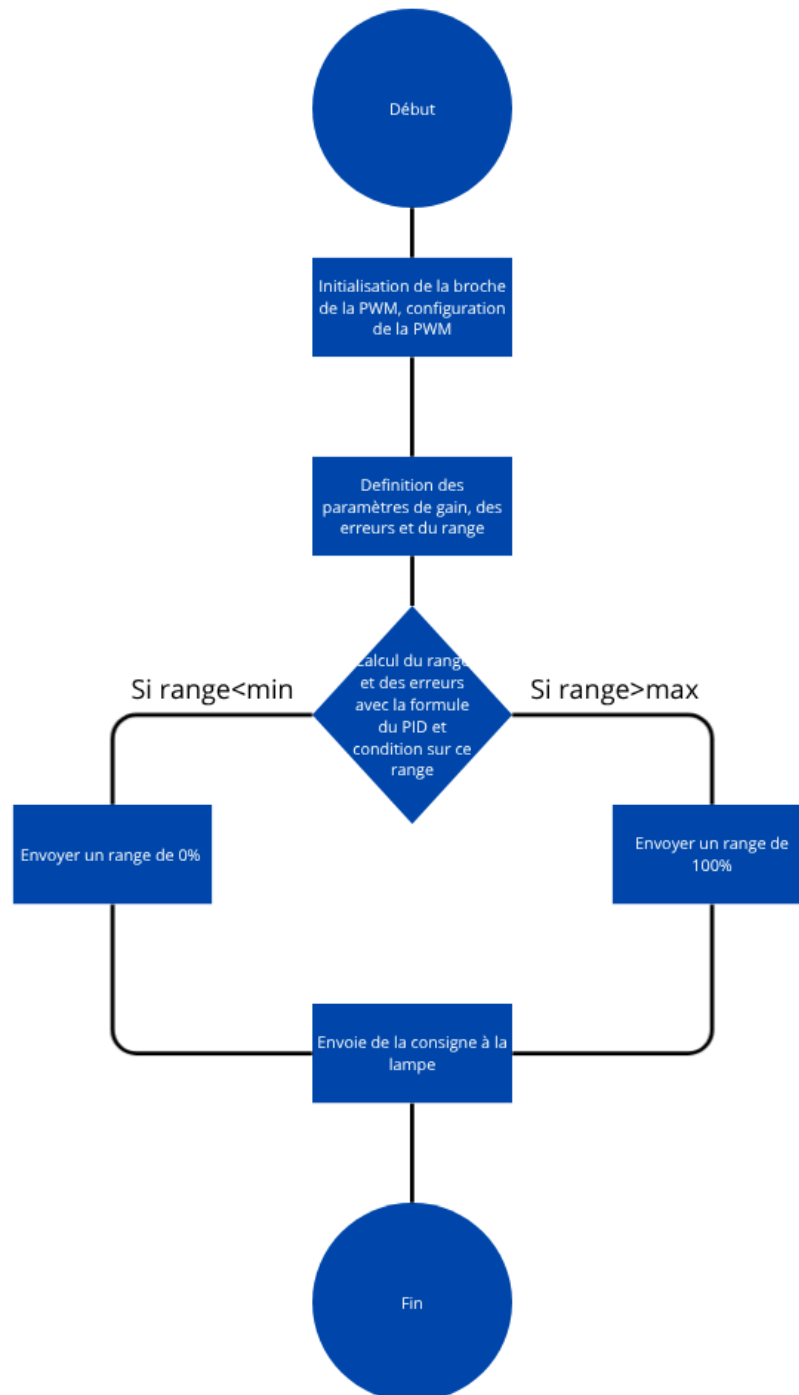


Figure 6 : Organigramme du code de la commande de type PID de la lampe

D'après la Figure 6, dans cette fonction (prenant en paramètres la température mesurée, la température de consigne et les trois gains  $K_p$ ,  $K_d$  et  $K_i$  de type double). Nous créons une variable de type `int` permettant une initialisation des paramètres d'erreurs et de retour au premier appel de la fonction. Ceux-ci sont static pour permettre une conservation de leurs valeurs après différents appels de la fonction. Nous calculons la valeur de retour avec les paramètres d'erreur et de retour précédent. Si la valeur calculée est inférieure au rang permettant une intensité de 0 % de la lampe, alors nous renvoyons un rang correspondant à une intensité de 0 % de la lampe. En revanche, si la valeur calculée est supérieure au rang permettant une intensité de 100 % de la lampe, alors nous renvoyons un rang correspondant à une intensité de 100 % de la lampe. Sinon nous renvoyons la valeur de rang calculé correspondant à une certaine intensité de la lampe. Nous utilisons cette fonction à l'intérieur d'une autre fonction (permettant d'allumer la lampe à une certaine intensité selon le rang envoyé par la première fonction) après qu'elle ait configuré la PWM. Enfin, cette deuxième fonction est appelée dans le thread créé pour l'allumage et l'extinction d'une DEL entre l'allumage et l'extinction de la DEL (tant que les deux boutons poussoirs ne sont pas pressés). Dans le cas contraire, nous arrêtons les threads, nous libérons les broches, nous éteignons les DEL et arrêtons le programme avec une valeur de retour.

La difficulté dans cette partie était de définir les bons gains pour avoir un système ne dépassant pas trop la consigne tout en conservant un temps de réponse assez faible et se stabilisant rapidement. Les valeurs de gain choisies permettaient de répondre à ces attentes. De plus, il fallait calculer les bonnes erreurs à chaque fois et calculer le rang adapté en conséquence.

Nous pouvons en conclure que ce deuxième système ne dépasse pas trop la commande, se stabilise assez rapidement et a un temps de réponse assez faible. Donc ce deuxième type de commande est plus adapté pour l'asservissement en température d'une lampe.

# Conclusion

Ce laboratoire nous aura permis de prendre en main le logiciel Geany, pour écrire, compiler et tester les programmes écrits en langage C. Nous avons appris à configurer des broches et à écrire des programmes sous la forme de threads.

De plus, nous avons appris à réaliser une première couche de perception avec l'asservissement de la température d'une lampe en utilisant une PWM qui agit sur l'intensité de la lampe à l'aide de deux types de commande : une commande de type thermostat et une commande de type PID.

Enfin, nous avons appris à lire la température d'une lampe à l'aide du capteur MAX31855 et à convertir les données analogiques reçues en données numériques en utilisant le protocole de communication SPI.

## Expérience personnelle

Je pense que ce laboratoire est bien agencé. Il permet de mieux appréhender les bases de la programmation en C et d'en comprendre le fonctionnement avec la raspberry pi au début du laboratoire.

De plus, celui-ci permet d'approfondir nos connaissances en programmation tout en implémentant une communication en suivant un protocole défini. Il m'a permis de découvrir la lecture d'une donnée analogique à l'aide d'un capteur et d'apprendre à la convertir en donnée numérique.

Ce laboratoire m'a également permis de revoir les connaissances sur les signaux PWM pour bien les configurer et de revoir les commandes de type PID.

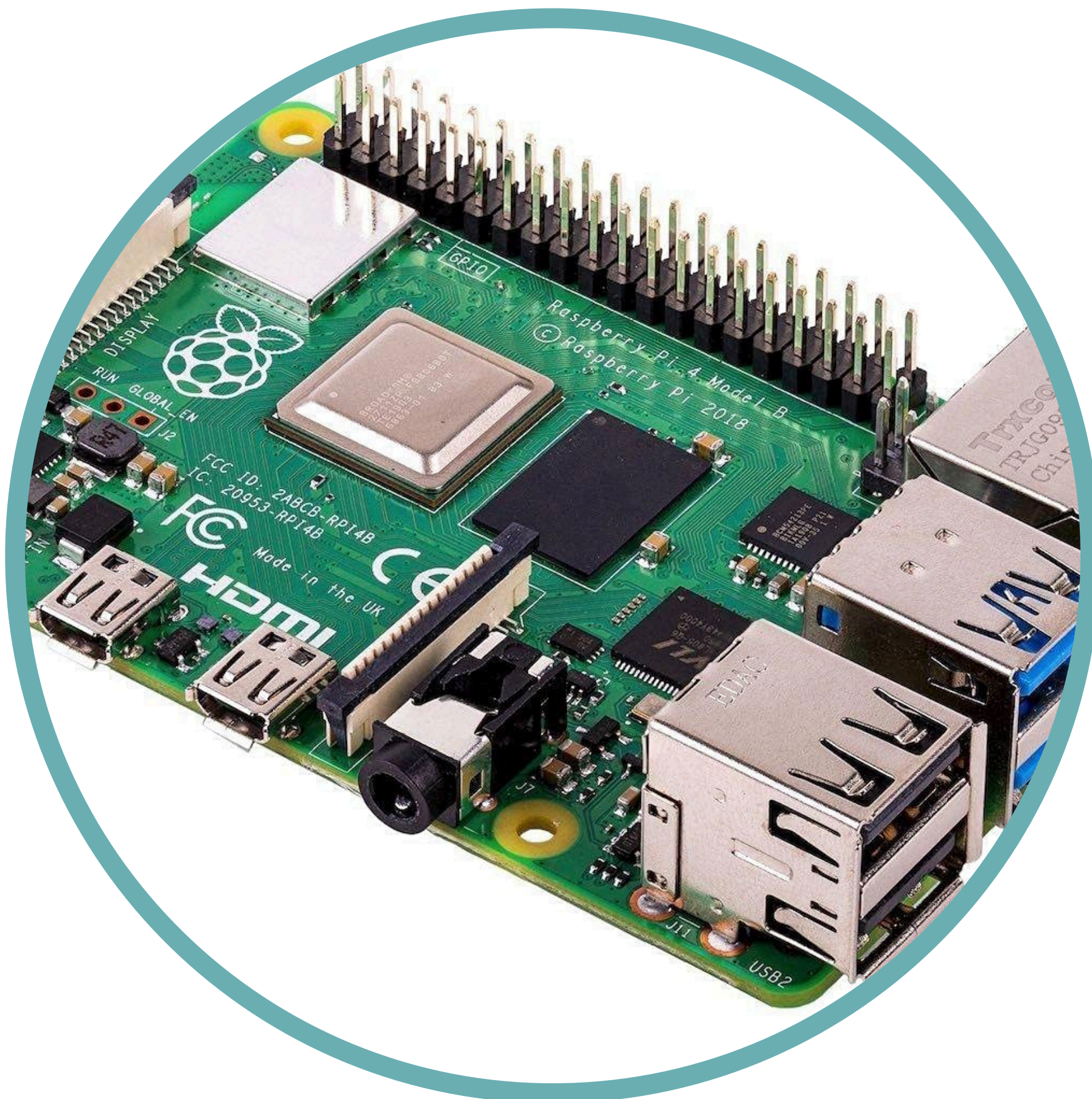
Enfin, ce laboratoire m'a donné la possibilité de mobiliser toutes ces connaissances, de les lier entre elles et de les utiliser pour mener à bien le laboratoire.

En conclusion, je dirai que ce laboratoire est un pertinent pour montrer l'intérêt des connaissances acquises précédemment et les utiliser dans un projet concret. Selon moi, c'est un bon premier laboratoire pour se mettre à niveau et revoir certains éléments.

Du 05/10 au 19/10

Rapport de laboratoire

Enseignant laboratoire : Bitouh Amar  
Enseignant cours : Lokman Sboui



Matys Benoist  
Programme court de 1er cycle 2025-2026

**ÉTS**

Le génie pour l'industrie

**ÉCOLE DE  
TECHNOLOGIE  
SUPÉRIEURE**

Université du Québec

**PIRANHAS  
ÉTS**



# Table des matières

|                                                                                    |           |
|------------------------------------------------------------------------------------|-----------|
| <b>Table des matières</b>                                                          | <b>2</b>  |
| <b>Table des figures</b>                                                           | <b>3</b>  |
| <b>Buts et objectifs du laboratoire</b>                                            | <b>4</b>  |
| <b>Démarche de conception</b>                                                      | <b>5</b>  |
| <b>Sauvegarde locale des données</b>                                               | <b>5</b>  |
| Premier fichier CSV sur 3 minutes                                                  | 5         |
| Second fichier CSV sur 10 minutes                                                  | 7         |
| <b>Mosquitto et le MQTT</b>                                                        | <b>9</b>  |
| Premier programme Publieur et Abonné sur 1 topic                                   | 9         |
| Second programme Publieur et Abonné sur 5 topics                                   | 12        |
| <b>Mosquitto sur MQTT Dash</b>                                                     | <b>15</b> |
| Troisième programme en ajoutant un bouton ON/OFF et une consigne sur l'application | 15        |
| <b>Notifications sur Android automate</b>                                          | <b>18</b> |
| <b>Utilisation de Node-RED</b>                                                     | <b>19</b> |
| <b>Conclusion</b>                                                                  | <b>20</b> |
| <b>Expérience personnelle</b>                                                      | <b>20</b> |

## Table des figures

|                                                                                                                                                                                           |    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 1 : Organigramme du programme de création de fichier CSV pendant 3 minutes                                                                                                         | 5  |
| Figure 2 : Graphique de l'évolution de la température de la lampe en fonction du nombre de mesures (1 mesure = 2 secondes) pendant 3 minutes avec un signal PWM constant de 50 %          | 6  |
| Figure 3 : Organigramme du programme de création de fichier CSV pendant 10 minutes avec la consigne et la commande                                                                        | 7  |
| Figure 4 : Graphique de l'évolution de la température de la lampe en fonction du temps écoulé pendant 10 minutes avec une commande PID                                                    | 8  |
| Figure 5 : Organigramme du programme de publication de la température de la lampe à intervalle régulier de 2 secondes                                                                     | 9  |
| Figure 6 : Organigramme du programme d'affichage de la température de la lampe à intervalle régulier de 2 secondes                                                                        | 10 |
| Figure 7 : Organigramme du programme de publication de données à intervalle régulier de 2 secondes                                                                                        | 12 |
| Figure 8 : Organigramme du programme d'affichage de données à intervalle régulier de 2 secondes                                                                                           | 13 |
| Figure 9 : Capture d'écran du publieur                                                                                                                                                    | 14 |
| Figure 10 : Capture d'écran de l'abonné                                                                                                                                                   | 14 |
| Figure 11 : Organigramme du programme de publication de données à intervalle régulier de 2 secondes avec l'abonnement à deux nouveaux topics                                              | 15 |
| Figure 12 : Organigramme du programme d'affichage de données à intervalle régulier de 2 secondes avec l'abonnement à deux nouveaux topics                                                 | 16 |
| Figure 13 : Capture d'écran de l'application MQTT Dash avec la DEL rouge allumée, le bouton ON/OFF non pressé et sans consigne entrée sur l'application                                   | 17 |
| Figure 14 : Capture d'écran de l'application MQTT Dash avec la DEL jaune allumée, le bouton ON/OFF pressé et avec une consigne entrée sur l'application                                   | 17 |
| Figure 15 : Capture d'écran de l'application Android Automate pour envoyer une notification sur la température et l'humidité à Montréal et une notification sur la luminosité de la salle | 18 |
| Figure 16 : Capture d'écran des notifications reçues sur l'application Android Automate                                                                                                   | 18 |
| Figure 17 : Capture d'écran de la configuration d'un paramètre                                                                                                                            | 19 |
| Figure 18 : Capture d'écran des messages affichés sur Node-RED                                                                                                                            | 19 |

## Buts et objectifs du laboratoire

Le but de ce laboratoire est de concevoir une première couche réseau IIoT. Pour ce faire, la maîtrise du protocole MQTT est nécessaire. De plus, une communication avec des capteurs connectés via un réseau LoRaWAN est essentielle. Enfin, une bonne maîtrise des applications MQTT Dash et Automate est fondamentale de même qu'une maîtrise de la plateforme Node-RED est nécessaire pour mener à bien ce laboratoire.

L'objectif de ce deuxième laboratoire est de concevoir une première couche réseau en communication avec un capteur de la couche de perception réalisée dans le premier laboratoire.

Enfin, le dernier objectif de ce laboratoire est de maîtriser le protocole MQTT pour publier et avoir accès aux données publiées sur les différents Topic. De plus, nous devons maîtriser les applications MQTT Dash pour accéder aux données en temps réel et pouvoir agir à distance sur le programme et Automate, afin d'envoyer des notifications à notre appareil concernant différentes données. Enfin, une maîtrise de la plateforme Node-RED est essentielle pour les futurs laboratoires.

Pour ce faire, une publication des données lues sur des topics et une lecture des données reçues sur ces différents topics avec le protocole MQTT est nécessaire. Ainsi, nous devons nous familiariser avec ce protocole pour publier et lire les données publiées sur les différents topics créés.

Ensuite, une observation des données sur l'application MQTT Dash est nécessaire afin d'obtenir le suivi en temps réel des différentes données publiées sur les divers topics. De plus, une publication sur l'application Automate est essentielle pour recevoir une notification présentant les données reçues sur notre appareil.

Enfin l'utilisation de Node-RED est fondamentale afin d'observer en temps réel les données publiées sur les différents topics.

# Démarche de conception

## Sauvegarde locale des données

### Premier fichier CSV sur 3 minutes

Durant cette manipulation, nous devons sauvegarder les données de la lampe et son thermocouple à intervalle régulier de 2 secondes, pendant une durée de 3 minutes, avec une commande constante de PWM à 50 % dans un fichier CSV localement dans la mémoire de la Raspberry Pi.

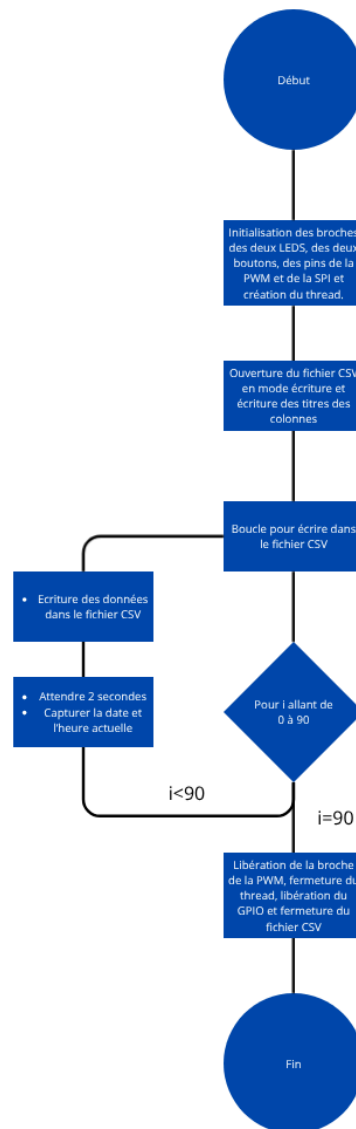


Figure 1 : Organigramme du programme de création de fichier CSV pendant 3 minutes

D'après la Figure 1, dans le programme, nous commençons par configurer les broches des deux boutons, des deux DEL, des pins de la PWM, et du protocole SPI. Nous créons le thread nécessaire pour allumer la lampe avec la PWM. Puis, nous ouvrons le fichier CSV et écrivons les titres des différentes colonnes de notre fichier CSV. Ensuite, nous entrons dans une boucle de 90 itérations ( $90 * 2s = 3 \text{ minutes}$ ) dans laquelle à chaque tour, nous attendons 2 secondes, nous capturons l'heure et la date actuelle et nous les renseignons dans le fichier CSV avec les données de la lampe. Enfin, à l'issue de la boucle, nous libérons la PWM, et le GPIO, nous fermons le thread et le fichier CSV et nous arrêtons le programme avec une valeur de retour.

Avec ce programme, nous obtenons la courbe suivante :

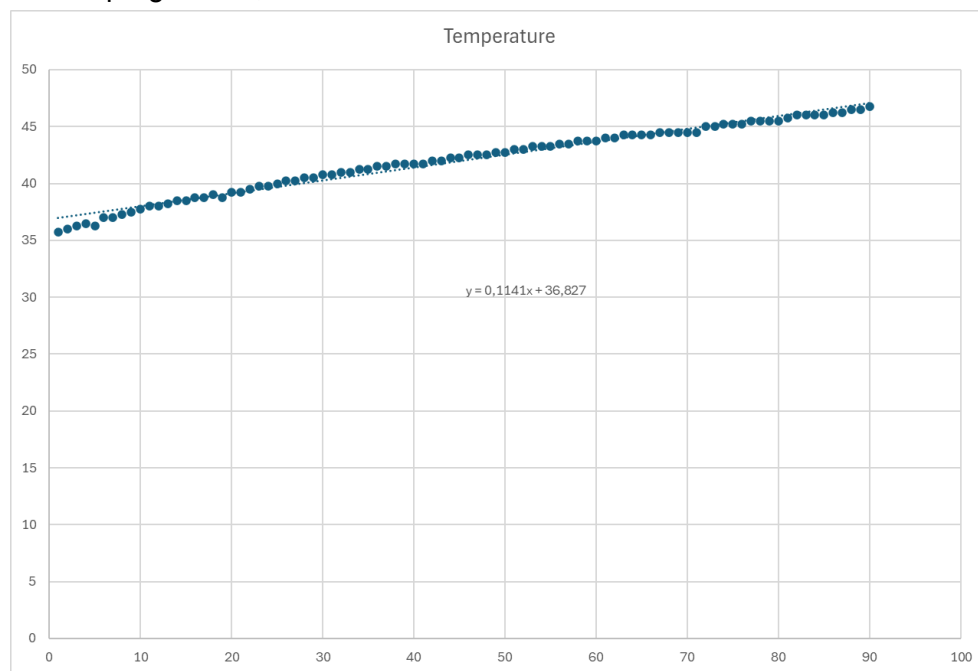


Figure 2 : Graphique de l'évolution de la température de la lampe en fonction du nombre de mesures (1 mesure = 2 secondes) pendant 3 minutes avec un signal PWM constant de 50 %

Nous observons que la courbe de tendance est une fonction affine, ce qui correspond bien à une évolution avec une PWM constante de 50 %. Ainsi, nous en déduisons l'équation de l'évolution de la température au cours du temps (avec  $t$  le temps en secondes) :

$$\Theta(t) = 36.827 + 0.2282t$$

La difficulté dans cette partie était d'écrire un programme qui réalise exactement le nombre d'itérations demandées tout en s'arrêtant immédiatement après avoir effectué ces itérations. De plus, il fallait actualiser la date et l'heure à chaque itération avant d'écrire les données dans le fichier CSV.

## Second fichier CSV sur 10 minutes

Durant cette manipulation, nous devons sauvegarder les données de la lampe et son thermocouple en ajoutant la consigne et la commande à intervalle régulier de 2 secondes pendant une durée de 10 minutes avec une commande PID dans un fichier CSV localement dans la mémoire de la Raspberry Pi.

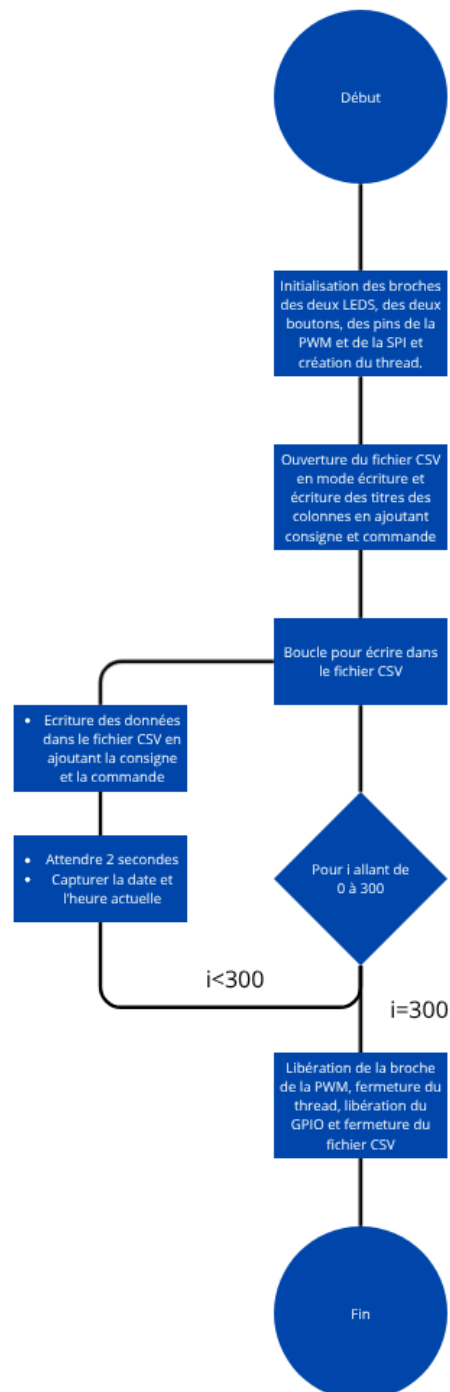


Figure 3 : Organigramme du programme de création de fichier CSV pendant 10 minutes avec la consigne et la commande

D'après la Figure 3, dans le programme, nous commençons par configurer les broches des deux boutons, des deux DEL des pins de la PWM et du protocole SPI. Puis, nous créons le thread nécessaire pour allumer la lampe avec la PWM. Ensuite, nous ouvrons le fichier CSV et écrivons les titres des différentes colonnes de notre fichier CSV en ajoutant la commande et la consigne par rapport au premier fichier. Après, nous entrons dans une boucle de 300 itérations ( $300 * 2s = 10 \text{ minutes}$ ) dans laquelle à chaque tour, nous attendons 2 secondes, nous capturons l'heure et la date actuelle et nous les inscrivons dans le fichier CSV avec les données de la lampe en ajoutant la commande et la consigne par rapport au premier fichier. En outre, à la fin de la boucle, nous libérons la PWM, et le GPIO, nous fermons le thread et le fichier CSV et nous arrêtons le programme avec une valeur de retour.

Avec ce programme, nous obtenons la courbe suivante :

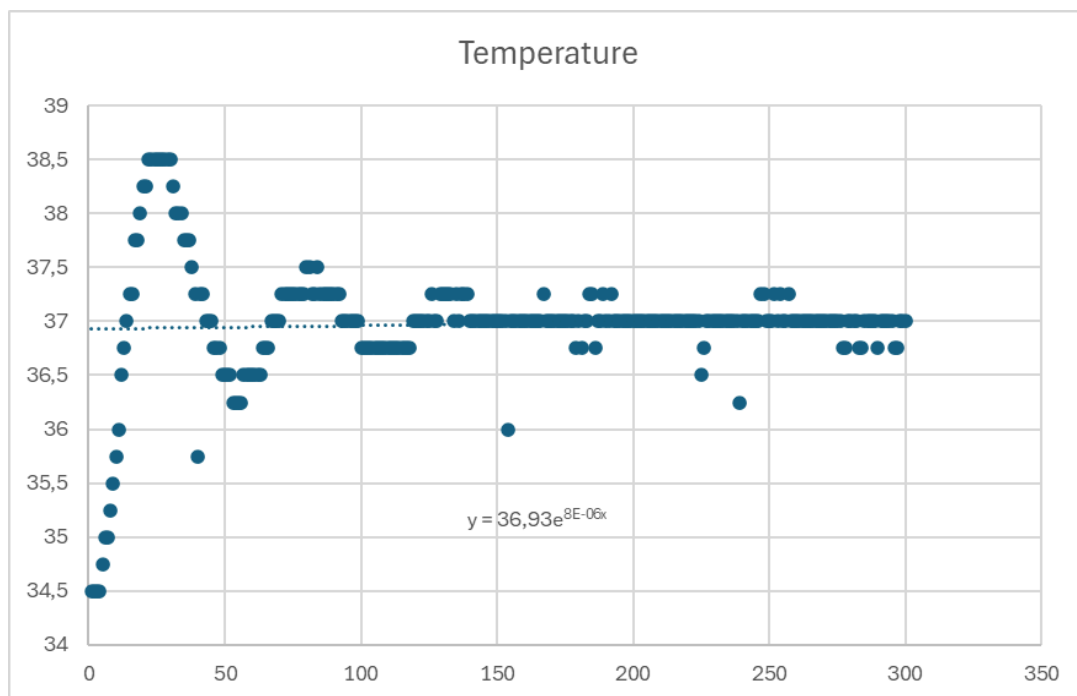


Figure 4 : Graphique de l'évolution de la température de la lampe en fonction du temps écoulé pendant 10 minutes avec une commande PID

Nous observons que la courbe de tendance est une fonction exponentielle, ce qui correspond bien à une évolution avec une commande de type PID. Ainsi, nous en déduisons l'équation de l'évolution de la température au cours du temps (avec  $t$  pour le temps en secondes) :

$$\Theta(t) = 36.93e^{1.6E-5t}$$

La difficulté dans cette partie était d'écrire un programme qui réalise exactement le nombre d'itérations demandées tout en s'arrêtant immédiatement après avoir effectué ces itérations. De plus, il fallait actualiser la date et l'heure à chaque itération avant de renseigner les données dans le fichier CSV. Enfin, il nous fallait ajouter la commande et la consigne pour chaque itération.

# Mosquitto et le MQTT

## Premier programme Publieur et Abonné sur 1 topic

Durant cette manipulation, nous devons créer un programme qui publie à intervalle régulier de 2 secondes la température de la lampe et qui s'arrête en cas de pression du bouton-poussoir rouge. Cependant, nous devons également créer un programme qui affiche à intervalle régulier de 2 secondes la température de la lampe précédemment publiée, et qui doit s'arrêter en cas de pression du bouton-poussoir rouge.

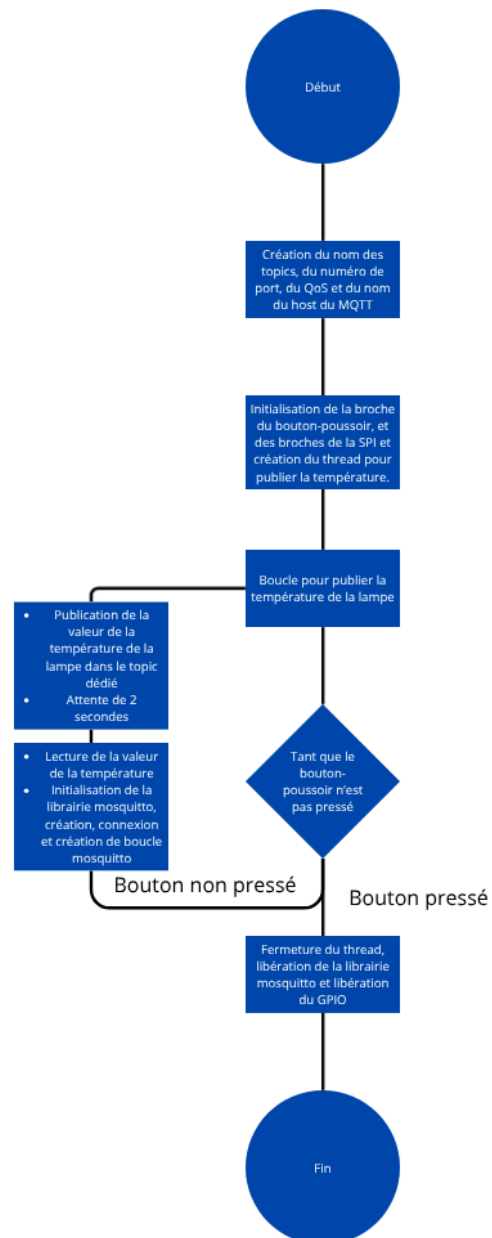


Figure 5 : Organigramme du programme de publication de la température de la lampe à intervalle régulier de 2 secondes

D'après la Figure 5, dans le programme, nous commençons par créer le nom du topic utilisé, du numéro de port, du QoS et du nom de host utilisé pour le protocole MQTT. Ensuite, nous initialisons la broche du bouton-poussoir, celles du protocole SPI et nous créons un thread permettant la publication de la valeur de la température relevée. Puis, nous entrons dans une boucle dans laquelle nous lisons la valeur de la température de la lampe, nous initialisons la librairie mosquitto que nous créons, après quoi nous connectons et créons la boucle du mosquitto. Enfin, nous publions la valeur de la température relevée dans le topic dédié et nous attendons 2 secondes. Tout ceci doit s'effectuer tant que le bouton-poussoir n'est pas pressé. Si celui-ci est pressé, nous fermons le thread, nous libérons la librairie mosquitto et le GPIO et nous renvoyons une valeur de retour.

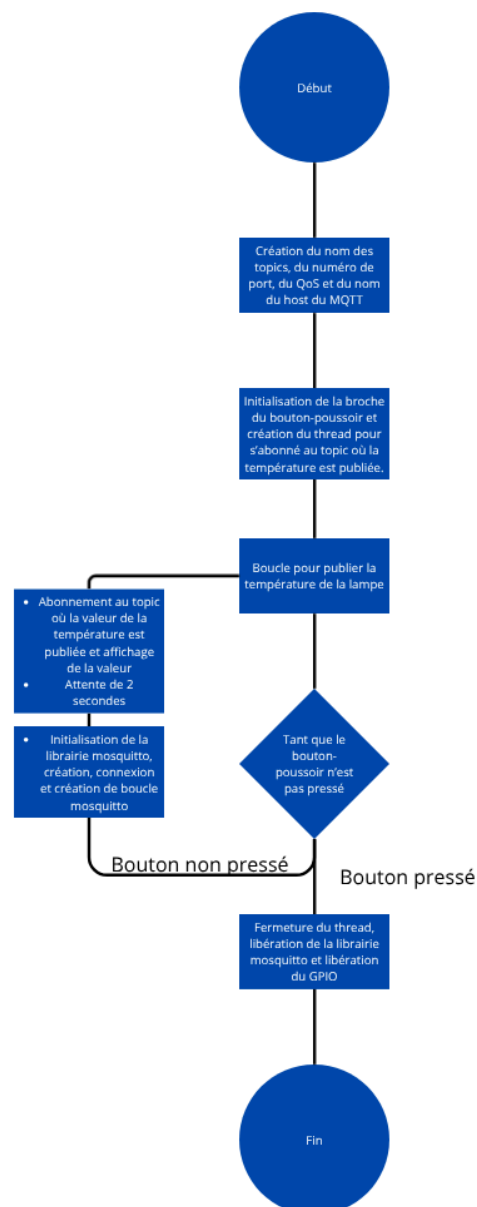


Figure 6 : Organigramme du programme d'affichage de la température de la lampe à intervalle régulier de 2 secondes

D'après la Figure 6, dans le programme, nous commençons par créer le nom du topic utilisé, du numéro de port, du QoS et du nom de host utilisé pour le protocole MQTT. Ensuite, nous initialisons la broche du bouton-poussoir et nous créons un thread permettant l'affichage de la valeur de la température publiée. Puis, nous entrons dans une boucle dans laquelle nous initialisons la librairie mosquitto que nous créons, nous connectons et créons la boucle du mosquitto. Enfin, nous nous abonnons au topic dans lequel la valeur de la température de la lampe est publiée, nous l'affichons et nous attendons 2 secondes. Tout ceci doit avoir lieu tant que le bouton-poussoir n'est pas pressé. Si celui-ci est pressé, nous fermons le thread, puis libérons la librairie mosquitto et le GPIO avant de renvoyer une valeur de retour.

La difficulté dans cette partie était d'adapter les codes fournis dans les listings pour publier la donnée dans le topic dédié mais aussi pour afficher les données publiées au bon format et avec la bonne fréquence.

## Second programme Publieur et Abonné sur 5 topics

Durant cette manipulation, nous devons créer un programme qui publie à intervalle régulier de 2 secondes la température de la lampe, la consigne de température, le pourcentage PWM calculé et l'état de deux DEL et qui s'arrête en cas de pression du bouton-poussoir rouge. De plus, le programme doit utiliser une commande de type PID avec des paramètres fixes.

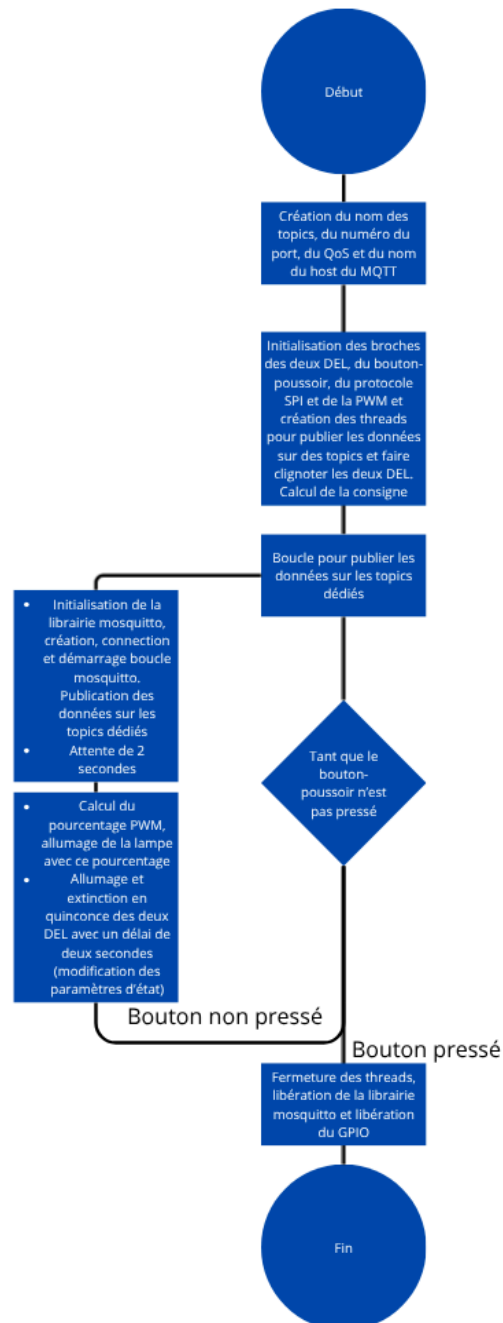


Figure 7 : Organigramme du programme de publication de données à intervalle régulier de 2 secondes

D'après la Figure 7, dans le programme, nous commençons par créer le nom des topics utilisés, du numéro de port, du QoS et du nom de host utilisé pour le protocole MQTT. Ensuite, nous initialisons la broche du bouton-poussoir, celles des deux DEL, celles de la PWM et celles du protocole SPI et nous créons les threads permettant la publication des données et l'allumage et l'extinction des deux DEL. Nous modifions en temps réel les variables d'état des deux DEL. Nous calculons la consigne puis, nous entrons dans une boucle dans laquelle nous calculons le pourcentage PWM à l'aide de la commande de type PID. Nous allumons et éteignons les deux DEL en quinconce avec un délai de 2 secondes. Enfin, nous initialisons la librairie mosquitto que nous créons avant de connecter et créer la boucle du mosquitto. En outre, nous publions les données relevées sur les topics dédiés et nous attendons 2 secondes, tout ceci tant que le bouton-poussoir n'est pas pressé. Si celui-ci est pressé, nous fermons les threads, nous libérons la librairie mosquitto et le GPIO et nous renvoyons une valeur de retour.

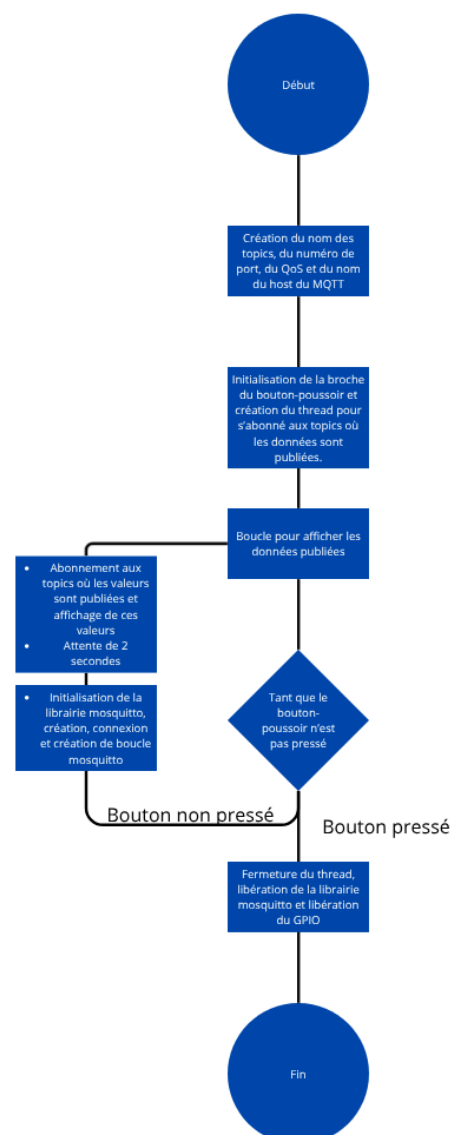


Figure 8 : Organigramme du programme d'affichage de données à intervalle régulier de 2 secondes

D'après la Figure 8, dans le programme, nous commençons par créer les noms des topics utilisés, du numéro de port, du QoS et du nom de host utilisé pour le protocole MQTT. Ensuite, nous initialisons la broche du bouton-poussoir et nous créons un thread permettant l'affichage des données publiées. Puis, nous entrons dans une boucle dans laquelle nous initialisons la librairie mosquitto que nous créons, et nous connectons et créons la boucle du mosquitto. Enfin, nous nous abonnons aux topics dans lesquels les valeurs sont publiées, nous les affichons et nous attendons 2 secondes, tout cela tant que le bouton-poussoir n'est pas pressé. Si celui-ci est pressé, nous fermons le thread, nous libérons la librairie mosquitto et le GPIO et nous renvoyons une valeur de retour.

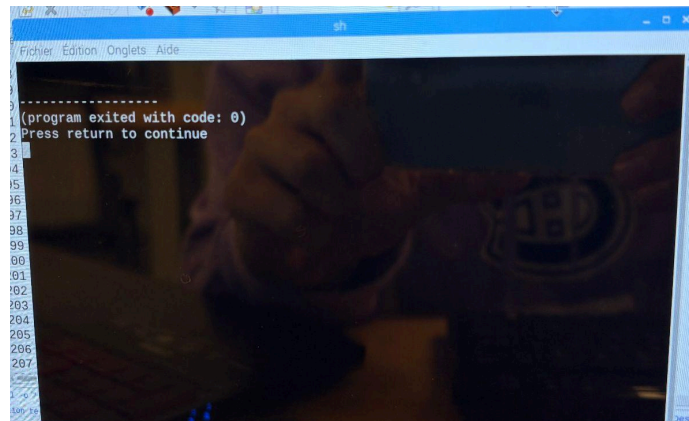


Figure 9 : Capture d'écran du publieur

Nous observons que le programme s'arrête lors de la pression du bouton-poussoir rouge.

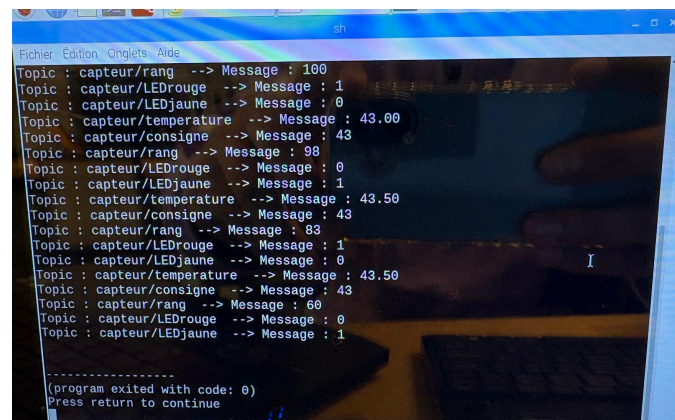


Figure 10 : Capture d'écran de l'abonné

Nous observons que le programme affiche chaque nouvelle donnée publiée sur les différents topics et qu'il s'arrête lors de la pression du bouton-poussoir rouge.

La difficulté dans cette partie était d'adapter le programme précédent pour publier les données dans les topics dédiés afin d'afficher les données publiées au bon format et avec la bonne fréquence. De plus, il fallait actualiser en temps réel l'état des deux DEL, la consigne, la commande et la température pour publier les bonnes données sur les topics dédiés.

## Mosquitto sur MQTT Dash

Troisième programme en ajoutant un bouton ON/OFF et une consigne sur l'application

Durant cette manipulation, nous devons nous abonner à l'aide de l'application MQTT Dash aux différents topics sur lesquels les différentes données sont publiées. De plus, nous devons ajouter sur cette application un bouton ON/OFF (permettant de stopper le programme avec la pression de ce bouton) et une consigne permettant de contrôler la température de la lampe avec la commande de type PID. Ces deux nouveaux éléments doivent publier des données sur deux nouveaux topics auxquels le publieur devra s'abonner.

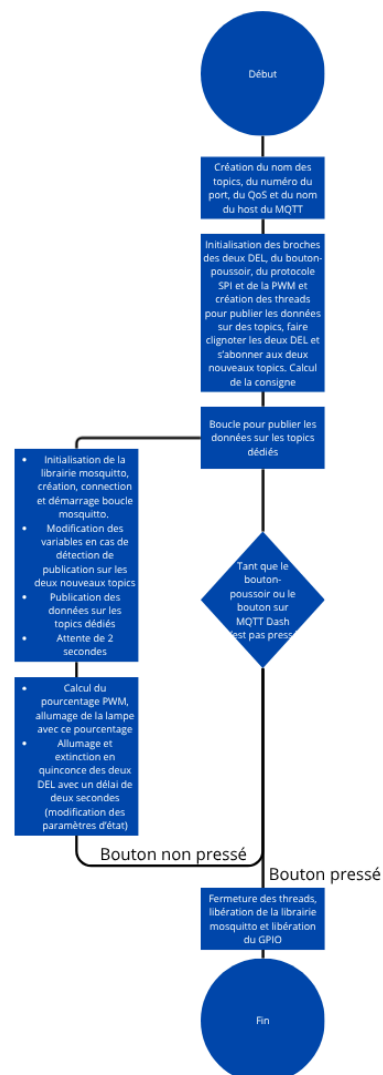


Figure 11 : Organigramme du programme de publication de données à intervalle régulier de 2 secondes avec l'abonnement à deux nouveaux topics

D'après la Figure 11, dans le programme, nous commençons par créer le nom des topics utilisés, du numéro de port, du QoS et du nom de host utilisé pour le protocole MQTT.

Ensuite, nous initialisons la broche du bouton-poussoir, celles des deux DEL, celles de la PWM et celles du protocole SPI. Puis, nous créons les threads permettant la publication des données, l'allumage et l'extinction des deux DEL et l'abonnement aux deux nouveaux topics où l'état du bouton sur MQTT Dash est publié ainsi que la valeur de la nouvelle consigne. Nous calculons la première consigne. Puis, nous entrons dans une boucle dans laquelle nous calculons le pourcentage PWM tout en utilisant la commande de type PID. Nous allumons et éteignons les deux DEL en quinconce avec un délai de 2 secondes et nous modifions en temps réel les variables d'état des deux DEL. Enfin, nous initialisons la librairie mosquitto que nous créons. Nous connectons et créons la boucle du mosquitto, nous modifions les variables de consigne et de d'état du bouton en cas de détection de publication de message sur les deux topics auxquels le publieur est abonné. Enfin, nous publions les données relevées sur les topics dédiés et nous attendons 2 secondes. Le tout s'effectue tant que le bouton-poussoir ou le bouton sur MQTT Dash n'est pas pressé. Si l'un des deux est pressé, nous fermons les threads, nous libérons la librairie mosquitto et le GPIO et nous renvoyons une valeur de retour.

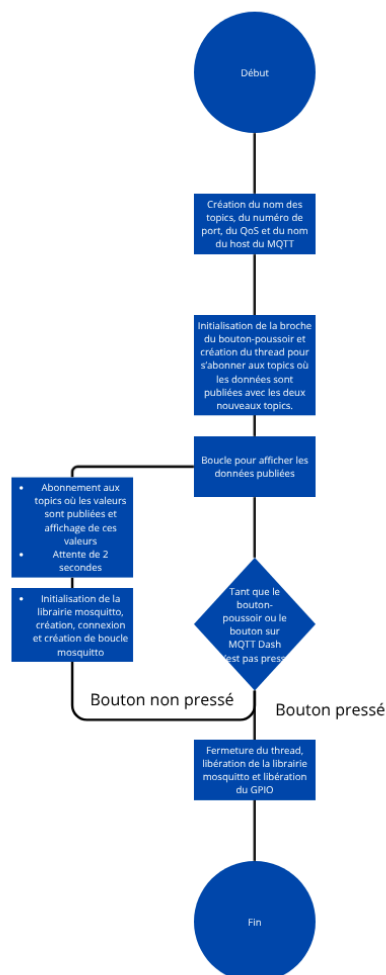


Figure 12 : Organigramme du programme d'affichage de données à intervalle régulier de 2 secondes avec l'abonnement à deux nouveaux topics

D'après la Figure 12, dans le programme, nous commençons par créer les noms des topics utilisés, du numéro de port, du QoS et du nom de host utilisé pour le protocole MQTT.

Ensuite, nous initialisons la broche du bouton-poussoir et nous créons un thread permettant l'affichage des données publiées. Puis, nous entrons dans une boucle dans laquelle nous initialisons la librairie mosquitto que nous créons avant de connecter et créer la boucle du mosquitto. Enfin, nous nous abonnons aux topics, y compris les deux nouveaux, dans lesquels les valeurs sont publiées. Nous les affichons et nous attendons 2 secondes. Tout ceci a lieu tant que le bouton-poussoir ou le bouton sur MQTT Dash n'est pas pressé. Si l'un des deux est pressé, nous fermons le thread, nous libérons la librairie mosquitto et le GPIO et nous renvoyons une valeur de retour.

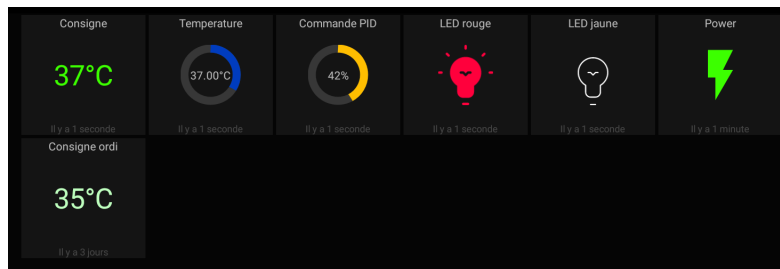


Figure 13 : Capture d'écran de l'application MQTT Dash avec la DEL rouge allumée, le bouton ON/OFF non pressé et sans consigne entrée sur l'application

Nous observons sur la figure 13, l'affichage de la consigne calculée par le programme, de l'évolution de la température de la lampe, de la commande PID (entre 0 et 100 %), de l'état des deux DEL (allumées ou éteintes), de l'état du bouton power et de la consigne fixée sur l'application.

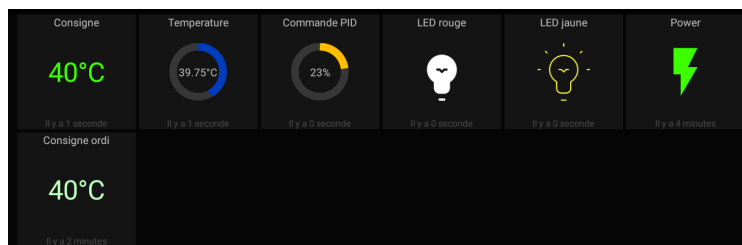


Figure 14 : Capture d'écran de l'application MQTT Dash avec la DEL jaune allumée, le bouton ON/OFF pressé et avec une consigne entrée sur l'application

Nous observons sur la figure 14 le même affichage que celui présent sur la figure 13. Cependant, on peut noter que celui-ci contient la consigne calculée variable en fonction de la valeur entrée sur l'application.

La difficulté dans cette partie était de recevoir les données envoyées par l'application MQTT Dash pour arrêter le programme ou pour modifier la consigne. Il fallait donc que le publieur s'abonne aux topics sur lesquels MQTT Dash publie et il fallait actualiser les nouvelles données lors de cette publication. Enfin, il fallait prendre en main l'application pour aboutir à un affichage adapté des données.

## Notifications sur Android automate

Durant cette manipulation, nous devons envoyer une requête HTTP vers l'API météo de Montréal. Le but est d'obtenir des données sur la météo à Montréal ainsi que de mesurer la luminosité de la salle. Nous devons ensuite envoyer une notification présentant le nom de la ville, sa température et son humidité et une autre notification présentant la valeur de la lumière ambiante. Nous devons également afficher un message de prévention si la valeur est en dehors de l'intervalle [100 - 400] luxs.

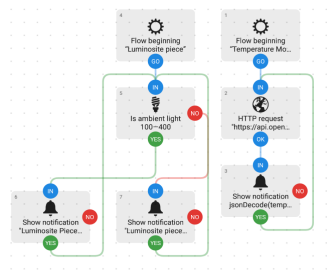


Figure 15 : Capture d'écran de l'application Android Automate afin d'envoyer une notification sur la température et l'humidité à Montréal et une notification sur la luminosité de la salle

Nous observons sur la figure 15 le programme sur Automate. Nous observons que nous démarrons deux nœuds: dans le premier, nous réalisons une requête HTTP vers l'API de Montréal. Puis, nous envoyons une notification vers le mobile présentant les données relevées. Dans le deuxième, nous relevons la luminosité de la pièce. Si celle-ci est dans l'intervalle [100 - 400] luxs, alors nous affichons la luminosité uniquement. Si ce n'est pas le cas, nous affichons la luminosité avec un message de prévention.

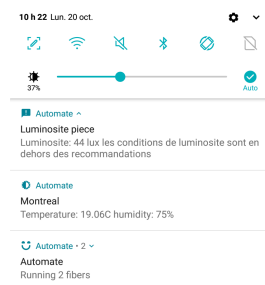


Figure 16 : Capture d'écran des notifications reçues sur l'application Android Automate

Nous observons sur la figure 16, les notifications reçues par l'appareil mobile. La première notification présente la température et l'humidité à Montréal. La deuxième présente la luminosité de la salle avec un message disant que les conditions de luminosité sont en dehors des recommandations, donc en dehors de l'intervalle [100 - 400] luxs.

La difficulté dans cette partie était de prendre en main l'application Android automate afin d'acquérir des données au format json mais aussi pour recevoir des données et envoyer des notifications avec le bon format.

## Utilisation de Node-RED

Durant cette manipulation, nous devons afficher la valeur du thermocouple de la lampe, la consigne et la commande en utilisant l'interface Node-RED.

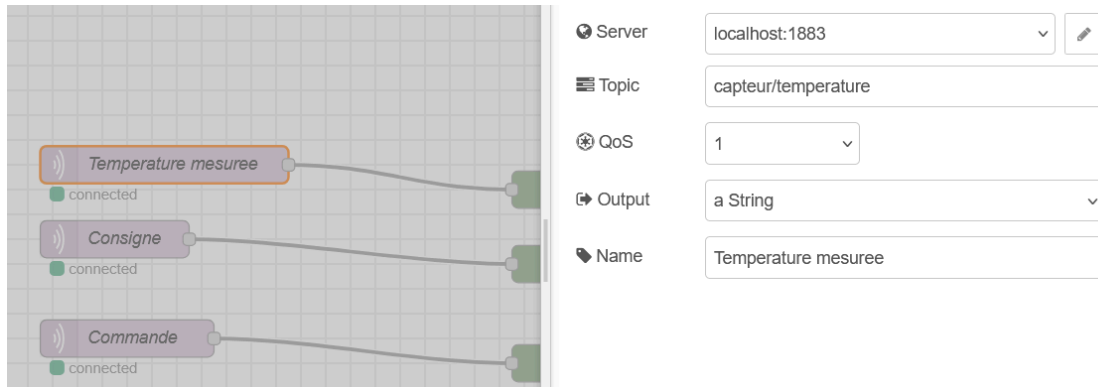


Figure 17 : Capture d'écran de la configuration d'un paramètre

Nous observons sur la figure 17, le paramétrage du message MQTT. Puis, nous entrons dans Server le nom du host et le numéro de port utilisé. Enfin, nous entrons dans Topic, le nom de topic utilisé. Nous effectuons le même paramétrage pour la consigne et la commande. Néanmoins, nous changeons les topics avec capteur/consigne pour la consigne et capteur/rang pour la commande.

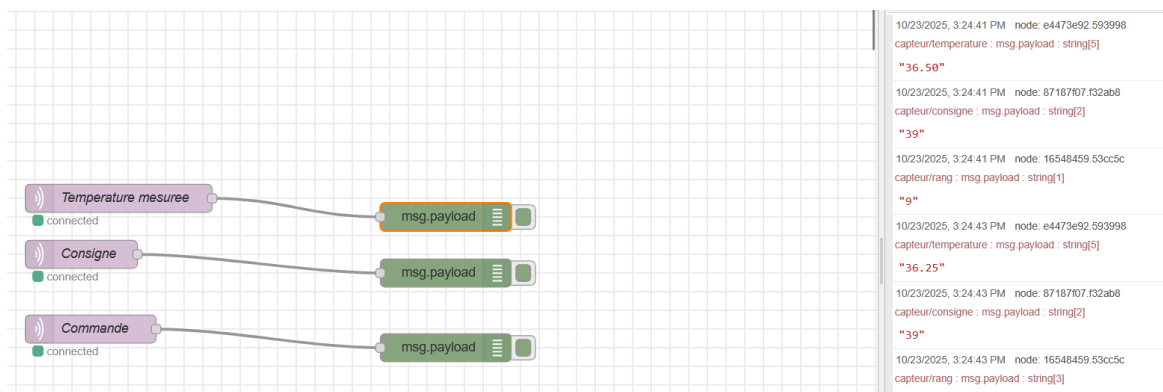


Figure 18 : Capture d'écran des messages affichés sur Node-RED

Nous observons sur la figure 18, que les messages publiés sur les trois topics s'affichent sur le debug de la page web avec la date et l'heure de réception, le numéro de nœud, le nom de topic, la taille du message et le message.

La difficulté dans cette partie était de prendre en main la page web Node-RED pour acquérir les données et pour les afficher.

# Conclusion

Ce laboratoire nous aura permis de concevoir une première couche réseau IIoT. Nous avons appris à créer un fichier de type CSV et à le modifier en respectant un certain délai. Nous avons appris à analyser ce même fichier en réalisant des courbes qui utilisent les données relevées et les comparent à une courbe de tendance.

De plus, nous avons appris à maîtriser le protocole MQTT en écrivant un programme capable de publier des données sur les topics dédiés, mais aussi en écrivant un programme qui puisse s'abonner à ces topics et afficher les messages publiés. Le programme qui publie les données est capable de s'abonner à d'autres topics afin d'aboutir à une actualisation d'informations ou une extinction de programme.

Enfin, nous avons appris à utiliser les applications MQTT Dash qui permettent d'afficher en temps réel les données publiées sur les différents topics. Elles permettent également de publier des données sur de nouveaux topics.

Nous avons appris à utiliser Android Automate qui permet d'envoyer une notification sur des données relevées à un appareil mobile. Enfin, nous avons appris à utiliser Node-RED qui permet de recevoir et d'afficher les messages envoyés sur les différents topics sur une page web.

## Expérience personnelle

Je pense que ce laboratoire est bien agencé. Il permet de connaître la méthode pour construire un fichier de type CSV. De surcroît, il permet de connaître le principe et l'utilisation d'un nouveau protocole de communication qui est le MQTT. Celui-ci permet de recevoir et d'envoyer des données ainsi que de connaître l'utilisation de diverses applications qui traitent des données à l'aide d'un appareil mobile ou d'une page web.

Ce laboratoire m'a permis de découvrir le protocole MQTT pour traiter les données ainsi que de découvrir les différentes applications pour recevoir les données transitées par le protocole MQTT, et pour envoyer des notifications sur des données relevées par un appareil mobile ou par une page web.

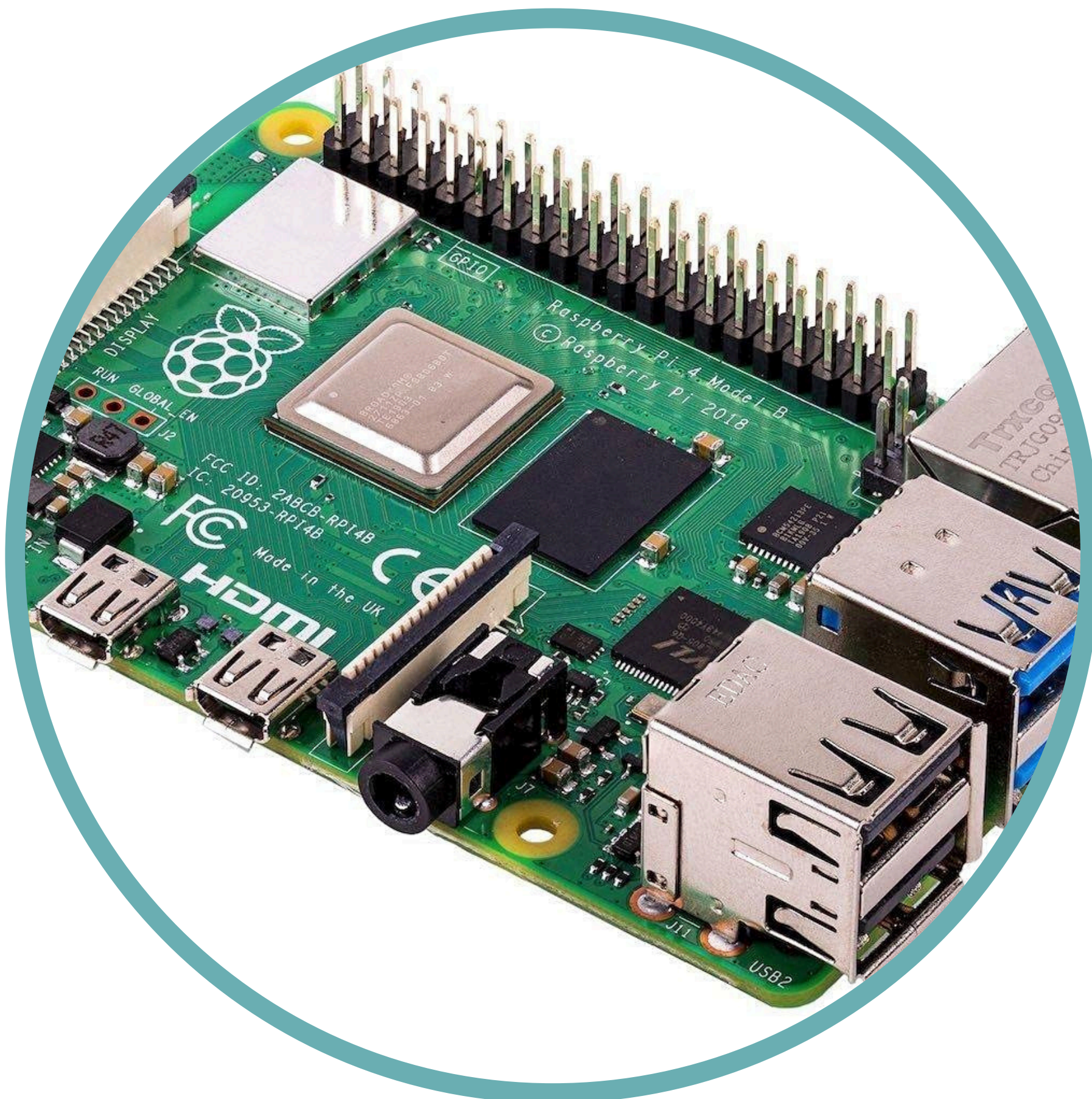
Enfin, ce laboratoire m'a donné la possibilité de mobiliser les connaissances précédemment acquises et les programmes précédemment implémentés pour réaliser de nouvelles tâches. Il m'a aussi donné l'opportunité de découvrir que mes connaissances précédemment acquises me permettent de réaliser des tâches plus complexes.

En conclusion, je dirais que ce laboratoire est pertinent dans la mesure où il permet de montrer l'intérêt des connaissances acquises précédemment et de les utiliser dans un projet concret. Selon moi, c'est un second laboratoire intéressant pour mobiliser ses connaissances, et ainsi réaliser des tâches plus complexes que dans le premier laboratoire, tout en utilisant les programmes précédemment implémentés.

Du 27/10 au 11/11

Rapport de laboratoire

Enseignant laboratoire : Bitouh Amar  
Enseignant cours : Lokman Sboui



Matys Benoist  
Programme court de 1er cycle 2025-2026

**ÉTS**

Le génie pour l'industrie

**ÉCOLE DE  
TECHNOLOGIE  
SUPÉRIEURE**

Université du Québec

**PIRANHAS  
ÉTS**



# Table des matières

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>Table des matières</b>                                  | <b>2</b>  |
| <b>Table des figures</b>                                   | <b>3</b>  |
| <b>Buts et objectifs du laboratoire</b>                    | <b>4</b>  |
| <b>Démarche de conception</b>                              | <b>5</b>  |
| <b>Communication avec capteurs Bluetooth via LoRaWAN</b>   | <b>5</b>  |
| Générer des fichiers CSV avec Node-RED                     | 8         |
| <b>Envoyer des fichiers CSV vers le cloud avec NodeRED</b> | <b>10</b> |
| Automatisation avec les Macros Excel                       | 15        |
| Premières macros                                           | 15        |
| Fusionner des fichiers CSV avec les macros                 | 16        |
| <b>Interface Graphique avec Node-RED</b>                   | <b>17</b> |
| Premier tableau de bord                                    | 17        |
| Importer des données à partir du Cloud                     | 21        |
| <b>Conclusion</b>                                          | <b>27</b> |
| <b>Expérience personnelle</b>                              | <b>27</b> |
| Annexe 1 : Code de la macro de fusion de fichiers CSV      | 29        |

# Table des figures

|                                                                                                                                                                                                  |    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 1 : Organigramme du noeud pour afficher les données du capteur S1                                                                                                                         | 6  |
| Figure 2 : Flux pour afficher les données du capteur S1                                                                                                                                          | 7  |
| Figure 3 : Captures d'écran des noeuds du flux pour afficher les données du capteur S1                                                                                                           | 8  |
| Figure 4 : Organigramme du noeud pour sauvegarder les données de la lampe avec une commande PWM constante de 25 % dans un fichier CSV                                                            | 9  |
| Figure 5 : Flux pour sauvegarder les données de la lampe avec une commande PWM constante de 25 % dans un fichier CSV                                                                             | 10 |
| Figure 6 : Captures d'écran des noeuds du flux pour sauvegarder les données de la lampe avec une commande PWM constante de 25 % dans un fichier CSV                                              | 10 |
| Figure 7 : Organigramme du noeud pour sauvegarder les données de la lampe avec une commande PID dans un fichier CSV envoyé au cloud                                                              | 11 |
| Figure 8 : Schéma représentant l'évolution de la commande et de la température de la lampe avec une consigne de 25 °C                                                                            | 12 |
| Figure 9 : Schéma représentant l'évolution de la commande et de la température de la lampe avec une consigne de 30 °C                                                                            | 12 |
| Figure 10 : Schéma représentant l'évolution de la commande et de la température de la lampe avec une consigne de 35 °C                                                                           | 13 |
| Figure 11 : Flux pour sauvegarder les données de la lampe avec une commande PID dans un fichier CSV envoyé au cloud                                                                              | 13 |
| Figure 12 : Captures d'écran des noeuds du flux pour sauvegarder les données de la lampe avec une commande PID dans un fichier CSV envoyé au cloud                                               | 14 |
| Figure 13 : Organigramme de la manipulation pour réaliser des macros Excel qui permettent de trier une liste                                                                                     | 16 |
| Figure 14 : Organigramme de la manipulation pour réaliser une macro Excel qui permet de fusionner des fichiers CSV                                                                               | 17 |
| Figure 15 : Organigramme du noeud pour afficher sur le tableau de bord les données sur la lampe                                                                                                  | 18 |
| Figure 16 : Flux pour afficher la consigne et la température d'une lampe sur un tableau de bord                                                                                                  | 19 |
| Figure 17 : Captures d'écran des noeuds du flux pour afficher les données d'une lampe sur un tableau de bord                                                                                     | 20 |
| Figure 18 : Captures d'écran du tableau de bord présentant les données de la lampe                                                                                                               | 21 |
| Figure 19 : Organigramme du noeud pour afficher sur le tableau de bord les données sur la lampe, l'heure et la date actuelle, les informations sur Montréal et les données des capteurs S1 et E7 | 22 |
| Figure 20 : Captures d'écran du tableau de bord final (après la séance du 17/11, le capteur S1 était éteint)                                                                                     | 23 |
| Figure 21 : Captures d'écran du tableau de bord final (avant la séance du 17/11, le capteur S1 était allumé)                                                                                     | 23 |
| Figure 22 : Flux pour afficher les données de la lampe, du capteur S1, du capteur E7, la date et l'heure et les données de Montréal sur un tableau de bord.                                      | 24 |
| Figure 23 : Captures d'écran des noeuds du flux pour afficher les données de la lampe, du capteur S1, du capteur E7, la date et l'heure et les données de Montréal sur un tableau de bord.       | 26 |

# Buts et objectifs du laboratoire

Le but de ce laboratoire est de continuer le travail sur la première couche réseau IIoT conçue dans le précédent laboratoire. Ainsi, nous allons voir comment réaliser un tableau de bord et comment récupérer des données sur le cloud.

L'objectif de ce troisième laboratoire est de communiquer avec les capteurs montés en bluetooth via un réseau LoRaWAN.

Le second objectif de ce laboratoire est de maîtriser les logiciels Excel en réalisant différents macros et NodeRED sur laquelle nous allons réaliser le tableau de bord.

Enfin, le dernier objectif de ce laboratoire est de réaliser des fichiers CSV et de les envoyer vers le cloud à l'aide de NodeRED.

Ce laboratoire est donc une continuité du précédent laboratoire, il nous apprend à utiliser les connaissances précédemment acquises et nous apprend à maîtriser différents logiciels utiles pour la sauvegarde et le transport de données.

# Démarche de conception

## Communication avec capteurs Bluetooth via LoRaWAN

Durant cette manipulation, nous devons afficher la température et l'humidité publiées par un capteur bluetooth de type S1 à l'aide du logiciel NodeRED.

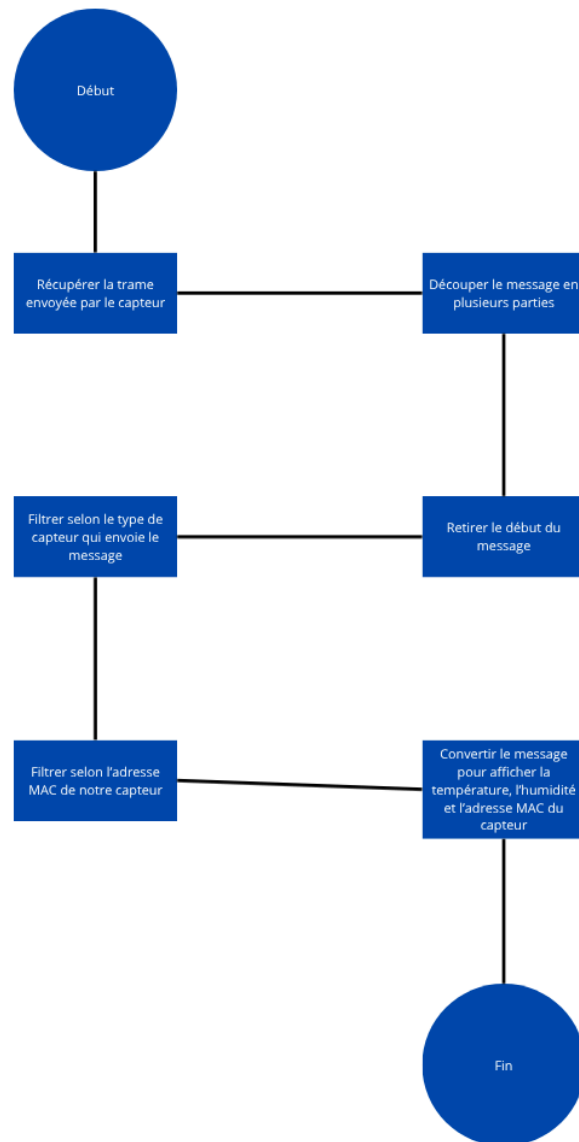


Figure 1 : Organigramme du noeud pour afficher les données du capteur S1

D'après la Figure 1, dans le noeud, nous commençons par nous connecter au websocket sur lequel le capteur publie, puis nous découpons notre message en plusieurs parties, nous enlevons le début, nous filtrons avec les différents types de capteurs et avec l'adresse MAC de notre capteur et enfin, nous affichons les différents messages reçus, c'est à dire la température, l'humidité et l'adresse MAC du capteur.

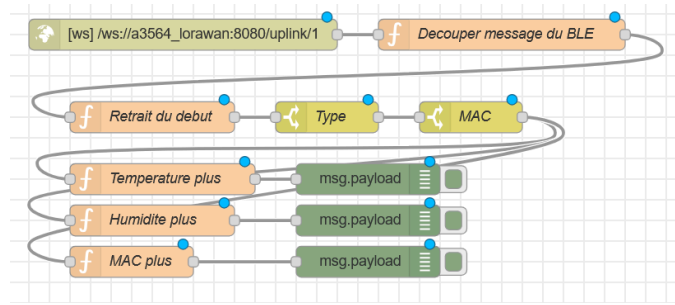


Figure 2 : Flux pour afficher les données du capteur S1

Nous pouvons observer sur la figure 2 le flux effectué sur Node-RED avec les différents nœuds utilisés.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Type: Listen on</p> <p>Path: /ws://a3564_lorawan:8080/uplink/1</p> <p>Name: Name</p>                                                                                                                                                                                                                                                                                                                                                                                                | <p>Name: Decouper message du BLE</p> <p>Function:</p> <pre> 1 var s = msg.payload.toString(); 2 3 var data = s.substr(3); 4 msg = [] 5 var tags = data.split(" "); 6 for (var w in tags){ 7   context.global.row = context.global.row+1; 8   msg.push({payload:tags[w]}) 9 } 10 return [msg]; </pre> |
| <p>Name: Retrait du debut</p> <p>Function:</p> <pre> 1 var data = msg.payload.substr(24); 2 msg.payload = data; 3 msg.type = data.substr(0,2); 4 msg.batt = data.substr(2,4); 5 msg.temperature = data.substr(4,8); 6 msg.humidite = data.substr(8,12); 7 msg.MAC1 = data.substr(12,14); 8 msg.MAC2 = data.substr(14,16); 9 msg.MAC3 = data.substr(16,18); 10 msg.MAC4 = data.substr(18,20); 11 msg.MAC5 = data.substr(20,22); 12 msg.MAC6 = data.substr(22,24); 13 return msg; </pre> | <p>Name: Type</p> <p>Property: msg.type</p> <p>Value: == 01</p>                                                                                                                                                                                                                                      |
| <p>Name: MAC</p> <p>Property: msg.MAC</p> <p>Value: == BB7AAB3F23AC</p>                                                                                                                                                                                                                                                                                                                                                                                                                | <p>Name: Temperature plus</p> <p>Function:</p> <pre> 1 t = (parseInt(msg.temperature,16))/256; 2 t = t.toFixed(1) 3 msg.payload = t 4 return msg; </pre>                                                                                                                                             |
| <p>Name: Humidite plus</p> <p>Function:</p> <pre> 1 t = (parseInt(msg.humidite,16))/256; 2 t = t.toFixed(1) 3 msg.payload = t 4 return msg; </pre>                                                                                                                                                                                                                                                                                                                                     | <p>Name: MAC plus</p> <p>Function:</p> <pre> 1 t = msg.MAC6+msg.MAC5+msg.MAC4+msg.MAC3+msg.MAC2+msg.MAC1; 2 return {payload: t }; </pre>                                                                                                                                                             |

Figure 3 : Captures d'écran des noeuds du flux pour afficher les données du capteur S1

D'après la figure 3, dans la fonction Découper message du BLE, nous découpons le message en plusieurs parties. Dans la fonction retrait du début, j'enregistre dans msg.type les bits 1 et 2, dans msg.temperature les bits 5 à 8, dans msg.humidite les bits 9 à 12 et dans 6 variables msg.MACx (x allant de 1 à 6) les bits de l'adresse MAC 2 à 2 (pour les remettre dans l'ordre ensuite). Dans le noeud Type, on ne prend en compte que les capteurs de type 01 donc les capteurs S1, dans le nœud MAC, j'ai écrit l'adresse MAC de mon capteur en format little Indian : BB7AA3F23AC. Dans la fonction Température plus, on convertit la valeur lue. Dans la fonction Humidité plus, j'effectue le même calcul que dans la fonction Température plus mais j'utilise msg.humidite. Dans la fonction MAC plus, je remets les bits 2 à 2 dans l'ordre. Enfin, j'affiche la température, l'humidité et l'adresse MAC du capteur S1.

## Générer des fichiers CSV avec Node-RED

Durant cette manipulation, nous devons sauvegarder les données de la lampe et son thermocouple à intervalle régulier de 2 secondes pendant 3 minutes avec une commande PWM constante de 25 %.

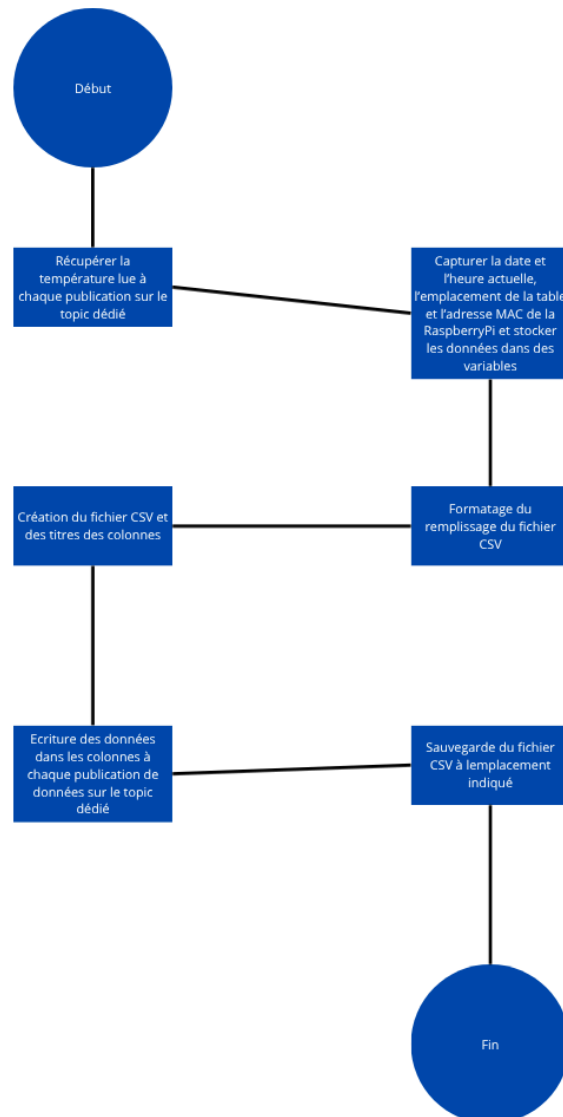


Figure 4 : Organigramme du noeud pour sauvegarder les données de la lampe avec une commande PWM constante de 25 % dans un fichier CSV

D'après la Figure 4, dans le programme, nous commençons par recevoir la température publiée sur le topic dédié, nous capturons ensuite la date et l'heure actuelle et nous stockons toutes les données dans les variables dédiées. Nous créons ensuite le fichier CSV avec le nom des colonnes, nous écrivons les différentes données dans les différentes colonnes dédiées à chaque publication de température dans le topic dédié (toutes les 2 secondes) pendant 4 minutes et enfin, nous sauvegardons le fichier CSV à l'emplacement que nous avons indiqué.

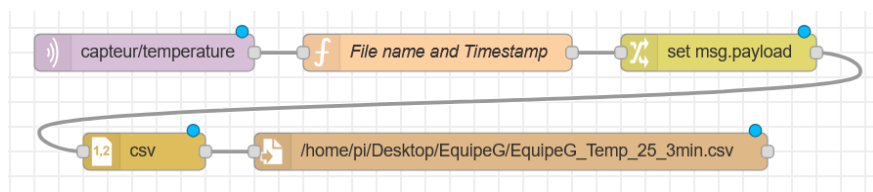


Figure 5 : Flux pour sauvegarder les données de la lampe avec une commande PWM constante de 25 % dans un fichier CSV

Nous pouvons observer sur la figure 5 le flux effectué sur Node-RED avec les différents nœuds utilisés.

|                                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-----------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Server: localhost:1883</p> <p>Topic: capteur/temperature</p> <p>QoS: 2</p> <p>Output: auto-detect (string or buffer)</p> <p>Name: Name</p> | <p>Name: File name and Timestamp</p> <p>Function:</p> <pre> 1 var now = new Date(); 2 var yyyy = now.getFullYear(); 3 var mm = now.getMonth() &lt; 9 ? "0" + (now.getMonth() + 1) : now.getMonth() + 1; 4 var dd = now.getDate() &lt; 10 ? "0" + now.getDate() : now.getDate(); 5 var hh = now.getHours() &lt; 10 ? "0" + now.getHours() : now.getHours(); 6 var mm = now.getMinutes() &lt; 10 ? "0" + now.getMinutes() : now.getMinutes(); 7 var ss = now.getSeconds() &lt; 10 ? "0" + now.getSeconds() : now.getSeconds(); 8 msg.timestamp = yyyy + "-" + mm + "-" + dd + " " + hh + ":" + mm + ":" + ss; 9 node.status({fill:"blue",shape:"ring",text:msg.timestamp}); 10 msg.MAC = "b8:27:eb:50:b0:24"; 11 msg.emplacement = 1; 12 msg.temperature = msg.payload; 13 return msg; </pre> |
| <p>Name: Name</p> <p>Rules:</p> <p>Set: msg.payload to [{"Timestamp":msg.timestamp,"A":msg.MAC,"B":msg.emplacement,"C":msg.temperature}]</p>  | <p>Columns: Timestamp, Adresse MAC, Emplacement, Température</p> <p>Separator: tab</p> <p>Name: Name</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <p>Filename: /home/pi/Desktop/EquipeG/EquipeG_Temp_25_3min.csv</p> <p>Action: append to file</p>                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |

Figure 6 : Captures d'écran des noeuds du flux pour sauvegarder les données de la lampe avec une commande PWM constante de 25 % dans un fichier CSV

D'après la figure 6, nous commençons par recevoir les données publiées sur le topic dédié à la température de la lampe. Dans la fonction File name and Timestamp, j'ajoute dans msg.timestamp la date et l'heure actuelle, dans msg.MAC, l'adresse MAC de la RaspberryPi, dans msg.emplacement, le numéro de la table sur laquelle nous effectuons le laboratoire et dans msg.temperature, le payload reçu par le MQTT in. Dans le nœud set msg.payload, nous configurons les 4 colonnes avec le timestamp, l'adresse MAC, l'emplacement et la température avec les msg correspondants. Dans le nœud CSV, nous réécrivons dans l'ordre les 4 colonnes Timestamp, Adresse MAC, Emplacement et Température. Et dans le nœud file, nous écrivons l'emplacement dans lequel nous souhaitons enregistrer le fichier avec son nom.

## Envoyer des fichiers CSV vers le cloud avec NodeRED

Durant cette manipulation, nous devons créer des fichiers CSV répertoriant les données de la lampe avec la consigne et la commande à intervalle régulier de 2 secondes pendant 4 minutes avec plusieurs consignes de 25, 30 et 35 °C avec une commande PID et envoyer les fichiers dans le cloud.

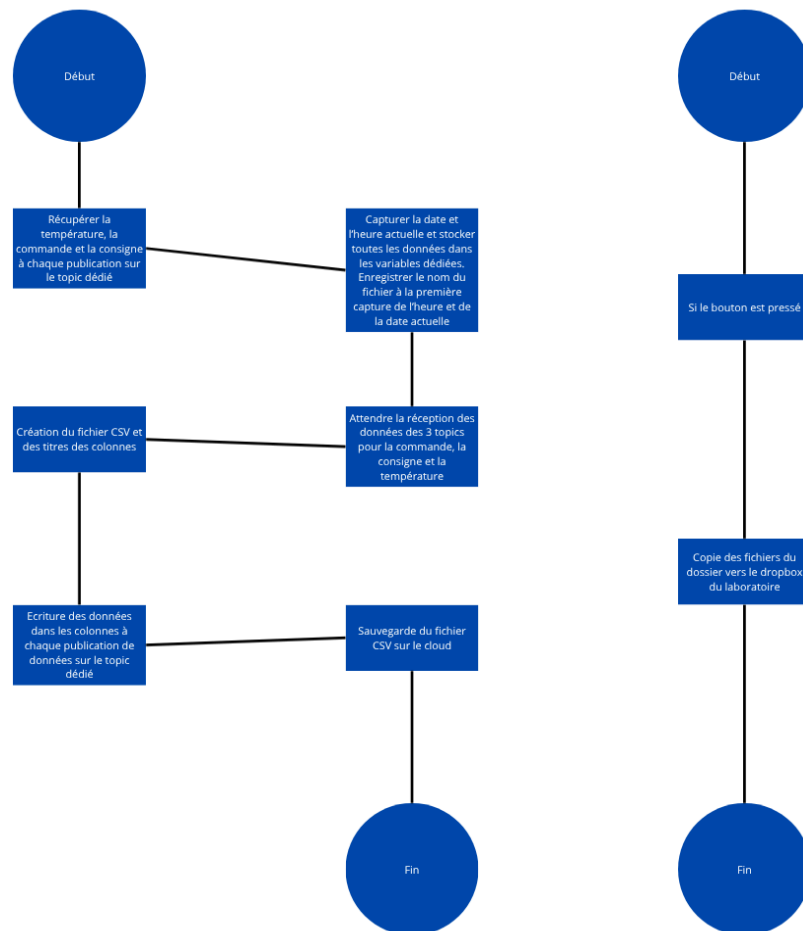


Figure 7 : Organigramme du noeud pour sauvegarder les données de la lampe avec une commande PID dans un fichier CSV envoyé au cloud

D'après la Figure 7, dans le programme, nous commençons par recevoir la température, la commande et la consigne publiées sur les topics dédiés, nous capturons ensuite la date et l'heure actuelle et nous stockons toutes les données dans les variables dédiées. De plus, nous modifions le nom du fichier à la première capture de la date et de l'heure actuelle. Nous attendons la réception des données des 3 topics dédiés et nous créons ensuite le fichier CSV avec le nom des colonnes, nous écrivons les différentes données dans les différentes colonnes dédiées à chaque publication de température dans le topic dédié (toutes les 2 secondes) pendant 4 minutes. Enfin, nous sauvegardons le fichier CSV sur le cloud. De plus, si nous appuyons sur le bouton, nous copions tous les fichiers enregistrés dans le dossier vers le dropbox du laboratoire.

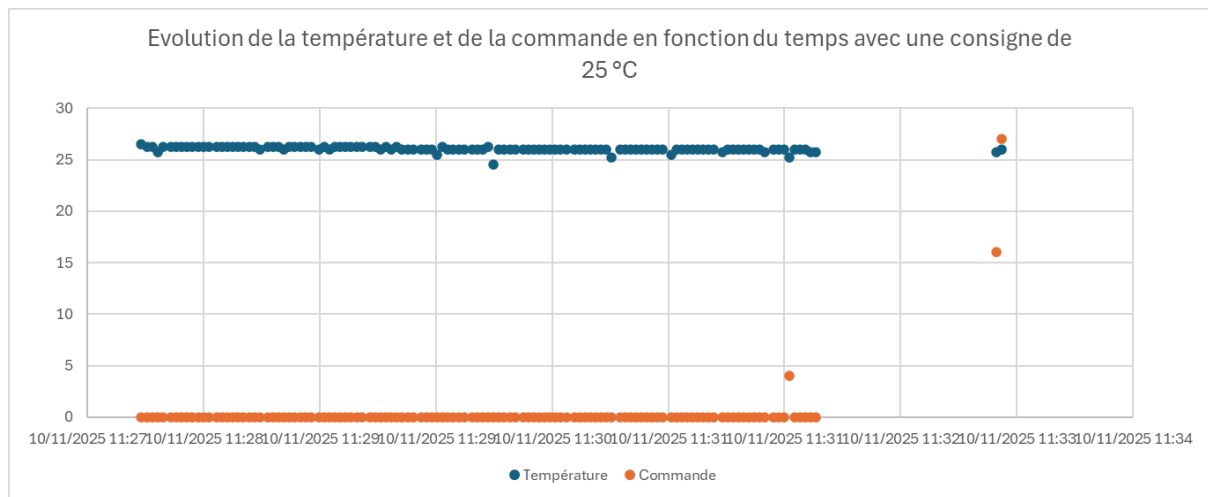


Figure 8 : Schéma représentant l'évolution de la commande et de la température de la lampe avec une consigne de 25 °C

Nous pouvons observer sur la figure 8 que nous maintenons une température de 26 °C (qui est la température de la salle) et que nous maintenons une commande de 0 % car nous souhaitons diminuer la température pour atteindre 25 °C, mais, nous sommes à la température minimale.

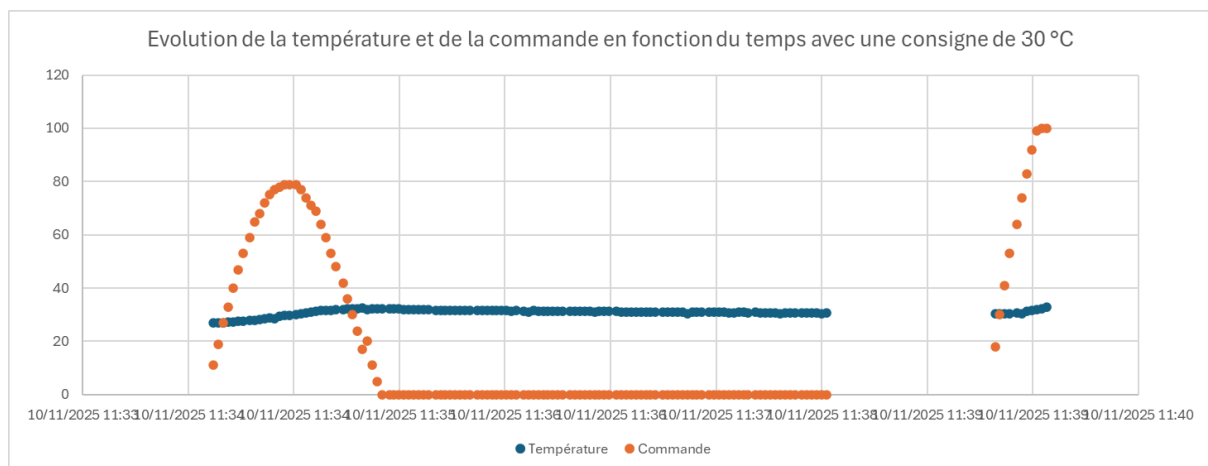


Figure 9 : Schéma représentant l'évolution de la commande et de la température de la lampe avec une consigne de 30 °C

Nous pouvons observer sur la figure 9 que nous maintenons une température de 30 °C qui est la température souhaitée. Nous remarquons également que nous avons une forte variation de la commande jusqu'à atteindre les 30 °C puis, la commande se stabilise pour maintenir cette température.

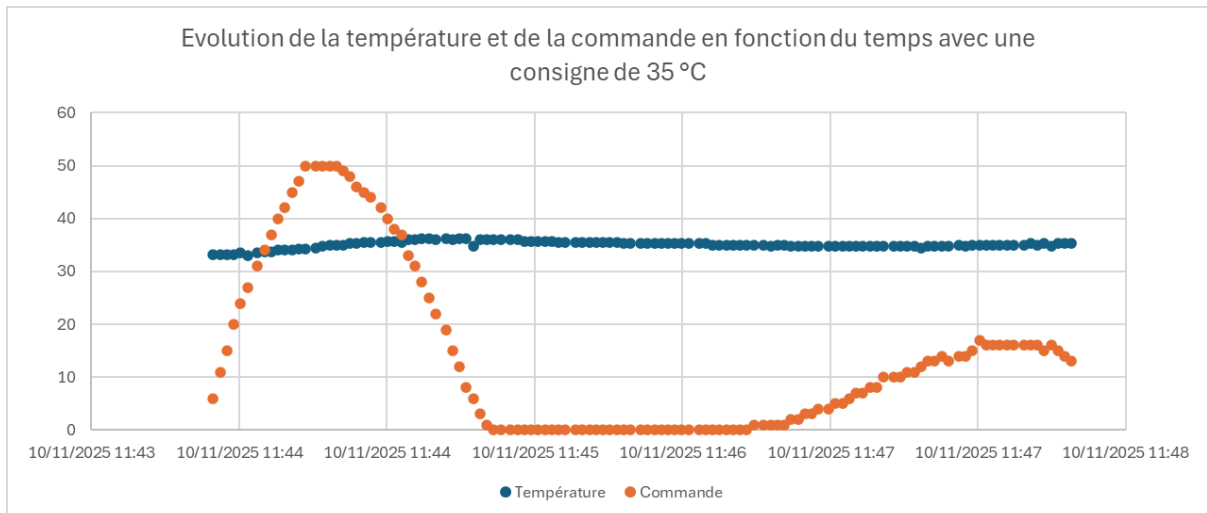


Figure 10 : Schéma représentant l'évolution de la commande et de la température de la lampe avec une consigne de 35 °C

Nous pouvons observer sur la figure 10 que nous maintenons une température de 30 °C qui est la température souhaitée. Nous remarquons également que nous avons une forte variation de la commande jusqu'à atteindre les 35 °C. Ensuite, la commande se stabilise pour maintenir cette température et nous avons une seconde variation de la commande plus faible pour maintenir la température de 35°C.

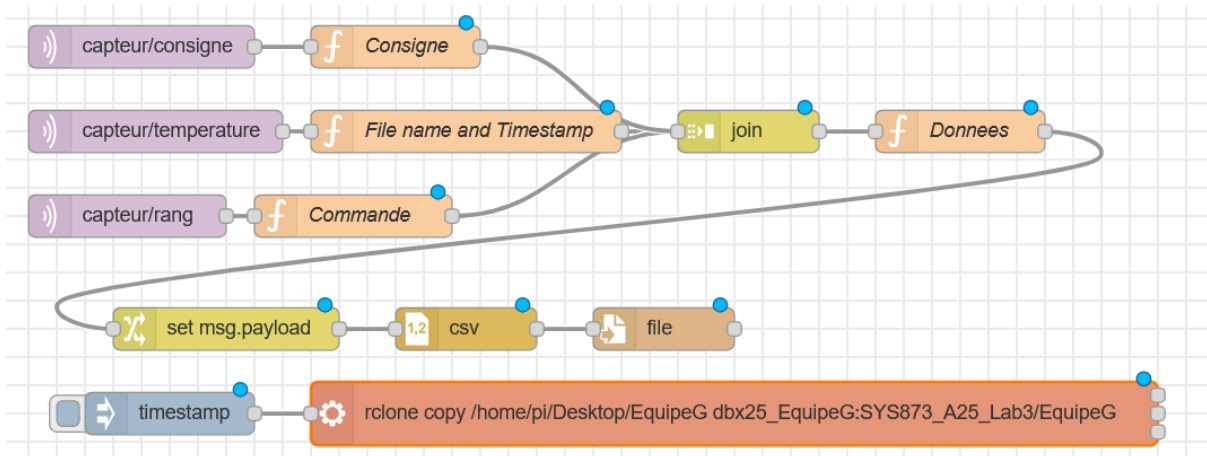


Figure 11 : Flux pour sauvegarder les données de la lampe avec une commande PID dans un fichier CSV envoyé au cloud

Nous pouvons observer sur la figure 11, le flux effectué sur Node-RED avec les différents nœuds utilisés.

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Server: localhost:1883</p> <p>Topic: capteur/consigne</p> <p>QoS: 2</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | <p>Server: localhost:1883</p> <p>Topic: capteur/temperature</p> <p>QoS: 2</p>                                                                                                                                       |
| <p>Server: localhost:1883</p> <p>Topic: capteur/rang</p> <p>QoS: 2</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | <p>Name: Consigne</p> <p>Function:</p> <pre>1 msg.consigne = msg.payload; 2 return msg;</pre>                                                                                                                       |
| <p>Name: File name and Timestamp</p> <p>Function:</p> <pre>1 let compteur = global.get("compteur")  0; 2 var now = new Date(); 3 var yyyy = now.getFullYear(); 4 var mm = now.getMonth() &lt; 9 ? "0" + (now.getMonth() + 1) : now.getMonth() + 1; 5 var dd = now.getDate() &lt; 10 ? "0" + now.getDate() : now.getDate(); 6 var hh = now.getHours() &lt; 10 ? "0" + now.getHours() : now.getHours(); 7 var mmn = now.getMinutes() &lt; 10 ? "0" + now.getMinutes() : now.getMinutes(); 8 var ss = now.getSeconds() &lt; 10 ? "0" + now.getSeconds() : now.getSeconds(); 9 msg.timestamp = yyyy + "-" + mm + "-" + dd + " " + hh + ":" + mmn + ":" + ss; 10 if(compteur===0){ 11   let filename = "/home/pi/Desktop/EquipeG/" + yyyy + "-" + mm + "-" + dd + " " + hh + ":" + mmn + ":" + ss + ".csv"; 12   global.set("filename", filename); 13   global.set("compteur",1); 14 } 15 msg.filename = global.get("filename"); 16 node.status({fill:"blue",shape:"ring",text:msg.timestamp}); 17 msg.MAC = "b8:27:eb:50:b0:24"; 18 msg.emplacement = 1; 19 msg.temperature = msg.payload; 20 return msg;</pre> | <p>Name: Commande</p> <p>Function:</p> <pre>1 msg.commande = msg.payload; 2 return msg;</pre>                                                                                                                       |
| <p>Mode: manual</p> <p>Combine each: msg. payload</p> <p>to create: a key/value Object</p> <p>using the value of: msg. topic as the key</p> <p>Send the message:</p> <ul style="list-style-type: none"> <li>After a number of message parts: 3</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | <p>Name: Donnees</p> <p>Function:</p> <pre>1 msg.consigne = msg.payload["capteur/consigne"] 2 msg.temperature = msg.payload["capteur/temperature"] 3 msg.commande = msg.payload["capteur/rang"] 4 return msg;</pre> |
| <p>Name: Name</p> <p>Rules:</p> <p>Set: msg. payload</p> <p>to: [{"Timestamp":msg.timestamp,"A ...</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | <p>Columns: Timestamp, Adresse MAC, Emplacement, Tempe</p> <p>Separator: tab</p> <p>Name: Name</p>                                                                                                                  |
| <p>Filename:</p> <p>Action: append to file</p> <p><input checked="" type="checkbox"/> Add newline (\n) to each payload?</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | <p>Command: rclone copy /home/pi/Desktop/EquipeG dbx25_Eq</p> <p>Append: <input checked="" type="checkbox"/> msg.payload</p>                                                                                        |

Figure 12 : Captures d'écran des noeuds du flux pour sauvegarder les données de la lampe avec une commande PID dans un fichier CSV envoyé au cloud

D'après la figure 12, nous commençons par recevoir les données publiées sur les topics dédiés à la température, la commande et la consigne de la lampe. Dans les fonctions Consigne et Commande, je reçois le message des MQTT in et je les mets dans msg.consigne et msg.commande. Dans la fonction File name and Timestamp, je capture la date et l'heure actuelle et je le mets dans msg.timestamp, je crée une variable globale compteur que j'initialise à 0, si il est à 0, alors je change le nom du fichier avec la date, l'heure actuelle et l'emplacement où je souhaite enregistrer le fichier et j'incrmente mon compteur de 1. De cette manière, je change une seule fois le nom de mon fichier et ainsi, un seul fichier sera créé. Je stocke dans msg.MAC, l'adresse MAC de la RaspberryPi, dans msg.emplacement, le numéro de table sur lequel je travaille et dans msg.temperature, le message reçu par le MQTT in. Dans le nœud join, je combine tous les payload reçus pour former un objet. Une fois que j'ai reçu les 3 messages des 3 topics, je transmets les valeurs pour les écrire dans le fichier CSV. Dans la fonction suivante, j'écris dans les variables dédiées les messages venant des différents topics associés. Dans le noeud set message.payload, je configure le fichier CSV avec les 6 colonnes Timestamp, Adresse MAC, Emplacement, Température, Consigne et commande avec les msg associés. Dans le nœud CSV, j'écris les colonnes dans le bon ordre : Timestamp, Adresse MAC, Emplacement, Température, Consigne et commande. Enfin, dans le fichier file je n'écris pas le nom du fichier car ceci est effectué dans la fonction File name and Timestamp. De plus, lorsque nous appuyons sur le bouton Timestamp, la commande rclone copy est effectuée pour copier les fichiers CSV de notre bureau vers le dropbox de notre groupe.

## Automatisation avec les Macros Excel

### Premières macros

Durant cette manipulation, nous devons une macro Excel capable de réaliser le tri par Date croissante, par Date décroissante, par ordre alphabétique et de supprimer les éléments triés d'une liste.



Figure 13 : Organigramme de la manipulation pour réaliser des macros Excel qui permettent de trier une liste

D'après la Figure 13, dans le programme, nous commençons par réaliser les manipulations pour trier la liste tout en enregistrant la macro, puis, nous créons un bouton et enfin, nous associons la macro à ce bouton.

## Fusionner des fichiers CSV avec les macros

Durant cette manipulation, nous devons fusionner plusieurs fichiers CSV en un seul et afficher un message lorsque la manipulation est terminée à l'aide d'une macro Excel.

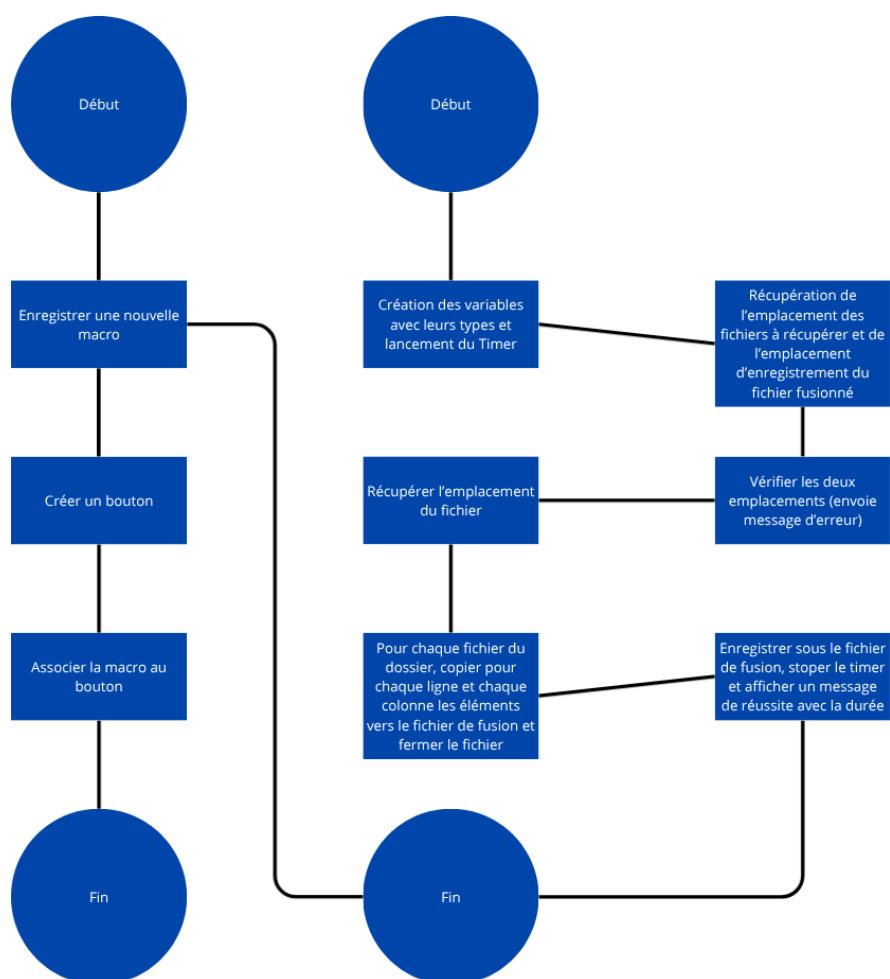


Figure 14 : Organigramme de la manipulation pour réaliser une macro Excel qui permet de fusionner des fichiers CSV

D'après la figure 14, dans le programme, nous commençons par créer une macro Excel, nous modifions son code dans lequel nous commençons par créer des variables avec leur type puis, nous lançons un timer. Ensuite, nous récupérons l'emplacement où sont stockés les fichiers à fusionner et l'emplacement où nous devons enregistrer le fichier de fusion. Après, nous vérifions l'existence de ces deux emplacements, et s'ils n'existent pas, alors nous envoyons un message d'erreur. Nous récupérons l'emplacement du premier fichier du dossier et, pour chaque fichier du dossier, nous copions chaque élément de chaque ligne et de chaque colonne vers le fichier de fusion. Puis, nous fermons le fichier du dossier. Enfin, nous enregistrons sous le fichier de fusion à l'emplacement qu'on a indiqué, nous stoppons le timer et nous affichons un message de réussite avec le temps d'exécution. Pour finir, nous créons un bouton et nous associons la macro Excel créée à ce bouton.

## Interface Graphique avec Node-RED

### Premier tableau de bord

Durant cette manipulation, nous devons concevoir une interface graphique permettant d'ajuster la température de commande de la lampe, de montrer la température de la lampe sur un cadran et d'afficher dans un graphique la température relevée durant les 15 dernières minutes.

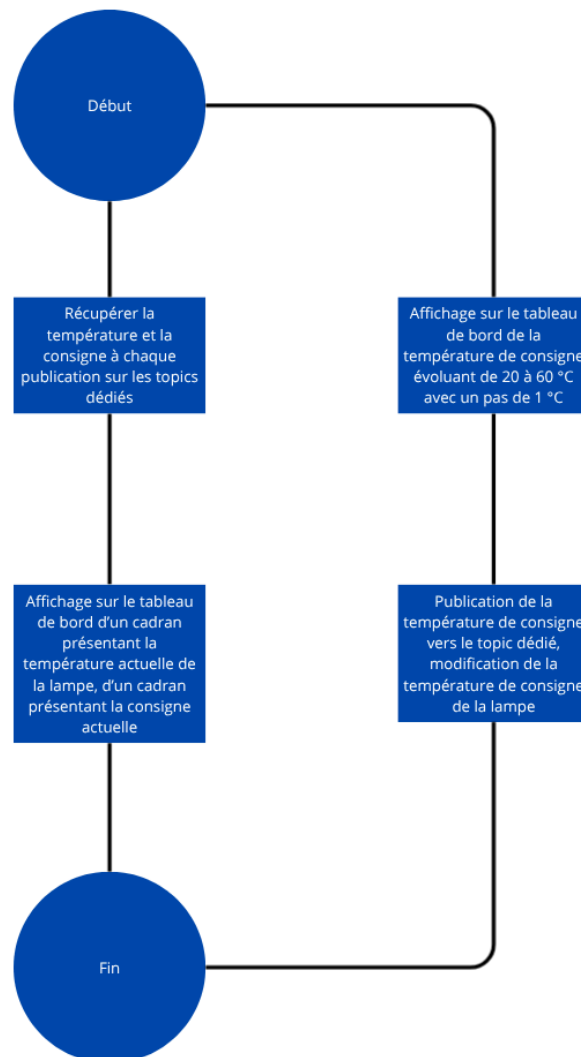


Figure 15 : Organigramme du noeud pour afficher sur le tableau de bord les données sur la lampe

D'après la figure 15, nous commençons par récupérer la température de la lampe et de la consigne à chaque publication sur leurs topics dédiés, puis nous affichons sur un cadran la température actuelle de la lampe, et, dans un graphique, la température de la lampe au cours des 15 dernières minutes. De plus, nous affichons sur un cadran la température de consigne actuelle de la lampe et nous affichons une entrée de texte permettant de modifier la température de la lampe entre 20 et 60 °C avec un pas de 1 °C.

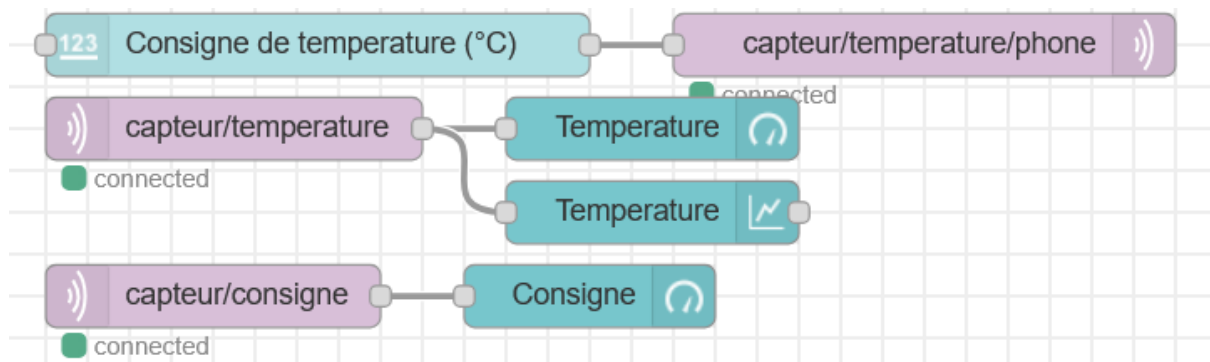


Figure 16 : Flux pour afficher la consigne et la température d'une lampe sur un tableau de bord

Nous pouvons observer sur la figure 16 le flux effectué sur Node-RED avec les différents nœuds utilisés.

|                                                                                                                                                                                                                                        |                                                                                                                                                                                                                         |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Group</b> [Laboratoire3] Lampe</p> <p><b>Size</b> auto</p> <p><b>Label</b> Consigne de temperature (°C)</p> <p><b>Tooltip</b> optional tooltip</p> <p><b>Value Format</b> {{value}}</p> <p><b>Range</b> min 20 max 80 step 1</p> | <p><b>Server</b> localhost:1883</p> <p><b>Topic</b> capteur/temperature/phone</p>                                                                                                                                       |
| <p><b>Server</b> localhost:1883</p> <p><b>Topic</b> capteur/temperature</p> <p><b>QoS</b> 2</p>                                                                                                                                        | <p><b>Group</b> [Laboratoire3] Lampe</p> <p><b>Size</b> auto</p> <p><b>Type</b> Gauge</p> <p><b>Label</b> Temperature</p> <p><b>Value format</b> {{value}}</p> <p><b>Units</b> °C</p> <p><b>Range</b> min 20 max 60</p> |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                                                                                                                             |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div> <div>Group</div> <div>[Laboratoire3] Lampe</div> </div> <div> <div>Size</div> <div>auto</div> </div> <div> <div>Label</div> <div>Temperature</div> </div> <div> <div>Type</div> <div>Line chart</div> <div><input type="checkbox"/> enlarge points</div> </div> <div> <div>X-axis</div> <div>last 15 minutes</div> <div>OR</div> <div>450 points</div> </div> <div> <div>X-axis Label</div> <div>HH:mm:ss</div> </div> <div> <div>Y-axis</div> <div>min 20</div> <div>max 60</div> </div> | <div> <div>Server</div> <div>localhost:1883</div> </div> <div> <div>Topic</div> <div>capteur/consigne</div> </div> <div> <div>QoS</div> <div>2</div> </div> |
| <div> <div>Group</div> <div>[Laboratoire3] Lampe</div> </div> <div> <div>Size</div> <div>auto</div> </div> <div> <div>Type</div> <div>Gauge</div> </div> <div> <div>Label</div> <div>Consigne</div> </div> <div> <div>Value format</div> <div>{{value}}</div> </div> <div> <div>Units</div> <div>°C</div> </div> <div> <div>Range</div> <div>min 20</div> <div>max 60</div> </div>                                                                                                              |                                                                                                                                                             |

Figure 17 : Captures d'écran des noeuds du flux pour afficher les données d'une lampe sur un tableau de bord

D'après la figure 17, nous commençons par recevoir les données publiées sur les topics dédiés à la température et la consigne de la lampe. Ensuite, nous affichons sur le tableau de bord du laboratoire 3 dans l'onglet Lampe, un cadran affichant la température entre 20 et 60 °C, un graphique présentant la température de lampe au cours des 15 dernières minutes entre 20 et 60 °C et un cadran affichant la consigne entre 20 et 60 °C. De plus, nous avons un texte d'entrée entre 20 et 80°C avec une incrémentation de 1°C pour définir la température de consigne de la lampe. Ensuite, cette donnée est publiée sur le topic capteur/temperature/phone qui actualise la nouvelle température de consigne.

## Laboratoire3

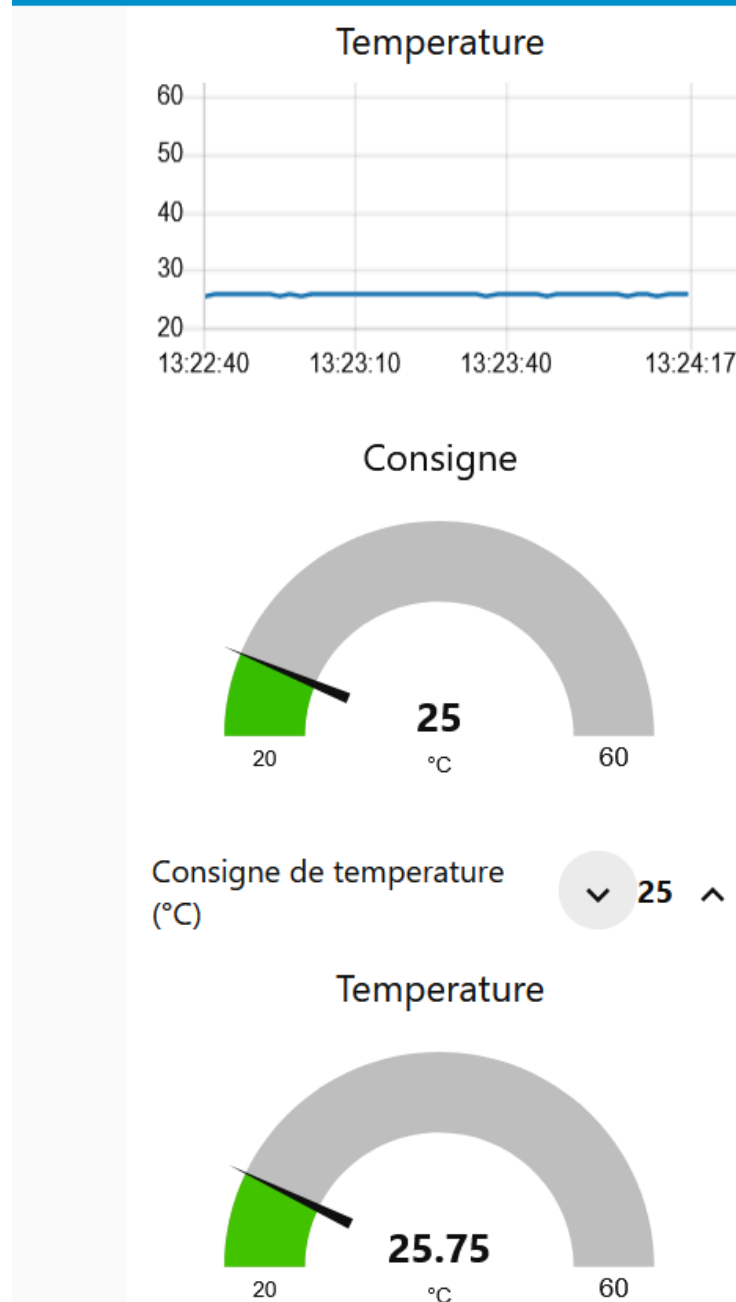


Figure 18 : Captures d'écran du tableau de bord présentant les données de la lampe

Nous observons sur la figure 18, dans la page laboratoire 3, dans l'onglet Lampe (non visible à cause de la taille du tableau de bord): le cadran présentant la température de la lampe, le cadran présentant la température de consigne, et le graphique présentant l'évolution de la température de la lampe au cours des 15 dernières minutes.

## Importer des données à partir du Cloud

Durant cette manipulation, nous devons concevoir une interface graphique permettant d'afficher la température, l'humidité et l'adresse MAC de notre capteur bluetooth, la date et l'heure actuelle, la température, l'humidité et la vitesse du vent de la ville de Montréal et générer un tableau retraçant toutes les données et qui s'actualise à chaque modification d'une donnée.

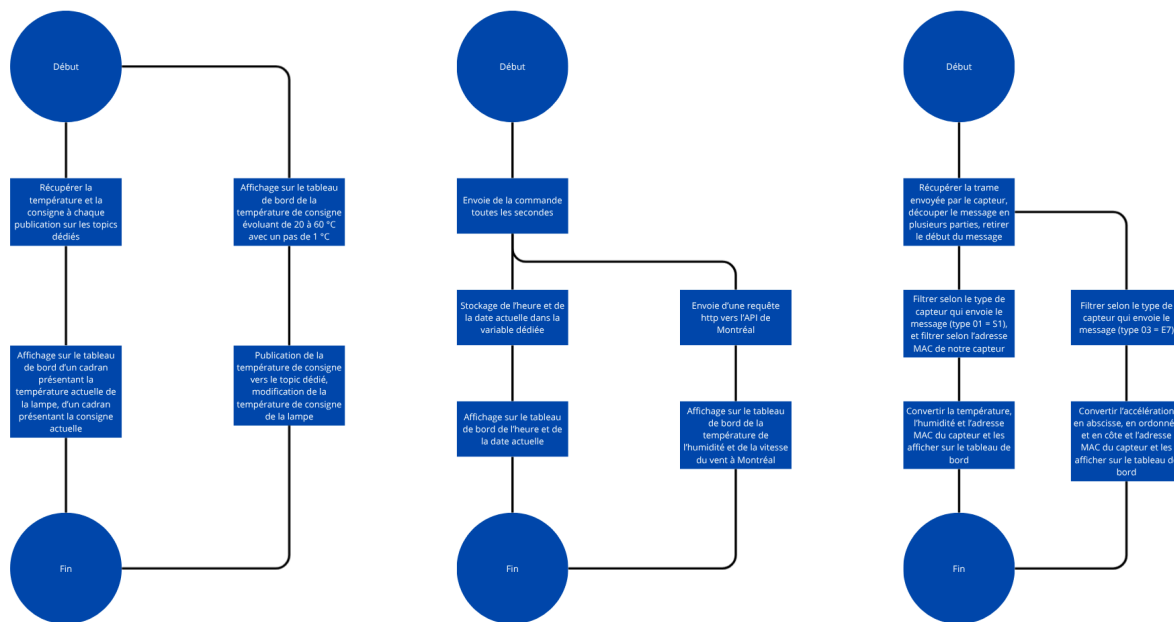


Figure 19 : Organigramme du noeud pour afficher sur le tableau de bord les données sur la lampe, l'heure et la date actuelle, les informations sur Montréal et les données des capteurs S1 et E7

D'après la figure 19, dans le programme, pour la lampe, nous commençons par récupérer les données publiées sur les topics dédiés. Ensuite, nous affichons les données sur le tableau de bord. De plus, la température de consigne indiquée sur le tableau de bord est publiée sur le topic dédié pour actualiser la température de consigne. Pour la date et l'heure ainsi que les informations sur Montréal, nous envoyons un timestamp toutes les secondes qui permet de capturer la date et l'heure actuelle et d'envoyer une requête http vers l'API de Montréal. Nous affichons ensuite toutes les données sur le tableau de bord. Enfin, pour les capteurs S1 et E7, nous récupérons la trame envoyée par le websocket, nous la découpons et enlevons le début. Ensuite, nous enregistrons les différentes données dans les variables associées, puis pour le capteur S1, nous filtrons selon le type de capteur (01) et selon l'adresse MAC. Après, nous convertissons les données, et pour le capteur E7, nous filtrons selon le type de capteur (03) et nous convertissons les données. Enfin, nous affichons toutes les données sur le tableau de bord, c'est à dire la température, l'humidité et l'adresse MAC pour le capteur S1 et l'accélération selon l'axe des abscisses, selon l'axe des ordonnées et l'axe des côtes et l'adresse MAC pour le capteur E7.

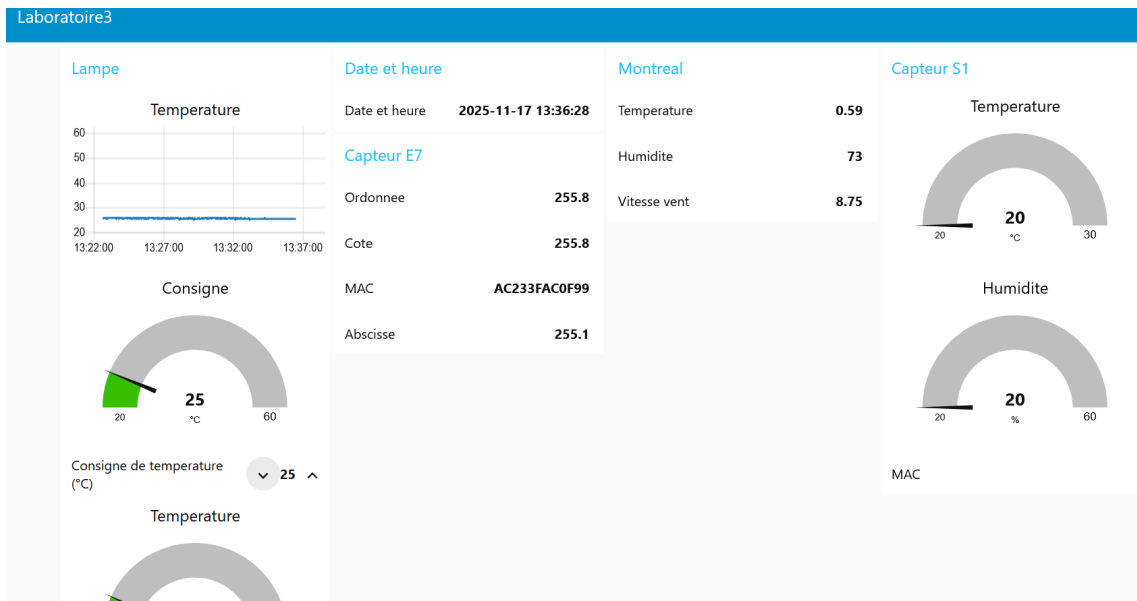


Figure 20 : Captures d'écran du tableau de bord final (après la séance du 17/11, le capteur S1 était éteint)

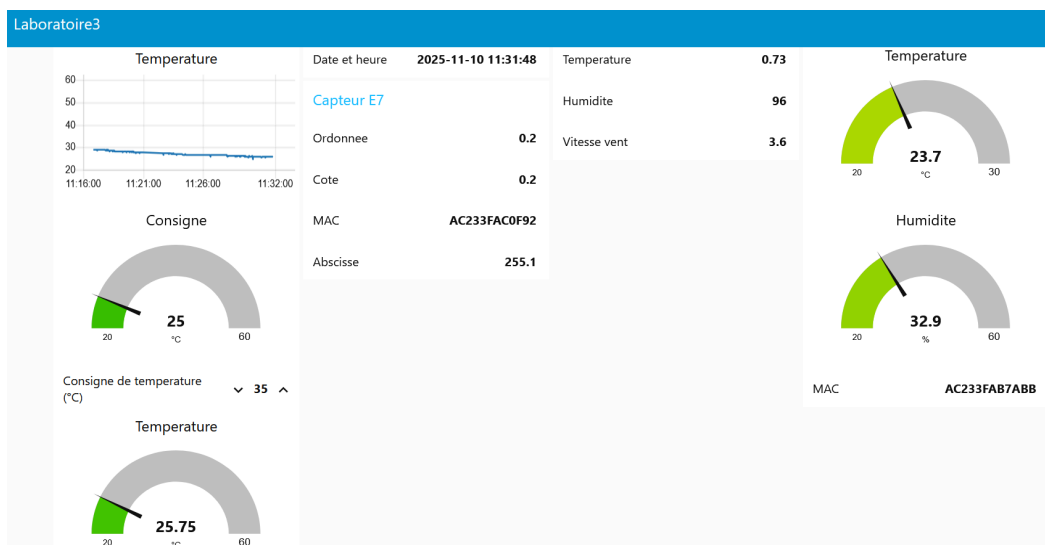


Figure 21 : Captures d'écran du tableau de bord final (avant la séance du 17/11, le capteur S1 était allumé)

Nous observons sur les figures 20 et 21, le tableau de bord laboratoire 3 avec l'onglet Lampe présentant la température et la consigne de celle-ci avec 2 cadrans, l'évolution de la température de la lampe au cours des 15 dernières minutes et le contrôle de la consigne de 20 à 80°C avec un pas de 1. Dans l'onglet date et heure, nous avons la date et l'heure actuelle. Dans l'onglet Capteur E7, nous avons l'accélération en abscisse, en ordonnée et en côte du capteur et nous avons son adresse MAC. Dans l'onglet Montréal, nous avons la température extérieure, l'humidité et la vitesse du vent dans la ville de Montréal. Enfin, dans l'onglet Capteur S1, nous avons la température et l'humidité relevées sur 2 cadrans et nous avons son adresse MAC affichée. Sur la figure 20, les données ont

été relevées après la séance du 17/11 donc le capteur S1 était éteint. Ainsi, les données du capteur S1 sont nulles, tandis que les données du capteur S1 sur la figure 21 sont non-nulles étant donné que les données ont été relevées avant la séance du 17/11, par conséquent, lorsque le capteur S1 était encore allumé.

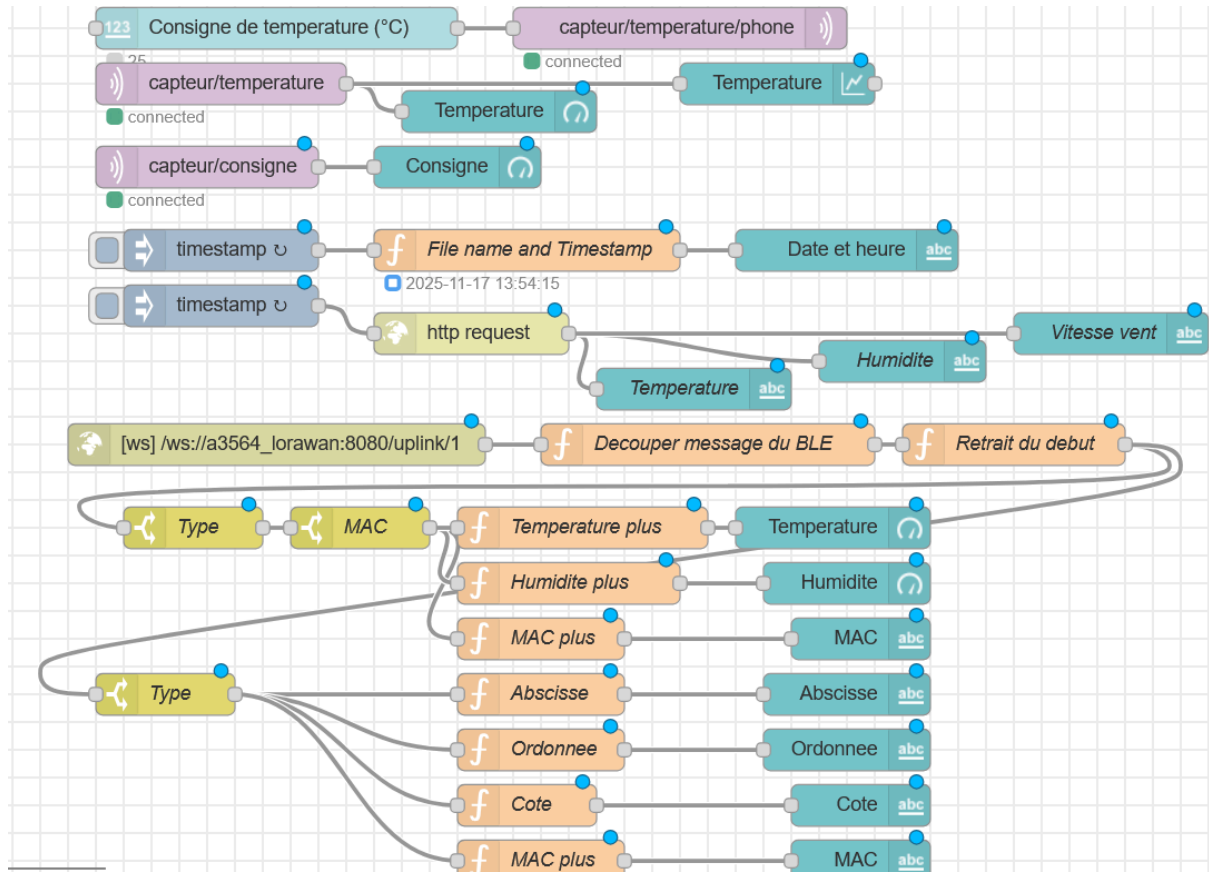


Figure 22 : Flux pour afficher les données de la lampe, du capteur S1, du capteur E7, la date et l'heure et les données de Montréal sur un tableau de bord.

Nous pouvons observer sur la figure 22 le flux effectué sur Node-RED avec les différents nœuds utilisés.

|                                                                                                                                                                         |                                                            |                                                                |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|----------------------------------------------------------------|
| Group: [Laboratoire3] Lampe<br>Size: auto<br>Label: Consigne de température (°C)<br>Tooltip: optional tooltip<br>Value Format: {{value}}<br>Range: min 20 max 80 step 1 | Server: localhost:1883<br>Topic: capteur/temperature/phone | Server: localhost:1883<br>Topic: capteur/temperature<br>QoS: 2 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------|----------------------------------------------------------------|

|                                                                                                                                                                               |                                                                                                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Group: [Laboratoire3] Lampe</p> <p>Size: auto</p> <p>Type: Gauge</p> <p>Label: Temperature</p> <p>Value format: {{value}}</p> <p>Units: °C</p> <p>Range: min 20 max 60</p> | <p>Group: [Laboratoire3] Lampe</p> <p>Size: auto</p> <p>Label: Temperature</p> <p>Type: Line chart <input type="checkbox"/> enlarge points</p> <p>X-axis: last 15 minutes OR 450 points</p> <p>X-axis Label: HH:mm:ss</p> <p>Y-axis: min 20 max 60</p>                                                | <p>Server: localhost:1883</p> <p>Topic: capteur/consigne</p> <p>QoS: 2</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| <p>Group: [Laboratoire3] Lampe</p> <p>Size: auto</p> <p>Type: Gauge</p> <p>Label: Consigne</p> <p>Value format: {{value}}</p> <p>Units: °C</p> <p>Range: min 20 max 60</p>    | <p>Payload: timestamp</p> <p>Topic:</p> <p><input checked="" type="checkbox"/> Inject once after 0.1 seconds, then</p> <p>Repeat: interval every 1 seconds</p>                                                                                                                                        | <p>Name: File name and Timestamp</p> <p>Function:</p> <pre> 1 var now = new Date(); 2 var yyyy = now.getFullYear(); 3 var mm = now.getMonth() &lt; 9 ? "0" + now.getMonth() : now.getMonth(); 4 var dd = now.getDate() &lt; 10 ? "0" + now.getDate() : now.getDate(); 5 var hh = now.getHours() &lt; 10 ? "0" + now.getHours() : now.getHours(); 6 var mm = now.getMinutes() &lt; 10 ? "0" + now.getMinutes() : now.getMinutes(); 7 var ss = now.getSeconds() &lt; 10 ? "0" + now.getSeconds() : now.getSeconds(); 8 msg.timestamp = yyyy + "-" + mm + "-" + dd + " " + hh + ":" + mm + ":" + ss; 9 node.status({fill:"blue",shape:"ring",text:msg.timestamp}); 10 return msg; </pre>                                                                                                                              |
| <p>Group: [Laboratoire3] Date et heure</p> <p>Size: auto</p> <p>Label: Date et heure</p> <p>Value format: {{msg.timestamp}}</p>                                               | <p>Payload: timestamp</p> <p>Topic:</p> <p><input checked="" type="checkbox"/> Inject once after 0.1 seconds, then</p> <p>Repeat: interval every 1 seconds</p>                                                                                                                                        | <p>Method: GET</p> <p>URL: https://api.openweathermap.org/data/2.5/weather</p> <p><input type="checkbox"/> Append msg.payload as query string parameters</p> <p><input type="checkbox"/> Enable secure (SSL/TLS) connection</p> <p><input type="checkbox"/> Use authentication</p> <p><input type="checkbox"/> Enable connection keep-alive</p> <p><input type="checkbox"/> Use proxy</p> <p>Return: a parsed JSON object</p>                                                                                                                                                                                                                                                                                                                                                                                      |
| <p>Group: [Laboratoire3] Montreal</p> <p>Size: auto</p> <p>Label: Temperature</p> <p>Value format: {{msg.payload.main.temp}}</p>                                              | <p>Group: [Laboratoire3] Montreal</p> <p>Size: auto</p> <p>Label: Humidite</p> <p>Value format: {{msg.payload.main.humidity}}</p>                                                                                                                                                                     | <p>Group: [Laboratoire3] Montreal</p> <p>Size: auto</p> <p>Label: Vitesse vent</p> <p>Value format: {{msg.payload.wind.speed}}</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <p>Type: Listen on</p> <p>Path: /ws://a3564_lorawan:8080/uplink/1</p> <p>Name: Name</p>                                                                                       | <p>Name: Decouper message du BLE</p> <p>Function:</p> <pre> 1 var s = msg.payload.toString(); 2 3 var data = s.substr(3); 4 msg = []; 5 var tags = data.split(" "); 6 for (var w in tags){ 7   context.global.row = context.global.row+1; 8   msg.push({payload:tags[w]}) 9 } 10 return [msg]; </pre> | <p>Name: Retrait du debut</p> <p>Function:</p> <pre> 1 var data = msg.payload.substr(24); 2 msg.payload = data; 3 msg.type = data.substr(0,2); 4 msg.batt = data.substr(2,4); 5 msg.abcisse = data.substr(4,8); 6 msg.temperature = data.substr(4,8); 7 msg.humidite = data.substr(8,12); 8 msg.ordonnee = data.substr(8,12); 9 msg.cote = data.substr(12,16); 10 msg.MAC1 = data.substr(12,14); 11 msg.MAC2 = data.substr(14,16); 12 msg.MAC3 = data.substr(16,18); 13 msg.MAC4 = data.substr(16,18); 14 msg.MAC5 = data.substr(18,22); 15 msg.MAC6 = data.substr(20,22); 16 msg.MAC12 = data.substr(16,18); 17 msg.MAC22 = data.substr(18,20); 18 msg.MAC32 = data.substr(20,22); 19 msg.MAC42 = data.substr(22,24); 20 msg.MAC52 = data.substr(24,26); 21 msg.MAC62 = data.substr(26,28); 22 return msg; </pre> |
| <p>Name: Type</p> <p>Property: msg.type</p> <p>Value: 01</p>                                                                                                                  | <p>Name: MAC</p> <p>Property: msg.MAC</p> <p>Value: BB7AAB3F23AC</p>                                                                                                                                                                                                                                  | <p>Name: Temperature plus</p> <p>Function:</p> <pre> 1 t = (parseInt(msg.temperature,16))/256; 2 t = t.toFixed(1); 3 msg.payload = t; 4 return msg; </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |

|                                                                                                                                                     |                                                                                                                                       |                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Group: [Laboratoire3] Capteur S1<br>Size: auto<br>Type: Gauge<br>Label: Temperature<br>Value format: {{value}}<br>Units: °C<br>Range: min 20 max 30 | Name: Humidite plus<br>Function: <pre>1 t = (parseInt(msg.humidite,16))/256; 2 t = t.toFixed(1) 3 msg.payload = t 4 return msg;</pre> | Group: [Laboratoire3] Capteur S1<br>Size: auto<br>Type: Gauge<br>Label: Humidite<br>Value format: {{value}}<br>Units: %<br>Range: min 20 max 60 |
| Name: MAC plus<br>Function: <pre>1 t = msg.MAC6+msg.MAC5+msg.MAC4+msg.MAC3+msg.MAC2+msg.MAC1; 2 return {payload: t};</pre>                          | Group: [Laboratoire3] Capteur S1<br>Size: auto<br>Label: MAC<br>Value format: {{msg.payload}}                                         | Name: Type<br>Property: msg.type<br>Value: 03                                                                                                   |
| Name: Abscisse<br>Function: <pre>1 t = (parseInt(msg.abscisse,16))/256; 2 t = t.toFixed(1) 3 msg.payload = t 4 return msg;</pre>                    | Group: [Laboratoire3] Capteur E7<br>Size: auto<br>Label: Abscisse<br>Value format: {{msg.payload}}                                    | Name: Ordonnee<br>Function: <pre>1 t = (parseInt(msg.ordonnee,16))/256; 2 t = t.toFixed(1) 3 msg.payload = t 4 return msg;</pre>                |
| Group: [Laboratoire3] Capteur E7<br>Size: auto<br>Label: Ordonnee<br>Value format: {{msg.payload}}                                                  | Name: Cote<br>Function: <pre>1 t = (parseInt(msg.ordonnee,16))/256; 2 t = t.toFixed(1) 3 msg.payload = t 4 return msg;</pre>          | Group: [Laboratoire3] Capteur E7<br>Size: auto<br>Label: Cote<br>Value format: {{msg.payload}}                                                  |
| Name: MAC plus<br>Function: <pre>1 t = msg.MAC62+msg.MAC52+msg.MAC42+msg.MAC32+msg.MAC22+msg.MAC12; 2 return {payload: t};</pre>                    | Group: [Laboratoire3] Capteur E7<br>Size: auto<br>Label: MAC<br>Value format: {{msg.payload}}                                         |                                                                                                                                                 |

Figure 23 : Captures d'écran des nœuds du flux pour afficher les données de la lampe, du capteur S1, du capteur E7, la date et l'heure et les données de Montréal sur un tableau de bord.

D'après la figure 23, pour l'onglet lampe, nous commençons par recevoir les données publiées sur les topics dédiés à la température et la consigne de la lampe. Ensuite, nous affichons sur le tableau de bord du laboratoire 3 dans l'onglet Lampe, un cadran affichant la température entre 20 et 60 °C, un graphique présentant la température de lampe au cours des 15 dernières minutes entre 20 et 60 °C et un cadran affichant la consigne entre 20 et 60 °C. De plus, nous avons un texte d'entrée entre 20 et 80°C avec une incrémentation de 1°C pour définir la température de consigne de la lampe. Ensuite, cette donnée est publiée sur le topic capteur/temperature/phone qui actualise la nouvelle température de consigne.

Pour l'onglet Date et heure, nous commençons par injecter un timestamp toutes les secondes. Ensuite, dans la fonction File name and Timestamp, nous capturons l'heure et la

date actuelle et nous les enregistrons dans msg.timestamp. Enfin nous les affichons dans le tableau de bord.

Pour l'onglet Montréal, nous commençons par injecter un timestamp toutes les secondes. Ensuite, nous effectuons une requête http vers l'API de montréal pour recevoir un objet json, puis, nous affichons sur le tableau de bord la température, l'humidité et la vitesse du vent de la ville de Montréal.

Pour l'onglet Capteur S1, nous commençons par nous connecter au websocket, ensuite, nous découpons le message du BLE, nous retirons le début en stockant dans différentes variables le type de capteur, les différents bits de l'adresse MAC, la température et l'humidité du capteur. Ensuite, nous vérifions que c'est un capteur S1 donc de type 01, et nous vérifions que son adresse MAC correspond à celle indiqué sur le capteur reçu. Puis, nous convertissons les valeurs de température et d'humidité pour obtenir les bons résultats et nous remettons l'adresse MAC dans le bon ordre. Enfin, nous affichons toutes ces données sur le tableau de bord.

Pour l'onglet Capteur E7, nous commençons par nous connecter au websocket, ensuite, nous découpons le message du BLE, nous retirons le début en stockant dans différentes variables le type de capteur, les différents bits de l'adresse MAC, et les accélérations du capteur selon l'axe des abscisses, des ordonnées et de la côte. Ensuite, nous vérifions que c'est un capteur E7 donc de type 03. Puis, nous convertissons les valeurs des accélérations du capteur selon l'axe des abscisses, des ordonnées et de la côte pour obtenir les bons résultats et nous remettons l'adresse MAC dans le bon ordre. Enfin, nous affichons toutes ces données sur le tableau de bord.

# Conclusion

Ce laboratoire nous aura permis de poursuivre la conception de notre première couche réseau IIoT. Nous avons appris à créer un fichier de type CSV et à le modifier en respectant un certain délai à l'aide de NodeRED et à le sauvegarder dans le cloud.

De plus, nous avons appris à récupérer les données publiées par un capteur monté en bluetooth.

Ensuite, nous avons appris à réaliser des macros Excel, notamment pour fusionner plusieurs fichiers CSV en un seul.

Enfin, nous avons appris à réaliser un tableau de bord répertoriant les différentes données relevées précédemment.

Nous avons appris à utiliser les logiciels NodeRED et Excel pour réaliser les différentes tâches vues dans les précédents laboratoires.

## Expérience personnelle

Je pense que ce laboratoire est bien agencé. Il permet de maîtriser le logiciel NodeRED pour réaliser des fichiers CSV et pour les stocker dans le cloud et pour réaliser un tableau de bord. De plus, il permet de maîtriser le logiciel Excel en réalisant des macros.

Ce laboratoire m'a permis de découvrir comment récupérer les données d'un capteur monté en bluetooth. De plus, il m'a permis de me familiariser avec le logiciel NodeRED et enfin, il m'a permis d'apprendre comment réaliser des macros Excel.

Enfin, ce laboratoire m'a donné la possibilité de mobiliser les connaissances précédemment acquises et les programmes précédemment implémentés pour réaliser de nouvelles tâches. Il m'a aussi donné l'opportunité de découvrir que mes connaissances précédemment acquises me permettent de réaliser des tâches plus complexes.

En conclusion, je dirais que ce laboratoire est pertinent dans la mesure où il permet de montrer l'intérêt des connaissances acquises précédemment et de les utiliser dans un projet concret. Selon moi, c'est un troisième laboratoire intéressant pour mobiliser ses connaissances, et ainsi réaliser des tâches plus complexes que dans le premier laboratoire, tout en utilisant les programmes précédemment implémentés.

```

Sub Fusion()
,
' Fusion Macro
,
' Touche de raccourci du clavier: Ctrl+y
,

    Dim monChemin As String
    Dim chemin_Cible As String
    Dim monFichier As String
    Dim wb As Workbook
    Dim ws As Worksheet
    Dim fichier_actuel As Workbook
    Dim derniereLigne As Long
    Dim derniereColonne As Long
    Dim ligneCollage As Long
    Dim debut As Double
    Dim duree As Double

    debut = Timer

monChemin = ActiveWorkbook.Sheets(1).Range("B3").Value & "\"
chemin_Cible = ActiveWorkbook.Sheets(1).Range("B4").Value & "\"

If Dir(monChemin, vbDirectory) = "" Then
    MsgBox "Le dossier des fichiers CSV n'existe pas : " & monChemin, vbCritical, "Erreur de chemin"
    Exit Sub
End If

If Dir(chemin_Cible, vbDirectory) = "" Then
    MsgBox "Le dossier de sauvegarde n'existe pas : " & chemin_Cible, vbCritical, "Erreur de chemin"
    Exit Sub
End If

Set wb = Workbooks.Add
Set ws = wb.Sheets(1)
ws.Name = "Fusion_CSV"

monFichier = Dir(monChemin & "*.csv")

Do While monFichier <> ""
Set fichier_actuel = Workbooks.Open(Filename:=monChemin & monFichier)
derniereLigne      =      fichier_actuel.Sheets(1).Cells(fichier_actuel.Sheets(1).Rows.Count,
1).End(xlUp).Row
derniereColonne          =          fichier_actuel.Sheets(1).Cells(1,
fichier_actuel.Sheets(1).Columns.Count).End(xlToLeft).Column
For r = 1 To derniereLigne

```

```

    For c = 1 To derniereColonne
        ws.Cells(ws.Cells(ws.Rows.Count, 1).End(xlUp).Row + 1, c).Value =
fichier_actuel.Sheets(1).Cells(r, c).Value
    Next c
Next r
fichier_actuel.Close SaveChanges:=False
monFichier = Dir()
Loop
wb.SaveAs Filename:=chemin_Cible & "EquipeG_CSV_Fusion.xlsx"
duree = Timer - debut
MsgBox "Tous les fichiers CSV ont été fusionnés avec succès !" & vbCrLf & _
    "Durée : " & Round(duree, 2) & " secondes.", vbInformation, "Fusion terminée"
End Sub

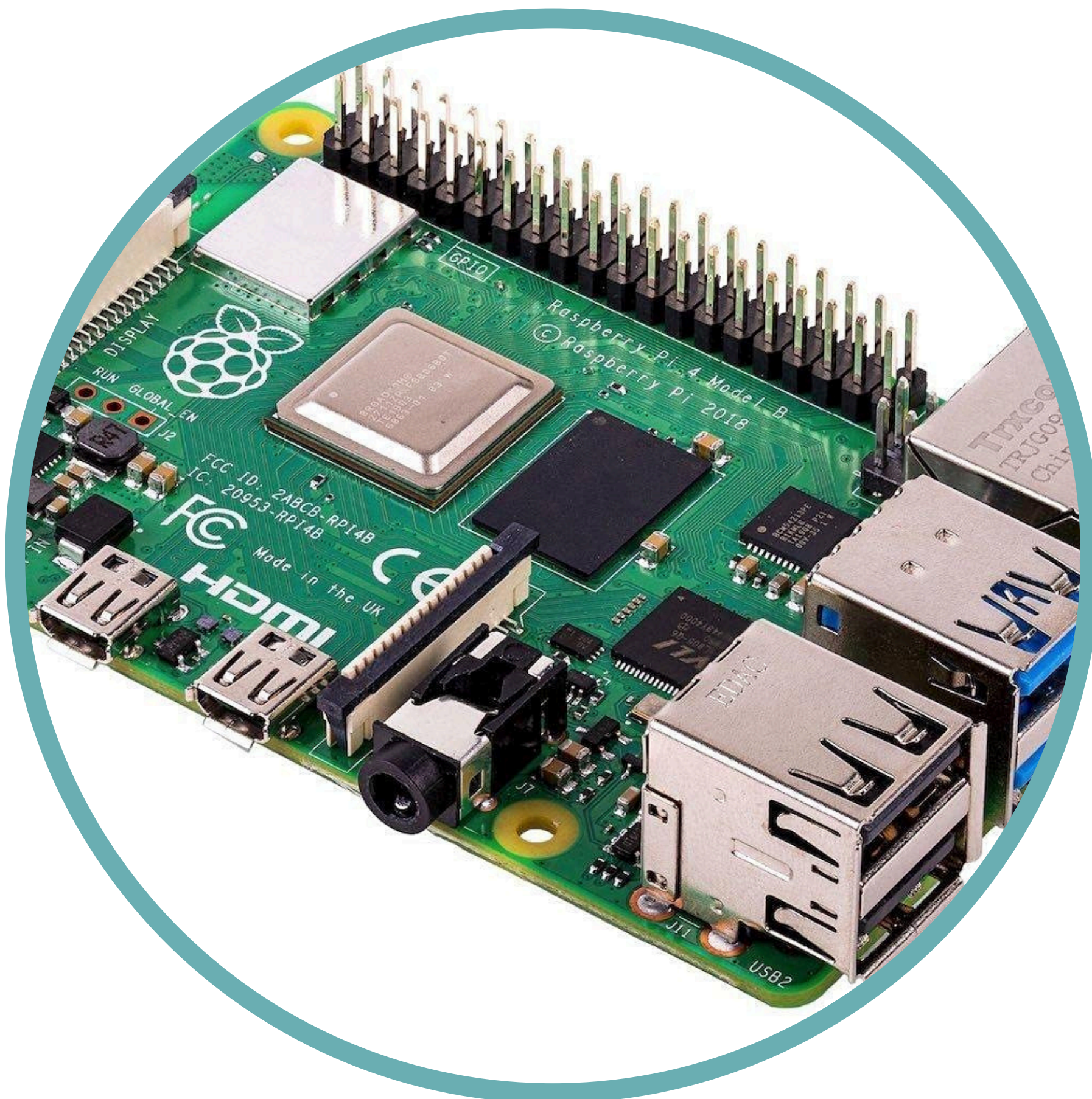
```

#### Annexe 1 : Code de la macro de fusion de fichiers CSV

Du 17/11 au 01/12

Rapport de laboratoire

Enseignant laboratoire : Bitouh Amar  
Enseignant cours : Lokman Sboui



Matys Benoist  
Programme court de 1er cycle 2025-2026

**ÉTS**

Le génie pour l'industrie

**ÉCOLE DE  
TECHNOLOGIE  
SUPÉRIEURE**

Université du Québec

**PIRANHAS  
ÉTS**



# Table des matières

|                                                                                              |           |
|----------------------------------------------------------------------------------------------|-----------|
| <b>Table des matières</b>                                                                    | <b>2</b>  |
| <b>Table des figures</b>                                                                     | <b>3</b>  |
| <b>Buts et objectifs du laboratoire</b>                                                      | <b>4</b>  |
| <b>Démarche de conception</b>                                                                | <b>5</b>  |
| <b>Affichage du tableau de bord Node-RED à distance</b>                                      | <b>5</b>  |
| Affichage catégorique des données avec Power BI                                              | 7         |
| Les limitations avec Excel                                                                   | 7         |
| Les avantages de Power BI                                                                    | 9         |
| <b>Modélisation basée sur les données</b>                                                    | <b>11</b> |
| Estimation de la durée de vie utile avec réseau de neurones                                  | 15        |
| <b>Conclusion</b>                                                                            | <b>17</b> |
| <b>Expérience personnelle</b>                                                                | <b>17</b> |
| Annexe 1 : Code MATLAB pour la modélisation basée sur les données                            | 21        |
| Annexe 2 : Code MATLAB pour l'estimation de la durée de vie utile avec un réseau de neurones | 26        |

# Table des figures

|                                                                                                                                                              |    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 1 : Organigramme pour se connecter au tableau de bord avec l'application Remote-RED                                                                   | 5  |
| Figure 2 : Captures d'écran du tableau de bord sur Remote-RED                                                                                                | 6  |
| Figure 3 : Organigramme de l'affichage de données d'un capteur sur Excel                                                                                     | 7  |
| Figure 4 : Graphique présentant l'évolution de la température perçue par le capteur 494208004C67 en fonction du temps la journée du 23/08/2022               | 8  |
| Figure 5 : Graphique présentant l'évolution de l'humidité perçue par le capteur AC233FA21248 en fonction du temps la journée du 09/08/2022                   | 8  |
| Figure 6 : Organigramme pour créer un tableau de bord sur Power BI                                                                                           | 9  |
| Figure 7 : Capture d'écran du tableau de bord sur Power BI                                                                                                   | 10 |
| Figure 8 : Organigramme du programme MATLAB pour une modélisation basée sur les données                                                                      | 11 |
| Figure 9 : Graphique représentant le courant, le voltage et les températures de mesure à travers un cycle de décharge pour la première pile                  | 12 |
| Figure 10 : Graphique représentant le voltage, la température de mesure et la capacité de décharge à travers un cycle de décharge pour la première pile      | 12 |
| Figure 11 : Graphique représentant l'évolution de la température et de la capacité de décharge d'une pile en fonction de son voltage après une interpolation | 13 |
| Figure 12 : Graphique représentant l'entraînement du réseau de neurones                                                                                      | 13 |
| Figure 13 : Graphique représentant la durée de vie restante réelle contre la durée de vie restante prédite                                                   | 14 |
| Figure 14 : Architecture du réseau                                                                                                                           | 14 |
| Figure 15 : Mesures de performances du modèle                                                                                                                | 14 |
| Figure 16 : Organigramme du programme MATLAB pour une estimation de la durée de vie utile de moteurs                                                         | 15 |
| Figure 17 : Captures d'écran de graphiques de tests aléatoires de prédiction de durée de vie des moteurs                                                     | 16 |

# Buts et objectifs du laboratoire

Le but de ce laboratoire est de continuer le travail sur le tableau de bord conçu dans le précédent laboratoire et d'avoir une introduction à la couche affaire. Ainsi, nous allons voir comment réaliser un tableau de bord sur Node-RED et Power BI et comment réaliser une maintenance prédictive à l'aide de MATLAB.

Le but de ce troisième laboratoire est de configurer un accès à distance du tableau de bord réalisé dans le précédent laboratoire avec Node-RED.

Le second objectif de ce laboratoire est de réaliser un second tableau de bord à l'aide de Power BI.

Enfin, ce laboratoire vise à réaliser une modélisation basée sur les données et estimer une durée de vie restante à l'aide d'un réseau de neurones convolutionnels avec MATLAB

Ce laboratoire s'inscrit donc dans la continuité du précédent laboratoire. Il nous apprend à utiliser les connaissances précédemment acquises pour accéder à l'ancien tableau de bord, ou pour en réaliser un autre, et nous apprend à maîtriser différents logiciels utiles pour la modélisation de données et l'estimation de durée de vie utile.

# Démarche de conception

## Affichage du tableau de bord Node-RED à distance

Durant cette manipulation, nous devons afficher nous connecter au tableau de bord réalisé dans le précédent laboratoire à l'aide de l'application Remote-RED.

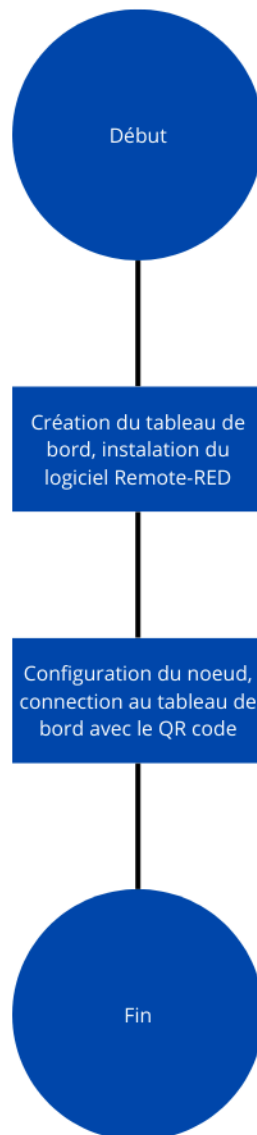


Figure 1 : Organigramme pour se connecter au tableau de bord avec l'application Remote-RED

D'après la Figure 1, dans le nœud, nous commençons par créer notre tableau de bord. Ensuite nous installons le logiciel Remote-RED sur notre appareil mobile. Enfin, nous configurons le nœud pour nous connecter au tableau de bord créé et nous nous connectons au tableau de bord avec notre appareil mobile avec le QR code affiché.

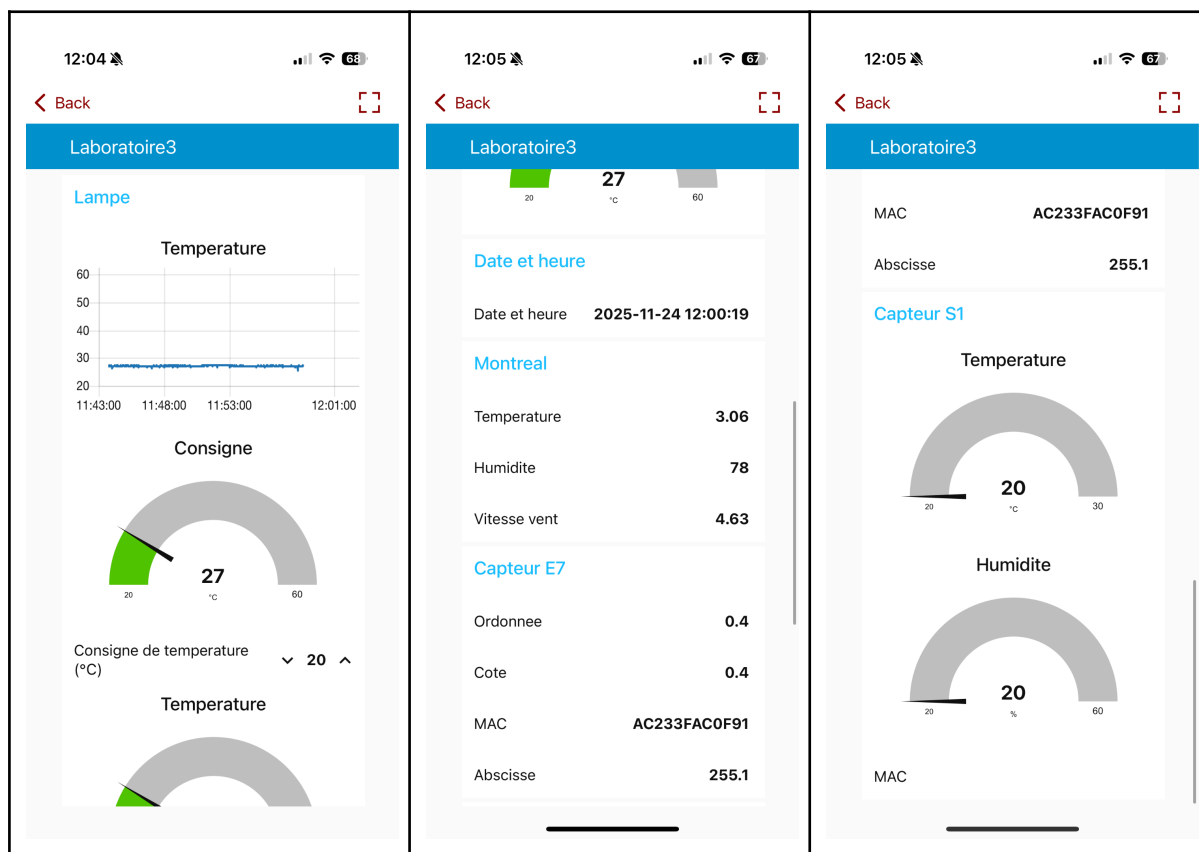


Figure 2 : Captures d'écran du tableau de bord sur Remote-RED

Nous pouvons observer, sur la figure 2, le tableau de bord effectué dans le précédent laboratoire mais cette fois, sur Remote-RED avec les données de la lampe, c'est-à-dire : sa température au cours des 15 dernières minutes, sa consigne actuelle et sa température actuelle, la date et l'heure actuelle, les données de la ville de Montréal avec la température, l'humidité et la vitesse de vent actuelles. On peut également observer les données du capteur E7 avec ses accélérations selon l'axe des abscisses, l'axe des ordonnées et l'axe des côtes et son adresse MAC et les données du capteur S1 avec sa température, son humidité et son adresse MAC. La capture d'écran a été effectuée pendant la séance du 24 novembre durant laquelle les capteurs S1 étaient éteints. Ainsi, aucune donnée du capteur S1 ne pouvait être recueillie, ce qui explique la nullité des données du capteur S1 affichées sur le tableau de bord.

Il n'y avait pas de difficultés dans cette partie, il suffisait de bien configurer le nœud avec le bon serveur, le bon port et le bon URL.

## Affichage catégorique des données avec Power BI

### Les limitations avec Excel

Durant cette manipulation, nous devons afficher la température d'un certain capteur durant une certaine journée et l'humidité d'un autre capteur un autre jour avec Excel.

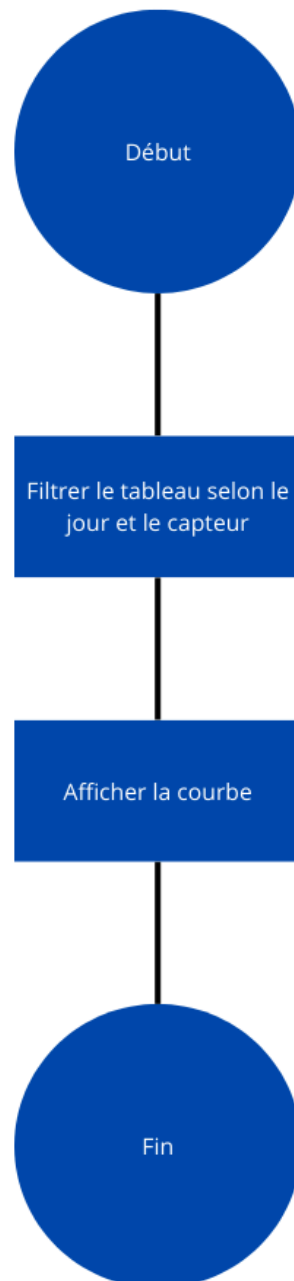


Figure 3 : Organigramme de l'affichage de données d'un capteur sur Excel

D'après la Figure 3, nous commençons par filtrer le tableau selon le jour et selon le nom du capteur. Puis, nous affichons la courbe des données sélectionnées.

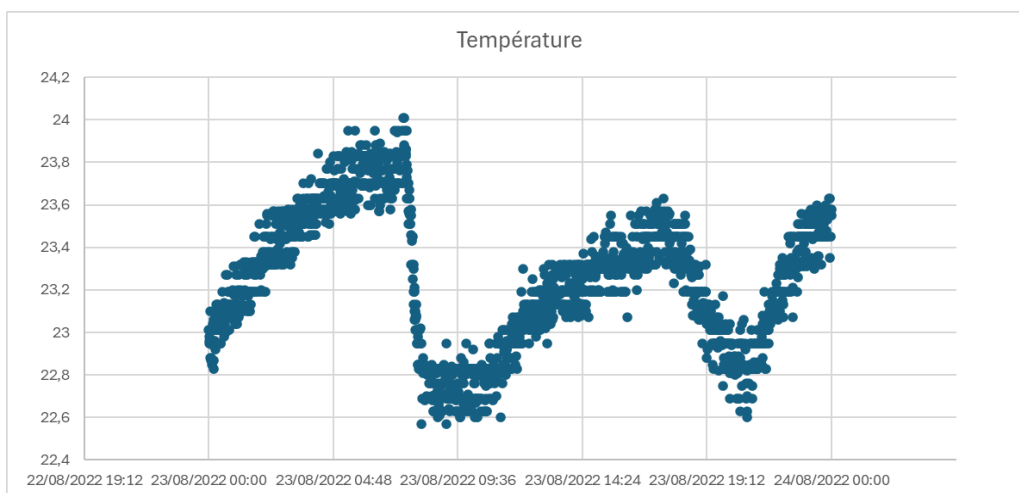


Figure 4 : Graphique présentant l'évolution de la température perçue par le capteur 494208004C67 en fonction du temps la journée du 23/08/2022

Nous observons sur la figure 4, l'évolution de la température perçue par le capteur 494208004C67 en fonction du temps la journée du 23/08/2022.

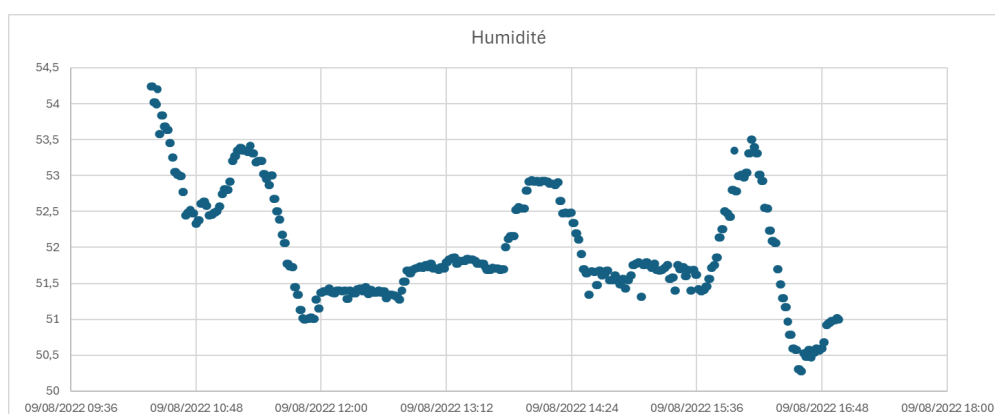


Figure 5 : Graphique présentant l'évolution de l'humidité perçue par le capteur AC233FA21248 en fonction du temps la journée du 09/08/2022

Nous observons sur la figure 5, l'évolution de l'humidité perçue par le capteur AC233FA21248 en fonction du temps la journée du 09/08/2022.

Il n'y avait pas de difficultés dans cette partie, il suffisait de filtrer le tableau pour réaliser les courbes sur les données définies.

Cependant, un problème persiste. En effet, il est impossible d'afficher les figures des autres capteurs pour les différentes journées en temps réel en un seul clic, le logiciel Excel ne permet pas de tracer les différentes données à cause des filtres qui sont appliqués sur le tableau.

## Les avantages de Power BI

Durant cette manipulation, nous devons créer un tableau de bord qui permet d'obtenir le graphique de la température, de l'humidité en fonction des salles et du capteur qui a enregistré les données.

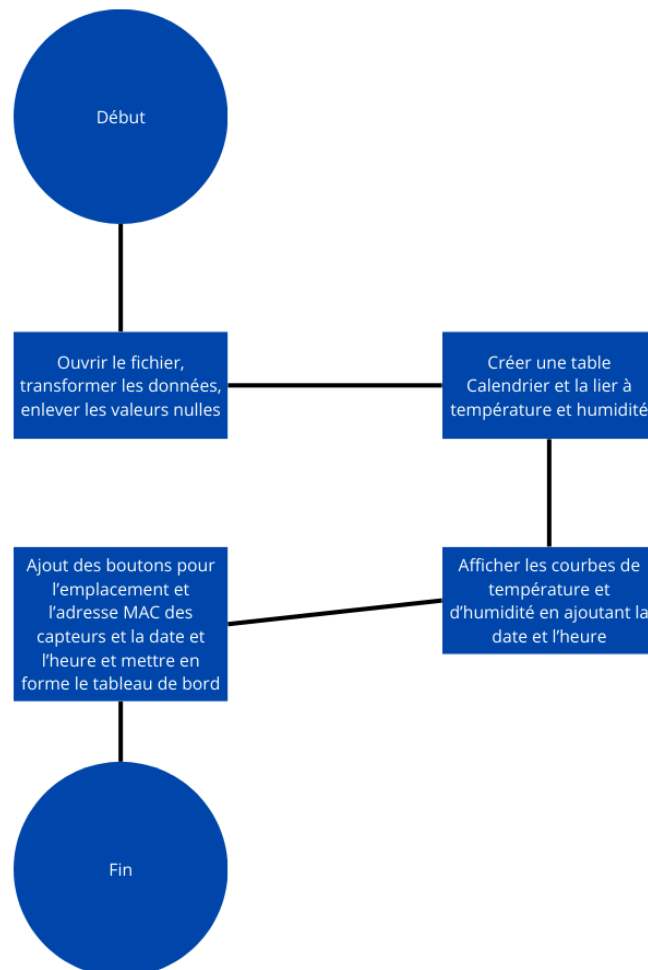


Figure 6 : Organigramme pour créer un tableau de bord sur Power BI

D'après la figure 6, nous commençons par ouvrir le fichier, puis, nous transformons les données et nous enlevons les valeurs nulles. Ensuite, nous créons une table Calendrier avec la date et l'heure et nous l'associons aux données température et humidité. De plus, nous affichons les courbes de la température et de l'humidité tout en affichant l'heure et la date auxquelles ils ont été capturés. Enfin, nous créons les boutons pour l'emplacement et l'adresse MAC des capteurs afin d'obtenir la date et l'heure de la capture des données. Il ne reste plus qu'à mettre en forme le tableau de bord.

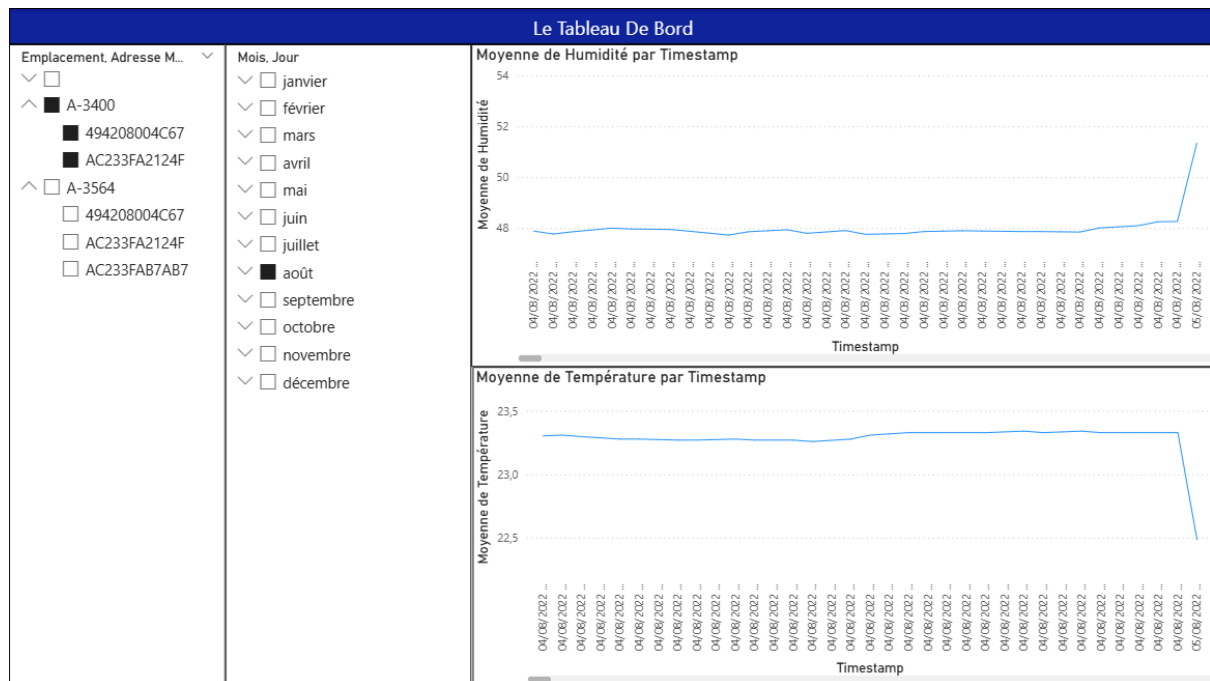


Figure 7 : Capture d'écran du tableau de bord sur Power BI

Nous observons sur la figure 7, le tableau de bord sur Power BI avec les boutons de la salle et de l'adresse MAC des capteurs et les boutons du mois, de la date et l'heure de la capture des données.

La difficulté dans cette partie était de prendre en main le logiciel Power BI. En effet, nous devons apprendre à transformer des données, à créer une table et l'associer à des données. Nous devons également apprendre à tracer des courbes et créer des boutons permettant d'afficher différentes données.

## Modélisation basée sur les données

Durant cette manipulation, nous devons créer une modélisation basée sur les données de 40 piles lithium-ion pour lesquelles nous souhaitons connaître la durée de vie restante en la prédisant par un modèle entraîné.

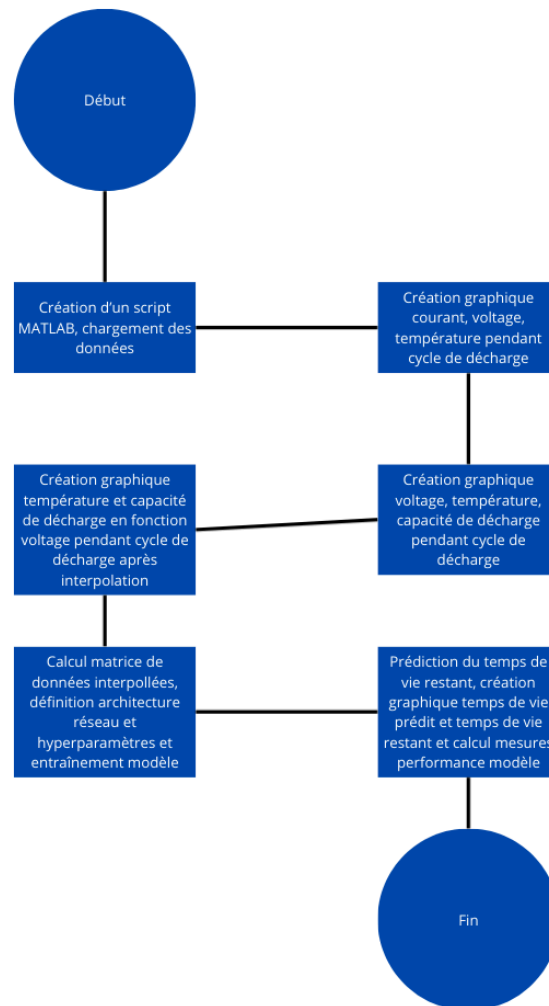


Figure 8 : Organigramme du programme MATLAB pour une modélisation basée sur les données

D'après la figure 8, nous commençons par créer un script MATLAB et nous chargeons les données. Ensuite nous créons trois graphiques : un présentant l'évolution du courant, du voltage et de la température pendant un cycle de décharge ; un autre présentant l'évolution du voltage, de la température et de la capacité de décharge pendant un cycle de décharge ; et un dernier présentant l'évolution de la température et de la capacité de décharge en fonction du voltage pendant un cycle de décharge et après une interpolation. Ensuite, nous calculons la matrice de données interpolées et nous définissons les hyperparamètres ainsi que l'architecture réseau avant d'entraîner le modèle avec l'optimiseur Adam. Enfin, nous calculons les prédictions de temps de vie restant et nous créons un graphique présentant le temps de vie prédit contre le temps de vie restant. Nous calculons ensuite les mesures de performances du modèle.

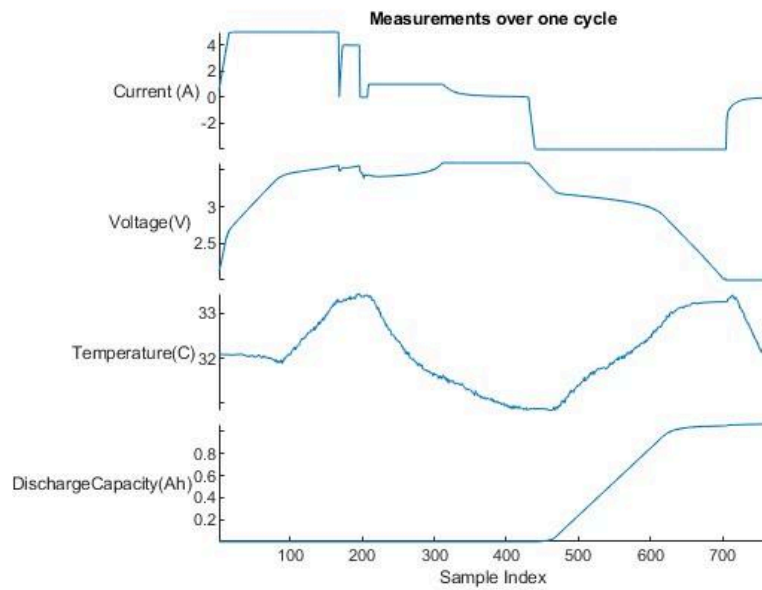


Figure 9 : Graphique représentant le courant, le voltage et les températures de mesure à travers un cycle de décharge pour la première pile

Nous observons sur la figure 9, le courant, le voltage et les températures de mesure à travers un cycle de décharge pour la première pile.

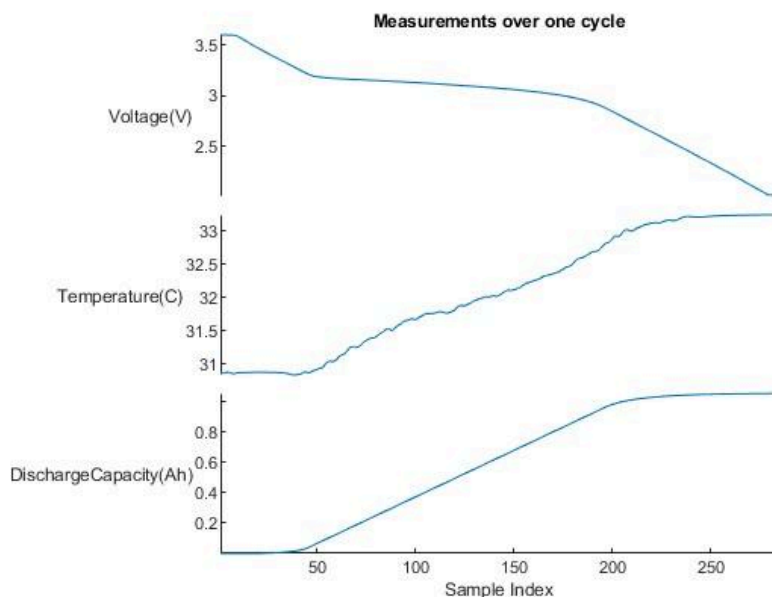


Figure 10 : Graphique représentant le voltage, la température de mesure et la capacité de décharge à travers un cycle de décharge pour la première pile

Nous observons sur la figure 10, le voltage, la température de mesure et la capacité de décharge à travers un cycle de décharge pour la première pile.

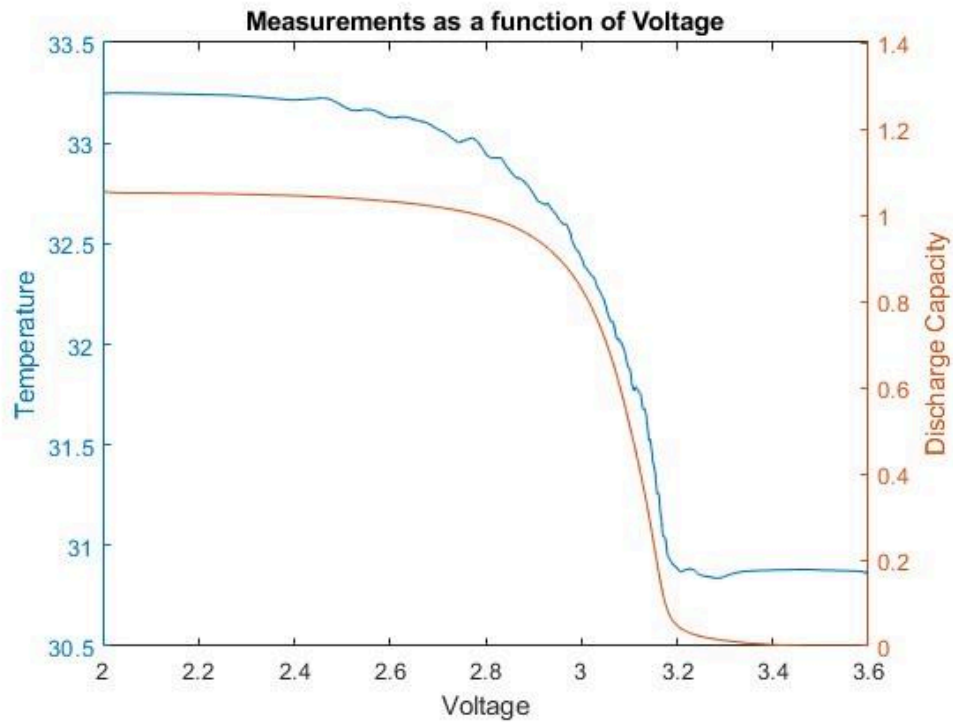


Figure 11 : Graphique représentant l'évolution de la température et de la capacité de décharge d'une pile en fonction de son voltage après une interpolation

Nous observons sur la figure 11, l'évolution de la température et de la capacité de décharge d'une pile en fonction de son voltage après une interpolation.

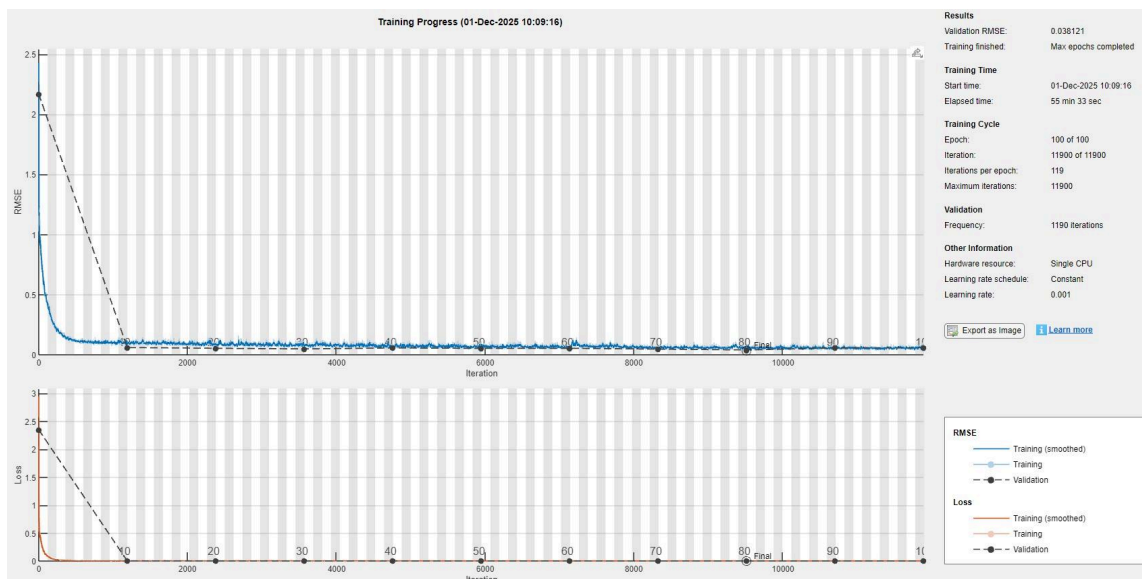


Figure 12 : Graphique représentant l'entraînement du réseau de neurones

Nous observons sur la figure 12, l'entraînement du réseau de neurones avec l'optimiseur Adam.

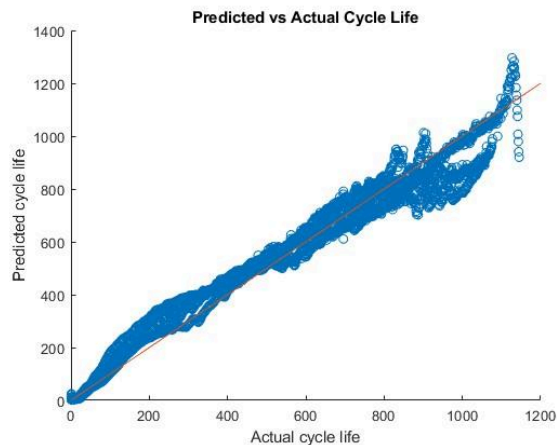


Figure 13 : Graphique représentant la durée de vie restante réelle contre la durée de vie restante prédite

Nous observons sur la figure 13, la durée de vie restante réelle contre la durée de vie restante prédite.

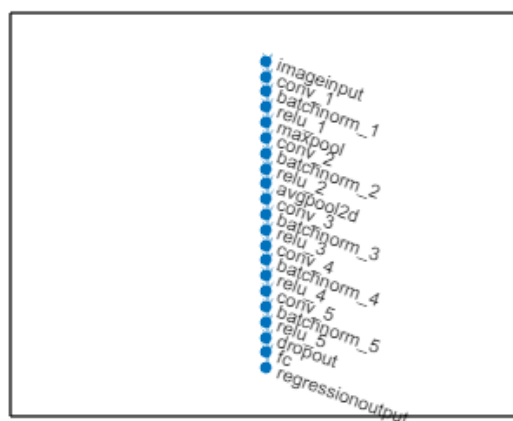


Figure 14 : Architecture du réseau

Nous observons sur la figure 14, l'architecture du réseau.

```
rmseTestModel = single
62.8907
errVal = single
15.9816
```

Figure 15 : Mesures de performances du modèle

Nous observons sur la figure 15, les mesures de performance du modèle.

Il n'y a pas eu de difficultés d'exécution dans cette partie, il suffisait de lancer les morceaux de code donnés. La difficulté était de comprendre ces morceaux de codes fournis.

## Estimation de la durée de vie utile avec réseau de neurones

Durant cette manipulation, nous devons prédire la vie utile restante des moteurs en utilisant les réseaux convolutionnels neuraux.

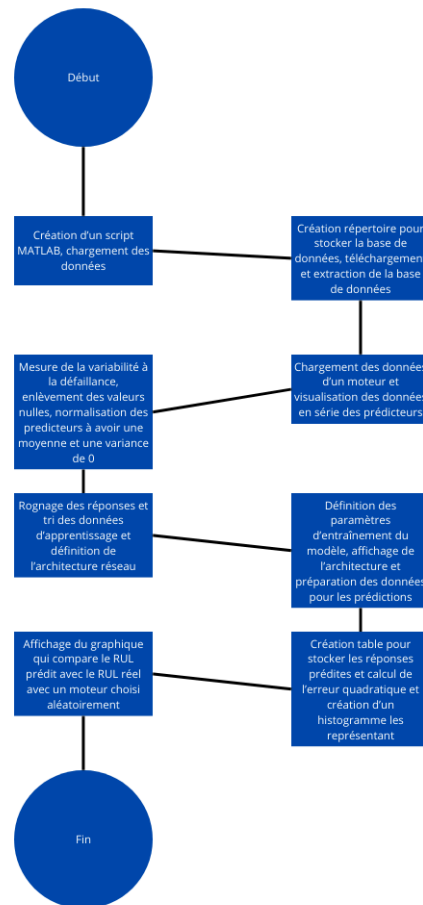


Figure 16 : Organigramme du programme MATLAB pour une estimation de la durée de vie utile de moteurs

D'après la figure 16, nous commençons par créer un script MATLAB et nous chargeons les données. Puis, nous créons un répertoire pour stocker la base de données avant de la télécharger et d'extraire ses données. Ensuite, nous chargeons les données d'un moteur et nous visualisons les données des prédicteurs en série. Puis, nous mesurons la variabilité à la défaillance, nous enlevons les valeurs nulles et nous normalisons les prédicteurs pour avoir une moyenne et une variance de 0. Ensuite, nous rognons les réponses, nous trions les données d'apprentissage et nous définissons l'architecture réseau. Puis, nous définissons les paramètres d'entraînement du modèle, nous affichons l'architecture et nous préparons les données pour la prédiction. Enfin, nous créons une table pour stocker les valeurs prédites ainsi que pour calculer les erreurs quadratiques avant de créer un histogramme les représentant. Nous affichons le graphique qui compare le RUL prédit avec le RUL réel pour un moteur choisi aléatoirement.

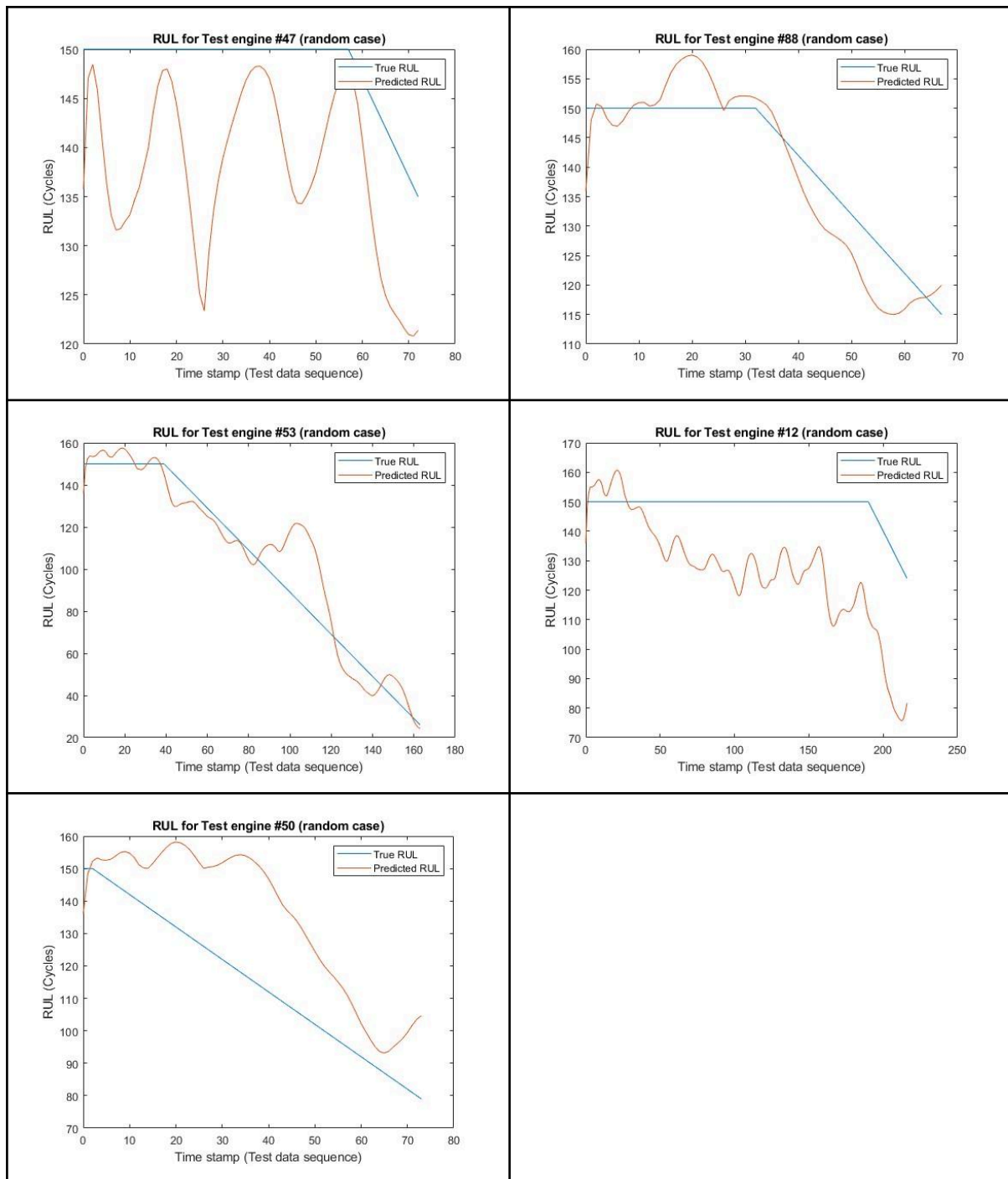


Figure 17 : Captures d'écran de graphiques de tests aléatoires de prédiction de durée de vie des moteurs

Nous observons sur la figure 17, le temps de vie utile restant des moteurs en fonction du temps.

Il n'y a pas eu de difficultés d'exécution dans cette partie, il suffisait de lancer les morceaux de code donnés. La difficulté était de comprendre ces derniers. Il suffisait également d'insérer une boucle pour générer 5 tests aléatoires.

# Conclusion

Ce laboratoire nous a permis de poursuivre la conception de notre premier tableau de bord. Nous avons appris à accéder à distance à notre tableau de bord avec l'application Remote-RED.

De plus, nous avons appris à réaliser un nouveau tableau de bord présentant de manière interactive les données de température et d'humidité de différents capteurs dans différentes pièces et selon plusieurs jours.

Ensuite, nous avons appris à modéliser des données à l'aide du logiciel MATLAB.

Enfin, nous avons appris à estimer la durée de vie utile restante à l'aide d'un réseau de neurones convolutionnels avec MATLAB.

Nous avons appris à utiliser les logiciels NodeRED, Power BI et MATLAB pour réaliser les différentes tâches du laboratoire.

## Expérience personnelle

Je pense que ce laboratoire est bien agencé. Il permet une meilleure maîtrise du logiciel NodeRED pour avoir un accès à distance du tableau de bord réalisé dans le précédent laboratoire. De plus, il permet de maîtriser les logiciels Power BI pour réaliser un nouveau tableau de bord et MATLAB pour la modélisation basée et la maintenance prédictive.

Ce laboratoire m'a permis de découvrir le logiciel Power BI pour réaliser un tableau de bord présentant des données de manière interactive permettant de trier les données que nous souhaitons montrer selon le jour, l'heure, le type de capteur et la salle où les données ont été récupérées. Enfin, ce laboratoire m'a donné la possibilité de comprendre une autre manière d'utiliser MATLAB avec la modélisation basée et la maintenance prédictive.

En conclusion, je dirais que ce laboratoire est pertinent dans la mesure où il permet de montrer l'intérêt des connaissances acquises précédemment et de les utiliser dans un projet concret. Selon moi, c'est un quatrième laboratoire intéressant pour mobiliser ses connaissances, et ainsi réaliser des tâches plus complexes que celles effectuées dans le dernier laboratoire, tout en utilisant les programmes précédemment implémentés. De plus, il permet de maîtriser deux nouveaux logiciels qui sont Power BI et MATLAB, même si nous n'avons qu'une faible interaction avec MATLAB.

Je pense qu'il aurait été intéressant de réaliser une introduction à MATLAB pour voir ses capacités et nous permettre de nous familiariser avec ce logiciel. A titre personnel, je l'avais précédemment utilisé. Il m'était donc mais je peux comprendre qu'une personne novice ait des difficultés à l'utiliser.

```

url =
'https://ssd.mathworks.com/supportfiles/predmaint/batterycyclelifeprediction/v2/batteryDischargeData.zip';

websave('batteryDischargeData.zip',url);

unzip('batteryDischargeData.zip')

load('batteryDischargeData');

```

```

battIndx = 1; cycleIndx = 1;

batteryMeasurements =
table(batteryDischargeData(battIndx).cycles(cycleIndx).I,batteryDischargeData(battIndx).cycles(cycleIndx).V,...
      batteryDischargeData(battIndx).cycles(cycleIndx).T,
batteryDischargeData(battIndx).cycles(cycleIndx).Qd);

stackedplot(batteryMeasurements, "Title","Measurements over one cycle",...
    "DisplayLabels", ["Current (A)","Voltage(V)","Temperature(C)","DischargeCapacity(Ah)],...
    ...
    "Xlabel", "Sample Index");

```

```

dischargeData = hExtractDischargeData(batteryDischargeData);

```

```

batteryMeasurements =
table(dischargeData{battIndx}.Vd{cycleIndx},dischargeData{battIndx}.Td{cycleIndx},...
      dischargeData{battIndx}.QdClipped{cycleIndx});

stackedplot(batteryMeasurements, "Title","Measurements over one cycle",...
    "DisplayLabels", ["Voltage(V)","Temperature(C)","DischargeCapacity(Ah)],...
    "Xlabel", "Sample Index");

```

```

[VInterpol,TInterpol,QdInterpol] = hLinearInterpolation(dischargeData);

```

```

figure

yyaxis left

plot(reshape(VInterpol{1}{1}, 900, 1),reshape(TInterpol{1}{1},900,1))

```

```

title('Measurements as a function of Voltage')
ylabel('Temperature')
xlabel('Voltage')
yyaxis right
plot(reshape(VInterpol{1}{1},900,1),reshape(QdInterpol{1}{1},900,1))
ylabel('Discharge Capacity')

```

```

testBatteryIndex = 2:8:40;
valBatteryIndex = 1:8:40;
trainBatteryIndex = setdiff(1:40,[2:8:40 1:8:40]);

```

```

[trainData,trainRuIData] = hreshapeData(VInterpol(trainBatteryIndex), ...
    TInterpol(trainBatteryIndex),QdInterpol(trainBatteryIndex));
[valData,ValRuIData] = hreshapeData(VInterpol(valBatteryIndex), ...
    TInterpol(valBatteryIndex),QdInterpol(valBatteryIndex));
[testData,testRuIData] = hreshapeData(VInterpol(testBatteryIndex), ...
    TInterpol(testBatteryIndex),QdInterpol(testBatteryIndex));

```

```

fprintf('Size of reshaped matrix of interpolated measurement data:%dx%dx%dx%d\n', ...
    size(trainData))

```

```

layers = [
    imageInputLayer([30 30 3],'Normalization','rescale-zero-one')
    convolution2dLayer(3,8,'Padding','same')
    batchNormalizationLayer
    reluLayer
    maxPooling2dLayer(2,'Stride',2)
    convolution2dLayer(3,16,'Padding','same')
    batchNormalizationLayer
    reluLayer

```

```

averagePooling2dLayer(2,'Stride',2)
convolution2dLayer(3,32,'Padding','same')
batchNormalizationLayer
reluLayer
convolution2dLayer(3,32,'Padding','same')
batchNormalizationLayer
reluLayer
convolution2dLayer(3,32,'Padding','same')
batchNormalizationLayer
reluLayer
dropoutLayer(0.5)
fullyConnectedLayer(1)
regressionLayer];
figure
plot(layerGraph(layers))

```

```

miniBatchSize = 256;
validationFrequency = 10*floor(numel(trainRuIData)/miniBatchSize);
options = trainingOptions("adam", ...
    "MaxEpochs",100, ...
    "MiniBatchSize",miniBatchSize, ...
    "Plots","training-progress", ...
    "Verbose",false, ...
    "Shuffle","every-epoch", ...
    "InitialLearnRate",0.001, ...
    "OutputNetwork","best-validation-loss", ...
    "ValidationData",{valData, ValRuIData}, ...
    "ValidationFrequency",validationFrequency, ...
    "ValidationPatience",10, ...
    "ResetInputNormalization",false);

```

```
rng("default")
```

```
batteryNet = trainNetwork(trainData, trainRulData, layers, options);
```

```
yPredTest = predict(batteryNet, testData)*2000;
```

```
testRulScaled = testRulData*2000;
```

```
figure;
```

```
scatter(testRulScaled, yPredTest)
```

```
hold on;
```

```
refline(1,0);
```

```
title("Predicted vs Actual Cycle Life")
```

```
ylabel("Predicted cycle life");
```

```
xlabel("Actual cycle life");
```

```
errTest = (yPredTest - testRulScaled);
```

```
rmseTestModel = sqrt(mean(errTest.^2))
```

```
n = numel(testRulScaled);
```

```
nr = abs(testRulScaled - yPredTest);
```

```
errVal = (1/n)*sum(nr./testRulScaled)*100
```

## Annexe 1 : Code MATLAB pour la modélisation basée sur les données

### Annexe 2

```
dataFolder = "data";
```

```
if ~exist(dataFolder, 'dir')
```

```
    mkdir(dataFolder);
```

```
end
```

```

filename =
matlab.internal.examples.downloadSupportFile("nnet","data/TurbofanEngineDegradationSimulationData.zip");
unzip(filename,dataFolder)

```

```

filenameTrainPredictors = fullfile(dataFolder,"train_FD001.txt");
rawTrain = localLoadData(filenameTrainPredictors);

```

```

head(rawTrain.X{1},8)

```

```

rawTrain.Y{1}(1:8)

```

```

stackedplot(rawTrain.X{1},[3,5,6,7,8,15,16,24],XVariable='timeStamp')

```

```

prog = prognosability(rawTrain.X,"timeStamp");

```

```

idxToRemove = prog.Variables==0 | isnan(prog.Variables);
featToRetain = prog.Properties.VariableNames(~idxToRemove);
for i = 1:height(rawTrain)
    rawTrain.X{i} = rawTrain.X{i}{:,featToRetain};
end

```

```

[~,Xmu,Xsigma] = zscore(vertcat(rawTrain.X{:}));
preTrain = table();
for i = 1:numel(rawTrain.X)
    preTrain.X{i} = (rawTrain.X{i} - Xmu) ./ Xsigma;
end

```

```

rulThreshold = 150;
for i = 1:numel(rawTrain.Y)
    preTrain.Y{i} = min(rawTrain.Y{i},rulThreshold);
end

```

```
end
```

```
for i = 1:size(preTrain,1)
    preTrain.X{i} = preTrain.X{i}';    %Transpose training data to have features in the first
dimension
    preTrain.Y{i} = preTrain.Y{i}';    %Transpose responses corresponding to the training
data
    sequence = preTrain.X{i};
    sequenceLengths(i) = size(sequence,2);
end
```

```
[sequenceLengths,idx] = sort(sequenceLengths,'descend');
XTrain = preTrain.X(idx);
YTrain = preTrain.Y(idx);
```

```
numFeatures = size(XTrain{1},1);
numHiddenUnits = 100;
numResponses = 1;
```

```
layers = [
    sequenceInputLayer(numFeatures)
    convolution1dLayer(5,32,Padding="causal")
    batchNormalizationLayer()
    reluLayer()
    convolution1dLayer(7,64,Padding="causal")
    batchNormalizationLayer
    reluLayer()
    convolution1dLayer(11,128,Padding="causal")
    batchNormalizationLayer
    reluLayer()
```

```

convolution1dLayer(13,256,Padding="causal")
batchNormalizationLayer
reluLayer()
convolution1dLayer(15,512,Padding="causal")
batchNormalizationLayer
reluLayer()
fullyConnectedLayer(numHiddenUnits)
reluLayer()
dropoutLayer(0.5)
fullyConnectedLayer(numResponses)
regressionLayer()];

```

```

maxEpochs = 40;
miniBatchSize = 16;

```

```

options = trainingOptions('adam',...
    LearnRateSchedule='piecewise',...
    MaxEpochs=maxEpochs,...
    MiniBatchSize=miniBatchSize,...
    InitialLearnRate=0.01,...
    GradientThreshold=1,...
    Shuffle='never',...
    Plots='training-progress',...
    Verbose=0);

```

```

net = trainNetwork(XTrain,YTrain,layers,options);

```

```

figure;
lgraph = layerGraph(net.Layers);
plot(lgraph)

```

```
filenameTestPredictors = fullfile(dataFolder,'test_FD001.txt');
filenameTestResponses = fullfile(dataFolder,'RUL_FD001.txt');
dataTest = localLoadData(filenameTestPredictors,filenameTestResponses);
```

```
for i = 1:numel(dataTest.X)
    dataTest.X{i} = dataTest.X{i}{:,featToRetain};
    dataTest.X{i} = (dataTest.X{i} - Xmu) ./ Xsigma;
    dataTest.Y{i} = min(dataTest.Y{i},rulThreshold);
end
```

```
predictions = table(Size=[height(dataTest) 2],VariableTypes=["cell","cell"],VariableNames=["Y","YPred"]);
```

```
for i=1:height(dataTest)
    unit = dataTest.X{i}';
    predictions.Y{i} = dataTest.Y{i}';
    predictions.YPred{i} = predict(net,unit,MiniBatchSize=1);
end
```

```
for i = 1:size(predictions,1)
    predictions.RMSE(i) = sqrt(mean((predictions.Y{i} - predictions.YPred{i}).^2));
end
```

```
figure;
histogram(predictions.RMSE,NumBins=10);
title("RMSE ( Mean: " + round(mean(predictions.RMSE),2) + " , StDev: " + round(std(predictions.RMSE),2) + " )");
ylabel('Frequency');
xlabel('RMSE');
```

```
for i = 1:5  
    figure;  
    localLambdaPlot(predictions,"random");  
end
```

Annexe 2 : Code MATLAB pour l'estimation de la durée de vie utile avec un réseau de neurones